

Math in ML

Mundarija

ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)	4
1-Dars: Vektorlar va Matritsalar (Vectors and Matrices)	4
1-QISM: Vektor (Vector) - Koinotdagi Nuqta va Raqamlar Ro'yxati	5
2-QISM: Vektor Amallari (Vector Operations)	5
3-QISM: Vektor Normasi (Vector Norm) - Uzunlikni O'lchash	6
4-QISM: Skalyar Ko'paytma (Dot Product) - E'tibor va O'xshashlik	7
5-QISM: Matritsa (Matrix) - Vektorlar Jamlanmasi	8
6-QISM: Matritsaning Shakli (Shape) va Indeksash	8
7-QISM: Matritsa Turlari (Matrix Types)	9
8-QISM: Matritsa Amallari va Transponirlash (Transpose)	9
9-QISM: Matritsalarini Ko'paytirish (Matrix Multiplication) - Asosiy Mexanizm!	10
10-QISM: NumPy da Vektorlar va Matritsalar (Python amaliyoti)	11
Xulosa	12
ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)	13
2-Dars: Chiziqli Kombinatsiya, Fazo Qamrovi (Span) va Chiziqli Mustaqillik	13
1-QISM: Chiziqli Kombinatsiya (Linear Combination) - Yangi Koinot Yaratish	13
2-QISM: Fazo Qamrovi (Span) - Imkoniyatlar Chegarasi	14
3-QISM: Chiziqli Qaramlik (Linear Dependence) - Keraksiz Takrorlanish	15
4-QISM: Chiziqli Mustaqillik (Linear Independence) - Sof Erkinlik	16
5-QISM: Bu tushunchalarning Machine Learning dagi O'rni (The ML Connection)	17
6-QISM: Baza (Basis) - Mukammal Fundamental To'plam	18
7-QISM: O'lcham (Dimension) - Haqiqiy Murakkablik	18
Xulosa	19
ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)	20
3-Dars: Muhim Matritsa Turlari, Amallar va Chiziqli Tenglamalar Sistemasi Yechish	20
1-QISM: Kvadrat Matritsa (Square Matrix) - Mukammal O'lcham	20
2-QISM: Diagonal Matritsa (Diagonal Matrix) - Sof O'lchov	21
3-QISM: Birlik Matritsa (Identity Matrix) - Matritsalar Olamidagi "1" Raqami	22
4-QISM: Transponirlash (Transpose) - Qatorlar va Ustunlar O'rnini Almashtirish	22
5-QISM: Matritsalarini Ko'paytirish (Matrix Multiplication) - Mantiqni Chuqurlashtiramiz	23

6-QISM: Teskari Matritsa (Inverse Matrix) - Vaqtni Orqaga Qaytarish.....	24
7-QISM: Chiziqli Tenglamalar Sistemasi Yechish (Teskari Matritsaning G'alabasi).....	26
Xulosa.....	28
ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)	28
4-Dars: Determinant, Xos Qiymatlar (Eigenvalues) va Xos Vektorlar (Eigenvectors)	28
1-QISM: Determinant - Kengayish yoki Qisqarish O'lchovi.....	28
2-QISM: Manfiy (Negative) va Nol (Zero) Determinantlar.....	29
3-QISM: Xos Vektorlar (Eigenvectors) va Xos Qiymatlar (Eigenvalues) - Fazoning Ustunlari	30
4-QISM: Matematik Tenglama va Yechim Mantiqi.....	31
5-QISM: Python'da Eigenvalues va Eigenvectors (NumPy mo'jizasi)	32
6-QISM: Data Science uchun eng katta Sovg'a - PCA (Principal Component Analysis).....	33
Xulosa.....	34
ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)	35
5-Dars: Vektor Transformatsiyalari va Rasmlar (Vector Transformations and Images)	35
1-QISM: Chiziqli Transformatsiya nima? (Linear Transformation)	35
2-QISM: Transformatsiyaning Matritsali Ifodasi (Matrix Representation)	36
3-QISM: 4 ta Asosiy Transformatsiya Turlari (The Big Four)	37
4-QISM: Transformatsiyalarni Birlashtirish (Composition of Transformations)	38
5-QISM: Rasm - bu Matritsa va Piksel Qiymatlari (Image as a Matrix).....	39
6-QISM: Image Augmentation (Rasmlarni Ko'paytirish san'ati).....	39
7-QISM: ML va Computer Visionning Yakuniy Bog'liqligi (CNN)	40
Xulosa.....	41
ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)	42
6-Dars: Limitlar va Uzluksezlik (Limits and Continuity).....	42
1-QISM: Limit nima o'zi? (The Concept of a Limit)	42
2-QISM: Bir Tomonlama Limitlar (One-sided Limits).....	43
3-QISM: Cheksiz Limitlar va Asimptotalar (Infinite Limits)	44
4-QISM: Uzluksezlik (Continuity)	46
5-QISM: Uzilish Turlari (Types of Discontinuity).....	47
6-QISM: Limit va Hosila (Derivative) ning Bog'liqligi.....	48
7-QISM: ML Intuition (Machine Learningda ularning nima aloqasi bor?).....	49
Xulosa (Conclusion)	51

ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)	51
7-Dars: Hosila va Gradient (Derivatives and Gradient)	51
1-QISM: Hosila Nima? O'zgarish Tezligi (Rate of Change)	52
2-QISM: Hosilaning Geometrik Ma'nosi (Slope va Tangent Line)	53
3-QISM: Hosila Olish Qoidolari (Derivative Rules)	54
4-QISM: Pythonda SymPy bilan Hosila Olish (Derivative with SymPy)	56
5-QISM: Xususiy Hosilalar (Partial Derivatives) - Koinotni Kengaytiramiz	57
6-QISM: Gradient Vektori (Gradient Vector - ∇f)	58
7-QISM: Gradient Yo'nalishi (Gradient Direction - Mantiqiy cho'qqi)	59
8-QISM: ML Intuition va Gradient Descent Algoritmi	60
Xulosa	61
ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)	61
8-Dars: Hosilaning Amaliyoti - Xatolik Funktsiyasi va Optimizatsiya (Cost Function and Optimization)	61
1-QISM: Biz Qanday Muammoni Yechyapmiz o'zi? (Optimizatsiya Asoslari)	62
2-QISM: Cost Function va MSE (O'rtacha Kvadratik Xatolik)	62
3-QISM: Zanjir Qoidasi Amalda (MSE ni Hosila bilan yechamiz)	63
4-QISM: Nega Oddiy Yo'l Bilan Yecha Olmaymiz?	64
5-QISM: Gradient Descent Qanday Ishlaydi? (Step-by-Step)	65
6-QISM: Gradient Descentning Uch Xil Jangavor Turi (Batch, Stochastic, Mini-Batch)	66
7-QISM: O'rganish Tezligi (Learning Rate - α) va Uning Sirlari	66
8-QISM: Local Minimum va Global Minimum (Yashirin Qopqonlar)	67
9-QISM: Asosiy Xulosa (Nega buni bilish muhim?)	68
ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)	68
9-Dars: Integral va Egri Chiziq Ostidagi Yuza (Integral and Area Under the Curve)	68
1-QISM: O'zi Nima U Integral? (Asosiy Mantiq)	69
2-QISM: Riman Yig'indisi (Riemann Sum) - Kompyuter qanday hisoblaydi?	70
3-QISM: Hosila va Integral (Antiderivative) Ning Yagona Olamligi	70
4-QISM: Aniq Integral (Definite Integral) va Uni Hisoblash	71
5-QISM: Raqamli Integrallash (Numerical Integration) va Pythonda Yechish	73
6-QISM: ML va Ehtimollik Nazariyasidagi Integrallarning Ahamiyati	74
Xulosa (Conclusion)	76
ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)	76
10-Dars: Ko'p O'zgaruvchili Funktsiyalar (Multivariable Functions)	76
1-QISM: Ko'p O'zgaruvchili Funktsiya o'zi nima? (Multivariable Function)	77

2-QISM: Vizualizatsiya - Biz Ularni Qanday Ko'ramiz? (Contour and Surface Plots)	77
3-QISM: Xususiy Hosilalar (Partial Derivatives) Mantiqiy Qiyalikni Topish	78
4-QISM: Gradient Vektori (∇f) Barcha kuchlarni Bir O'qqa (Vektorga) Birlashtirish	79
5-QISM: Ikki Parametrli Gradient Descent (Two-Parameter Gradient Descent)	80
6-QISM: Bir nechta o'lchamli koinotda optimallashtirish (Optimization in Multiple Dimensions) Sirlari	81
Xulosa (Conclusion)	82
ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)	83
11-Dars: Zanjir Qoidasi va Orqaga Tarqalish (Chain Rule and Backpropagation)	83
1-QISM: Murakkab (Ichma-ich) Funksiyalar (Composite Functions)	83
2-QISM: Zanjir Qoidasi o'zi nima? (The Chain Rule)	84
3-QISM: Ko'p O'zgaruvchili Zanjir Qoidasi (Multivariable Chain Rule)	85
4-QISM: Computational Graph (Hisoblash Grafikasi - Orqadagi Sehrli Vizualizatsiya)	86
5-QISM: Backpropagation (Orqaga Tarqalish) - The Algorithm!	87
6-QISM: Local va Global Minimum (Optimizatsiya G'alati Olamda va ML Model xatoliklarsizligi)	89
Xulosa (Conclusion)	90
ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)	90
12-Dars: Mini Loyiha - End-to-End Optimizatsiya (Gradient Descent in Action)	91
Loyiha Senariysi: Biz Nima Quryapmiz?	91
1-QADAM: Kutubxonalarni Yuklash (The Arsenal)	91
2-QADAM: Simvolik Kalkulyus (SymPy bilan Cost Function Hosilasi)	92
3-QADAM: Data (Ma'lumotlar) va Raqamli Funktsiyalar (NumPy)	92
4-QADAM: Gradient Descent (Oltin Algoritmni ishga tushirish)	93
5-QADAM: Koinotni Vizualizatsiya Qilish (3D Surface va Cost Contour Plot)	95
Mantiqiy Yakun va Buyuk Xulosa (The Ultimate Conclusion)	97

ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)

1-Dars: Vektorlar va Matritsalar (Vectors and Matrices)

Assalomu alaykum, bo'lajak Sun'iy Intellect muhandisi! Biz siz bilan mashinali o'rganishning (ayniqsa Deep Learning va Neyron tarmoqlarining) "Yuragi" bo'lmish Chiziqli Algebra olamiga qadam qo'ymoqdamiz.

Agar Statistika va Ehtimollar fani AI ga "Mantiq va Xulosa" qilishni o'rgatsa, Chiziqli Algebra unga "Tezlik va Quddat" beradi. Tasavvur qiling, sizda bitta uyning narxini hisoblash uchun 10 xil matematik amalni bajarish kerak. Agar sizda 1 millionta uy bo'lsa, siz **for** tsikli (loop) yordamida har biriga bittadan kirib chiqsangiz, kompyuteringiz bunga soatlar sarflaydi. Ammo, agar shu 1 million uyni bitta "Matritsa" degan qutiga solib, bitta "Vektor"ga ko'paytirsangiz, kompyuter buni GPU (videokarta) yordamida bir soniyaning mingdan bir bo'lagida bajarib tashlaydi!

Chiziqli algebra — bu raqamlarni guruhlab, ular bilan ommaviy tarzda ishlash san'ati. Bugun biz ushbu san'atning eng asosiy ikki qahramoni: **Vektorlar** va **Matritsalar** bilan tanishamiz. Qahvangizni hozirlang, tasavvuringizni kengaytiring, biz boshladik!

1-QISM: Vektor (Vector) - Koinotdagi Nuqta va Raqamlar Ro'yxati

"Vektor" so'zini eshitganda, maktab fizika darsi esingizga tushishi mumkin: "Vektor — bu yo'nalish va uzunlikka ega bo'lgan kesma (strelka)". Va bu mutlaqo to'g'ri. Lekin Computer Science (Kompyuter ilmi) va Machine Learningda vektorga biroz boshqacharoq, ancha amaliyroq ta'rif beriladi.

ML da Vektor nima? Vektor — bu shunchaki **bir o'lchamli raqamlar ro'yxati (massiv)**. U ma'lum bir obyektning xususiyatlarini (features) o'zida saqlaydi.

Tasavvur qiling, biz bitta uyni tahlil qilyapmiz. Uyning maydoni 120 kv.m, xonalar soni 4 ta, yoshi esa 15 yil. Biz bu uyni ML modeliga berish uchun uni qavslar ichiga olib, ustun ko'rinishida yozamiz:

$$\vec{x} = \begin{bmatrix} 120 \\ 4 \\ 15 \end{bmatrix}$$

Mana shu narsa **Vektor** deb ataladi! Odatda vektorlar kichik harflar bilan ustida chiziqchasi ($\vec{x}$) yoki qalin harflar (x) bilan belgilanadi. Bu vektorning ichida 3 ta raqam bo'lgani uchun biz uni **3 o'lchamli (3D) vektor** deymiz. Uni matematikada $\vec{x} \in R^3$ deb yozishadi. Ya'ni haqiqiy sonlardan iborat uch o'lchamli fazodagi bitta nuqta.

Agar siz yuzni taniydigan (Face Recognition) model qursangiz va rasm 100×100 pikseldan iborat bo'lsa, sizning rasmingiz bitta uzun vektorga aylantiriladi va uning o'lchami 10,000 ta raqamdan iborat bo'ladi! Bu inson miyasi tasavvur qila olmaydigan 10,000 o'lchamli fazodagi yagona bitta nuqta deganidir.

2-QISM: Vektor Amallari (Vector Operations)

Vektorlar bilan ishlash juda mantiqiy va oson. Biz bitta raqam (skalyar) bilan qanday amallar bajarsak, vektorlar bilan ham shunday qilamiz, faqat "element-by-element" (ma'nosi: har bir qavat o'zi bilan) tartibida.

A. Vektorlarni qo'shish va ayirish

Siz faqatgina **o'lchami bir xil** bo'lgan vektorlarni qo'shishingiz yoki ayirishingiz mumkin

(ikkalasi ham 3D yoki ikkalasi ham 5D bo'lishi shart). Agar $\vec{a} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}$ va $\vec{b} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$ bo'lsa, ularning yig'indisi:

$$\vec{a} + \vec{b} = \begin{bmatrix} 2+3 \\ 5+1 \end{bmatrix} = \begin{bmatrix} 5 \\ 6 \end{bmatrix} \quad \vec{a} + \vec{b} = \begin{bmatrix} 2+3 \\ 5+1 \end{bmatrix} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

Geometrik ma'nosi: Tasavvur qiling, $\vec{a}$ shamolning tezligi va yo'nalishi, $\vec{b}$ esa samolyotning motori berayotgan tezlik. Samolyot havoda qayerga uchadi? Aynan shu ikki vektorning yig'indisi ($\vec{a} + \vec{b}$) ko'rsatgan yangi diagonal yo'nalishga qarab uchadi!

B. Vektorni Skalyarga (Oddiy songa) ko'paytirish

Skalyar — bu yonalishsiz, oddiygina yagona raqam. Vektorni skalyarga ko'paytirish uning yo'nalishini o'zgartirmaydi, faqatgina uning **uzunligini chuzadi yoki qisqartiradi**. Agar skalyar $c = 3$ bo'lsa:

$$3 \cdot \begin{bmatrix} 2 \\ 5 \end{bmatrix} = \begin{bmatrix} 3 \cdot 2 \\ 3 \cdot 5 \end{bmatrix} = \begin{bmatrix} 6 \\ 15 \end{bmatrix} \quad 3 \cdot \begin{bmatrix} 2 \\ 5 \end{bmatrix} = \begin{bmatrix} 3 \cdot 2 \\ 3 \cdot 5 \end{bmatrix} = \begin{bmatrix} 6 \\ 15 \end{bmatrix}$$

Agar siz vektorni manfiy songa (masalan, -1 ga) ko'paytirsangiz, u 180 gradusga ortga o'giriladi (yo'nalishi mutlaqo teskari bo'ladi).

3-QISM: Vektor Normasi (Vector Norm) - Uzunlikni O'lchash

Machine Learningda ko'pincha "Bu ikkita nuqta bir-biridan qanchalik uzoqda?" yoki "Mening modelimning xatosi qanchalik katta bo'lib ketdi?" degan savollarga javob berish kerak bo'ladi. Vektorning haqiqiy uzunligini (hajmini) o'lchaydigan tushuncha — **Norma (Norm)** deb ataladi.

Eng mashhur va eng ko'p ishlatiladigan norma bu **L2 Norma (yoki Evklid Normasi)** dir. Uni $\|\vec{v}\|_2$ deb belgilanadi. Maktabdagi Pifagor teoremasini ($c^2 = a^2 + b^2$) eslaysizmi? L2 Norma aynan shuning n-o'lchamli koinot uchun kengaytirilgan variantidir!

$$\vec{v} = \begin{bmatrix} 3 \\ 4 \\ 0 \end{bmatrix}$$

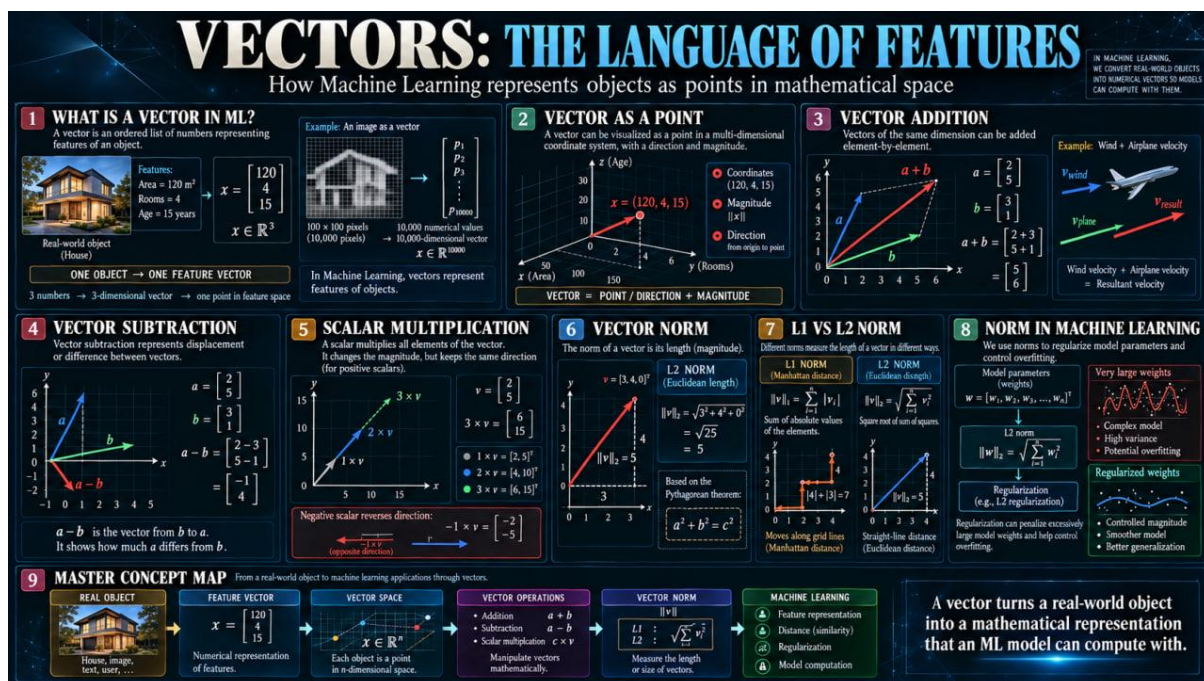
Agar bizda uch o'lchamli $\begin{bmatrix} 3 \\ 4 \\ 0 \end{bmatrix}$ vektor bo'lsa, uning uchi koordinata boshidan (0,0,0) qancha masofada joylashgan? Formulasi — har bir elementni kvadratga ko'tarib qo'shamiz va ildizdan chiqaramiz:

$$\|\vec{v}\|_2 = \sqrt{3^2 + 4^2 + 0^2} = \sqrt{9 + 16 + 0} = \sqrt{25} = 5$$

$$\|\vec{v}\|_2 = \sqrt{3^2 + 4^2 + 0^2} = \sqrt{9 + 16 + 0} = \sqrt{25} = 5$$

Demak, bu vektorning "uzunligi" 5 ga teng. Deep Learning'da biz ko'pincha modeldagi millionlab o'zgaruvchilarning "L2 Normasi" haddan tashqari kattalashib ketmasligini nazorat qilamiz (bunga Regularization deyiladi), aks holda model yodlab olishga (Overfitting) o'tib ketadi.

Shuningdek L1 Norma (Manhattan distance) ham bor, unda biz kvadratga ko'tarmaymiz, shunchaki barcha raqamlarning moduli (absolyut qiymatini) bir-biriga qo'shib chiqamiz.



4-QISM: Skalyar Ko'paytma (Dot Product) - E'tibor va O'xshashlik

Mana biz Chiziqli algebraning eng buyuk va ML dagi eng muhim operatsiyalaridan biriga keldik. Bu — **Dot Product (Skalyar Ko'paytma)** dir. Nega u "Skalyar" ko'paytma deyiladi? Chunki siz ikkita qop (vektor) ni bir-biriga urishtirasiz (ko'paytirasiz), lekin javob sifatida qop emas, balki bitta yakka **Raqam (Skalyar)** tushadi!

Matematik Hisoblash: Siz faqat o'lchamlari bir xil bo'lgan vektorlarni Dot Product qila olasiz. Bunda birinchi element birinchisiga, ikkinchisi ikkinchisiga ko'paytiriladi, va chiqqan barcha

javoblar bir-biriga qo'shib yuboriladi. Misol uchun:

$$\vec{a} = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix} \quad \vec{b} = \begin{bmatrix} 1 \\ 0 \\ 5 \end{bmatrix}$$

va

$$\vec{a} \cdot \vec{b} = (2 \cdot 1) + (3 \cdot 0) + (4 \cdot 5) = 2 + 0 + 20 = 22$$

Javob: 22. Bitta oddiy raqam.

Geometrik va ML ma'nosi (Cosine Similarity): Xo'sh, bu 22 raqami nimani anglatadi? Dot Product, aslini olganda, ikkita vektorning **bir-biriga qanchalik o'xshashligini (bir yo'nalishga qarab ketayotganini)** o'lchaydi! Geometriyada Dot Productning yana bir formulasi bor:

$$\vec{a} \cdot \vec{b} = \|\vec{a}\| \|\vec{b}\| \cos(\theta) \quad \vec{a} \cdot \vec{b} = \|\vec{a}\| \|\vec{b}\| \cos(\theta)$$

Bu yerdagi θ (Teta) — ikkita vektor orasidagi burchak.

- Agar ikkita vektor butunlay bir tomonga, bir xil parallel yo'nalishda qaragan bo'lsa (burchak 0°), ularning kosinusi eng yuqori bo'ladi, natijada Dot Product juda katta musbat son chiqadi (O'xshashlik **100%**).

2. Agar ikkita vektor bir-biriga to'g'ri burchak ostida (perpendikulyar, burchak 90°) bo'lsa, kosinus nolga teng bo'ladi. Dot Product = 0. (Bu "Ular bir-biriga umuman daxli yo'q, butunlay mustaqil" degani).
3. Agar ular mutlaqo qarama-qarshi tomonga qarab tursa, Dot Product manfiy chiqadi.

Tavsiya Tizimlari (Recommendation Systems) qanday ishlaydi? Siz YouTube'da yoki Netflix'da kino ko'ryapsiz. Sizning nima yoqtirishingiz bitta uzun vektorga aylanadi. Bazadagi har bir kinoning ham o'z vektori bor. YouTube sizning vektoringizni bazadagi millionlab kinolarning vektorlari bilan birma-bir **Dot Product** qilib chiqadi. Qaysi kinoda chiqqan Skalyar ko'paytma (Kosinus o'xshashligi) eng katta chiqsa, u tushunadiki "Bu kinoning vektori va foydalanuvchining vektori bir xil yo'nalishga qarabtypti, o'xshash!". Va sizga aynan shu kinoni tavsiya qiladi. Hammasi ortida mana shu oddiy formula yotibdi!

5-QISM: Matritsa (Matrix) - Vektorlar Jamlanmasi

Endi tasavvurni yana bir pog'ona oshiramiz. Bitta obyekt (bitta uyni) raqamlar ro'yxati qildik, bu vektor. Agar bizda 1000 ta shunday uy bo'lsachi? Ularning 1000 ta vektorini nima qilamiz? Biz ularni yonma-yon qilib, bitta katta to'rtburchak qutiga, jadvalga joylashtiramiz. Mana shu to'rtburchak raqamlar jadvali **Matritsa (Matrix)** deb ataladi!

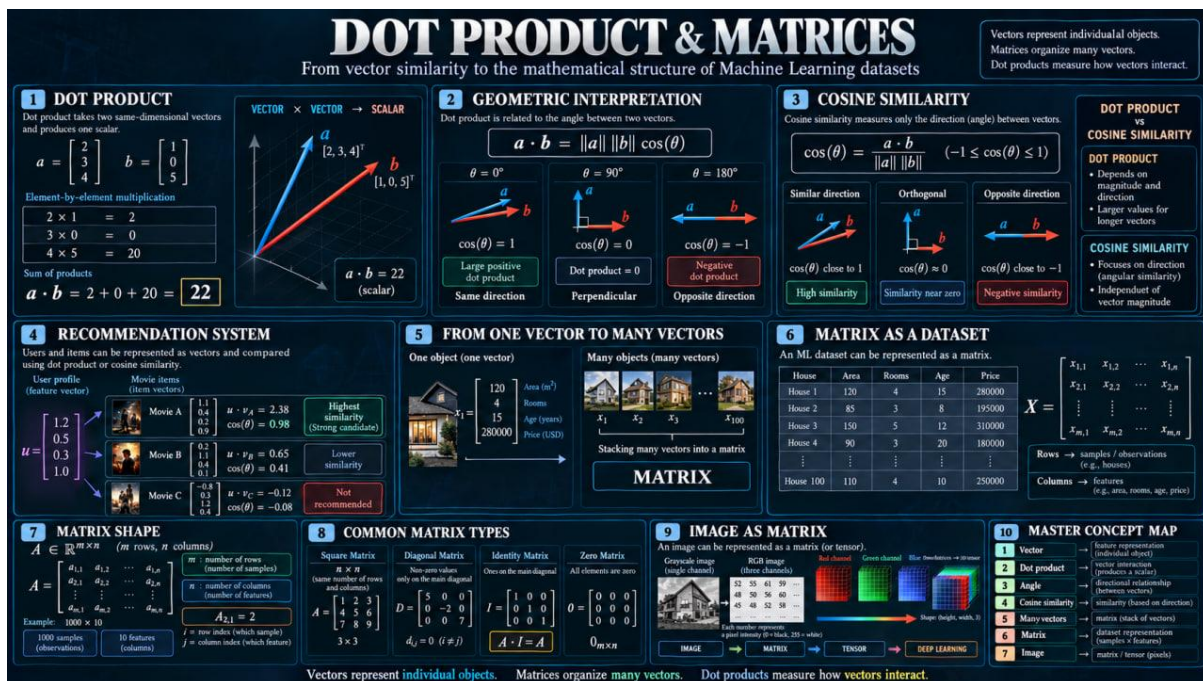
Matritsa odatda bosh harflar bilan (masalan, A , B , X) belgilanadi. ML da sizning butun boshli Dataset'ingiz (masalan CSV fayl) bu bitta ulkan X matritsasidir. Rasmlar (Images) ham aslida matritsa hisoblanadi. Oq-qora rasm — bu shunchaki piksel yorqinliklaridan (0 dan 255 gacha bo'lgan raqamlardan) iborat katta shaxmat doskasi (matritsa). Rangli (RGB) rasmlar esa uchta shunday matritsaning ustma-ust qatlami (Tensor) hisoblanadi.

$$A = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$$

6-QISM: Matritsaning Shakli (Shape) va Indeksflash

Matritsaning eng muhim xususiyati bu uning shakli, o'lchamidir. U har doim (**Qatorlar soni**, **Ustunlar soni**) tarzida, odatda $m \times n$ (m ga n) deb aytiladi. Yuqoridagi A matritsaning shakli 3×3 (uch qator va uch ustun). Agar menda 1000 ta mijoz va har bir mijozning 10 xil anketasi bo'lsa, mening ma'lumotlar matritsam 1000×10 shaklda bo'ladi.

Indeksflash: Matritsa ichidagi aniq bir raqamni chaqirib olish. Matematikada raqamning qavati va xonasi yoziladi. $A_{i,j}$ (bu yerda i - qator (row), j - ustun (column)). Yuqoridagi matritsada: $A_{2,1} = 2$ (Ikkinchi qator, birinchi ustundagi raqam). $A_{1,3} = 7$ (Birinchi qator, uchinchi ustundagi raqam). (*Dasturlashda ko'pincha raqamlash 0 dan boshlanishini unutmang, lekin sof matematikada u 1 dan boshlanadi*).



7-QISM: Matritsa Turlari (Matrix Types)

Lineer algebrada biz tez-tez uchratadigan va nomini bilishimiz shart bo'lgan maxsus matritsalar bor:

- Kvadrat Matritsa (Square Matrix):** Qatorlar va ustunlar soni teng bo'lgan matritsa ($n \times n$). Yuqoridagi misolimiz aynan kvadrat matritsadir (3×3).
- Diagonal Matritsa (Diagonal Matrix):** Bu faqatgina "bosh diagonalida" (chap tepadan o'ng pastga qiya chiziqda) raqamlari bor, qolgan barcha burchaklari aynan Nol (0) ga teng bo'lgan kvadrat matritsa.

$$D = \begin{bmatrix} 5 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 8 \end{bmatrix}$$

- Birlik Matritsa (Identity Matrix - I):** Bu shunday maxsus diagonal matritsaki, uning diagonalidagi barcha sonlar roppa-rosa 1 ga teng. Birlik matritsaning mo'jizasi nimada? U oddiy arifmetikadagi "1" sonining vazifasini bajaradi! Oddiy matematikada qanday sonni 1 ga ko'paytirsangiz yana o'zi chiqqani kabi, har qanday matritsani I ga ko'paytirsangiz hech narsa o'zgarmaydi, o'zi qaytib chiqadi ($A \cdot I = A$).

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Nol Matritsa (Zero Matrix):** Barcha elementlari 0 bo'lgan matritsa. Oddiy 0 sonidek harakat qiladi.

8-QISM: Matritsa Amallari va Transponirlash (Transpose)

Matritsalarini qo'shish va skalyarga ko'paytirish xuddi vektorlardagidek juda oddiy ishlaydi. Ikkita bir xil shakldagi matritsani qo'shsangiz, ularning mos katakchadagi raqamlari bir-biriga qo'shiladi.

Ammo, alohida diqqat qilish kerak bo'lgan, ML da juda ko'p ishlatiladigan bitta muhim xususiyat bor. Bu **Transponirlash (Transpose - A^T)** dir.

Tasavvur qiling, sizning matritsangiz to'rtburchak lego bo'lagi. Transponirlash bu uni 90° gradusga burib qo'yish degani. Aniqrog'i, Transponirlash jarayonida **barcha Qatorlar Ustunlarga aylanadi, barcha Ustunlar esa Qatorlarga aylanadi!**

Agar Asl matritsa shakli 3×2 bo'lsa:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

Uning Transponirlangan varianti (A^T) ning shakli teskarisiga, 2×3 bo'lib qoladi:

$$A^T = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix} A^T = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$$

Machine Learningda biz ko'pincha bitta matritsani ikkinchisiga ko'paytirish uchun o'lchamlari mos kelmay qolganda (buni keyingi qismda ko'ramiz), ulardan birining qaddini to'g'rilash uchun darhol Transpose (A^T) dan foydalanamiz.

9-QISM: Matritsalarini Ko'paytirish (Matrix Multiplication) - Asosiy Mexanizm!

Butun Neural Networks (Neyron Tarmoqlari), Transformers (ChatGPT) kabi sun'iy intellekt arxitekturalarining yagona va eng ko'p bajaradigan ishi — aynan bitta matritsani boshqasiga ko'paytirishdir. Agar buni yaxshi tushunib olsangiz, siz modelning ichida nima bo'layotganini seza boshlaysiz.

Diqqat: Matritsalarini ko'paytirish oddiy sonlarni ko'paytirishdek (masalan, $2 \times 3 = 3 \times 2$) qoidaga bo'ysunmaydi. Matritsalarda $A \cdot B \neq B \cdot A$! Ba'zan ularning o'rnini almashtirsangiz umuman hisoblab bo'lmay qoladi.

O'lcham Qoidasi (The Dimension Rule - Eng Muhimi!): Ikkita matritsani (A va B) bir-biriga ko'paytirish uchun, qat'iy oltin qoida bor: **Birinchi matritsaning Ustunlar soni, Ikkinchi matritsaning Qatorlar soniga roppa-rosa Teng bo'lishi SHART!** Ya'ni: Agar A matritsasi ($m \times n$) o'lchamda bo'lsa, B matritsasi albatta ($n \times p$) o'lchamda bo'lishi kerak. O'rtadagi ikkita n harfi bir xil bo'lishi shart! Ular bir-birini qisqartirib yuboradi. Yangi hosil bo'lgan C matritsasining shakli esa chekkada qolgan o'lchamlardan iborat bo'ladi: ($m \times p$).

Misol: Agar siz rasm (100×64) ni neyron tarmog'idagi qatlamga (64×10) berib ko'paytirsangiz, sizning yangi natijangiz shakli qanday bo'ladi? Javob: O'rtadagi 64 lar mos tushdi (demak ishlaydi), yangi qatlam natijasi 100×10 shaklda chiqadi! Bu MLdagi ma'lumotlarning "siqilishi" (Dimensionality reduction) deb ataladi.

Qanday hisoblanadi? (Row by Column Rule) Keling, ikkita kichik matritsani ko'paytirib

ko'ramiz. $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ va $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$ Ularni ko'paytirib, $C = A \cdot B$ ni yaratamiz. Mantiq bunday: **Birinchi matritsadan doim faqat yotiq qator (row) ni olasiz, Ikkinchi**

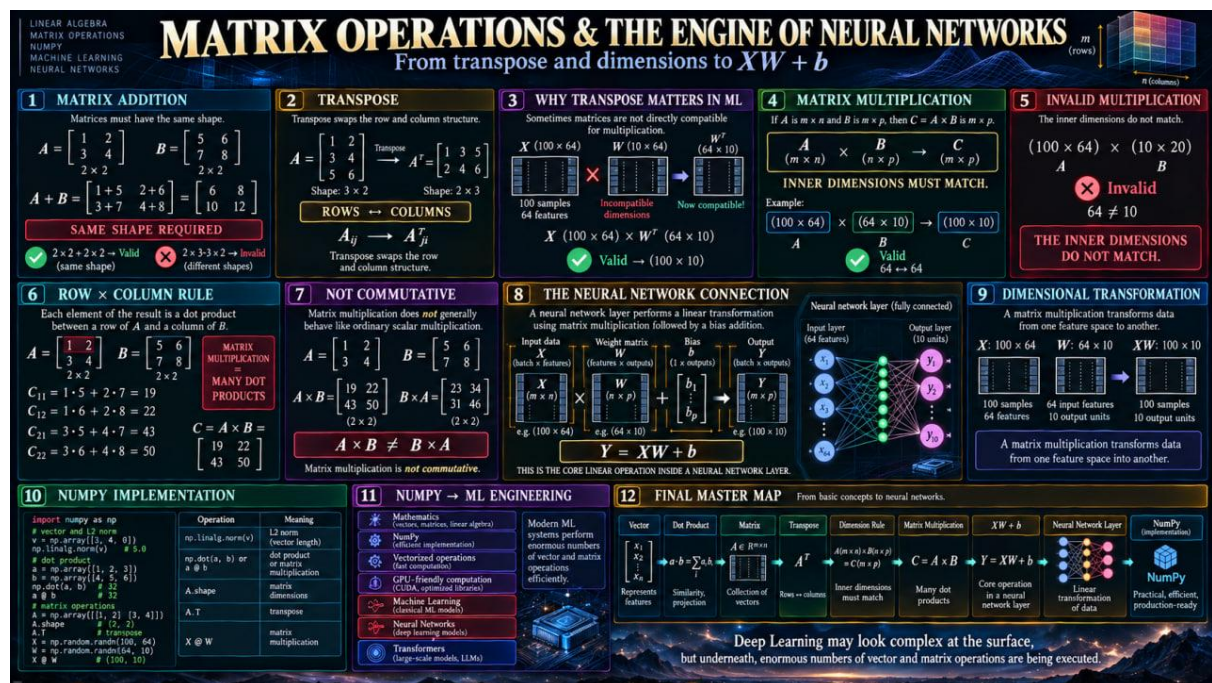
matritsadan esa har doim faqat tikka ustun (column) ni olasiz va ularni Dot Product (Skalyar ko'paytma) qilasz!

- $C_{1,1}$ (Birinci katakcha javobi) ni topish uchun: A ning 1-qatorini ($\begin{bmatrix} 1 & 2 \end{bmatrix}$) B ning 1-ustuniga ($\begin{bmatrix} 5 \\ 7 \end{bmatrix}$) dot product qilamiz. $C_{1,1} = (1 \cdot 5) + (2 \cdot 7) = 5 + 14 = 19$.
- $C_{1,2}$ (Birinci qator, ikkinchi ustun) ni topish uchun: A ning 1-qatorini, B ning 2-ustuniga dot product qilamiz. $C_{1,2} = (1 \cdot 6) + (2 \cdot 8) = 6 + 16 = 22$.
- Qolganlarini ham xuddi shunday davom ettiramiz: $C_{2,1} = (3 \cdot 5) + (4 \cdot 7) = 15 + 28 = 43$. $C_{2,2} = (3 \cdot 6) + (4 \cdot 8) = 18 + 32 = 50$.

Demak, yakuniy javobimiz:

$$C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Ko'rib turganingizdek, matritsa ko'paytmasi bu aslida son-sanoqsiz yashirin "Dot Product"lar fabrikasidir! Neyron tarmog'idagi bitta qatlamning ishlash mantiqi: $Y = X \cdot W + b$ (Kiruvchi ma'lumot X ko'paytirilgan Og'irliklar matritsasi W plus Bias vektor b). Barcha hisob-kitob aynan shu yerda biz qilib ko'rsatgan usulda ketadi.



Yaxshi xabar shundaki, siz yuz minglab nuqtali matritsalarini qog'ozda qo'lda hisoblamaysiz. Buni Pythonda NumPy kutubxonasi yashin tezligida bajaradi. Keling, o'rgangan bilimlarimizni kodga ko'chiramiz.

```
import numpy as np
```

```
# 1. Vektor va uning normasi
v = np.array([3, 4, 0])
```

```
norm_v = np.linalg.norm(v) # L2 normani hisoblash
print("Vektor:", v)
print("Norma (Uzunlik):", norm_v) # Javob: 5.0
```

```
# 2. Skalyar ko'paytma (Dot Product)
a = np.array([2, 3, 4])
b = np.array([1, 0, 5])
dot_result = np.dot(a, b) # yoki shunchaki a @ b
print("Dot Product:", dot_result) # Javob: 22
```

```
# 3. Matritsa yaratish va Shakli (Shape)
A = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
print("Matritsa shakli:", A.shape) # Javob: (2, 3) - 2 qator, 3 ustun
```

```
# 4. Transponirlash (Transpose)
A_T = A.T
print("Transponirlangan:\n", A_T)
print("Yangi shakli:", A_T.shape) # Javob: (3, 2)
```

```
# 5. Matritsalarini Ko'paytirish (Matrix Multiplication)
X = np.array([[1, 2], [3, 4]])
W = np.array([[5, 6], [7, 8]])
```

```
# Matritsalarini ehtiyotkorlik bilan ko'paytirish uchun @ operatori ishlatiladi
Y = X @ W # yoki np.matmul(X, W) yoki np.dot(X, W)
print("Ko'paytma Natijasi:\n", Y)
# Javob:
# [[19 22]
#  [43 50]]
```

NumPy o'zining "broadcasting" (avtomatik o'lcham moslashtirish) va C-tilida yozilgan motori bilan bu operatsiyalarni minglab marta tezroq bajaradi. ML muhandisi sifatida siz **A @ B** yozuvini har kuni, har soatda ishlatasiz!

Xulosa

Tabriklayman! Bugungi dars bilan siz shunchaki oddiy arifmetika qutisidan chiqib, olamni **ko'p o'lchamli chiziqli fazolar** orqali ko'rishni boshladingiz.

Siz vektor nima ekanini va u qanday qilib yagona raqamlar orqali odamlar fe'l-atvorini, rasmlarni va ovozlarni ML modellariga etkazib berishini tushundingiz. Matritsalar esa shu o'n

minglab vektorlarni ustma-ust yig'adigan markaziy baza ekanini bildingiz. Va eng asosiysi, Dot Product va Matritsa ko'paytmasi nima uchun butun Sun'iy Intellekt olamini tebratib turgan yurak urishi ekanligini anglab yetdingiz.

Lekin bu faqat boshlanishi! Matritsalar ba'zan shunchaki "ma'lumotlar to'plami" emas, balki qandaydir mo'jizaviy funksiyani bajaruvchi, koinotni burib, qisib va aylantirib beruvchi **Transformator** (O'zgartiruvchi asbob) rolini ham bajaradi. Bu haqida biz keyingi darsimizda (2 va 3-darslarda) batafsil to'xtalamiz. Ular chiziqli algebra sirlariga yana bir yorqin oyna ochadi. Tayyorgarligingizni pasaytirmang, muvaffaqiyat tomon olg'a!

ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)

2-Dars: Chiziqli Kombinatsiya, Fazo Qamrovi (Span) va Chiziqli Mustaqillik

Assalomu alaykum, aziz va bo'lajak Sun'iy Intellekt muhandisi! O'tgan darsda biz Chiziqli algebraning asosiy g'ishtlari bo'lmish Vektorlar va Matritsalar bilan koinot qurishni boshlagan edik. Biz u yerda vektorlarni qo'shishni va skalyarga ko'paytirishni o'rgandik.

Bugun esa biz shu oddiy tuyulgan amallarning ortida yotgan ulkan falsafani ochib tashlaymiz. Agar o'tgan dars "Alifbo" bo'lgan bo'lsa, bugungi dars o'sha harflardan "So'zlar va Gaplar" tuzish san'atidir.

Siz Machine Learningda ko'p eshitasiz: "Featurelar (Xususiyatlar) bir-biriga chiziqli qaram bo'lmasligi kerak", yoki "PCA algoritmi orqali ma'lumotlar o'lchamini (Dimension) qisqartiramiz". Bu nima degani o'zi? Qanday qilib ma'lumot qaram bo'lishi mumkin? Qanday qilib o'lchamni qisqartirishadi? Shu va shunga o'xshash barcha sehrli atamalar aynan bugungi darsimizning mag'zini tashkil qiladi. Qahvangizni hozirlang, o'rindiqqa suyaning, koinot geometriyasi bo'ylab sayohatimizni boshlaymiz!

1-QISM: Chiziqli Kombinatsiya (Linear Combination) - Yangi Koinot Yaratish

Chiziqli algebrada biz faqat ikkita asosiy harakatni (amalni) "qonuniy" deb bilamiz:

1. Vektorlarni bir-biriga qo'shish ($\vec{v} + \vec{w}$)
2. Vektorni qandaydir raqamga (skalyarga) ko'paytirish ($c \cdot \vec{v}$)

Agar biz shu ikkala qonuniy amalni birlashtirsak, nima yuz beradi? Biz **Chiziqli Kombinatsiyaga** ega bo'lamiz! **Chiziqli kombinatsiya** — bu qo'lingizda mavjud bo'lgan bir nechta vektorlarni har xil miqdorda (skalyarlarga ko'paytirib) olib, ularni bir-biriga qo'shish orqali mutlaqo **yangi vektor** yaratish jarayonidir.

Matematik ta'rif: Agar bizda bir qancha $\vec{v}_1, \vec{v}_2, \dots, \vec{v}_n$ vektorlar bo'lsa va ularga mos ravishda $c_1, c_2, \dots, c_n$ oddiy raqamlar (skalyarlar - ularni "vaznlar" (weights) deb ham ataymiz) berilgan bo'lsa, ularning chiziqli kombinatsiyasi quyidagicha yoziladi:

$$\vec{v}_{yangi} = c_1\vec{v}_1 + c_2\vec{v}_2 + \dots + c_n\vec{v}_n \quad \vec{v}_{yangi} = c_1\vec{v}_1 + c_2\vec{v}_2 + \dots + c_n\vec{v}_n$$

Hayotiy va Oshxona Analogiyasi: Tasavvur qiling, siz pazandasiz va qo'lingizda ikkita asosiy masalliq (Vektor) bor: Un ($\vec{v}_1$) va Shakar ($\vec{v}_2$). Siz ulardan har xil miqdorda (Skalyar - c) olib, turli xil taomlar tayyorlashingiz mumkin:

- Agar siz $3 \cdot \text{Un} + 1 \cdot \text{Shakar}$ qilsangiz, siz Non (yangi vektor) yopasiz.
 - Agar siz $1 \cdot \text{Un} + 4 \cdot \text{Shakar}$ qilsangiz, siz Shirinlik (boshqa yangi vektor) pishirasiz.
- Demak, Non ham, Shirinlik ham — Un va Shakarning **Chiziqli kombinatsiyasi** ekan!

Raqamlardagi Misol: Faraz qilaylik, bizning 2 o'lchamli fazomizda ikkita vektor bor: $\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$

va $\vec{b} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$ Keling, shu ikki vektordan $c_1 = 2$ va $c_2 = -1$ skalyarlar yordamida chiziqli kombinatsiya yasaymiz:

$$\vec{c} = 2 \cdot \begin{bmatrix} 1 \\ 2 \end{bmatrix} + (-1) \cdot \begin{bmatrix} 0 \\ 3 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \end{bmatrix} + \begin{bmatrix} 0 \\ -3 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$\vec{c} = 2 \cdot \begin{bmatrix} 1 \\ 2 \end{bmatrix} + (-1) \cdot \begin{bmatrix} 0 \\ 3 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \end{bmatrix} + \begin{bmatrix} 0 \\ -3 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

Biz butunlay boshqa yo'nalishga qarab ketayotgan yangi $\vec{c} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ vektorini yaratdik.

Machine Learning bilan bevosita bog'liqligi: Esingizdami, Chiziqli Regressiya (Linear Regression) yoki Neyron tarmoqlari (Neural Networks) qanday ishlardi? Ular har bir kiruvchi ma'lumotni (X_1, X_2) qandaydir vaznlarga (W_1, W_2) ko'paytirib, bir-biriga qo'shib chiqadi: $y = W_1X_1 + W_2X_2 + \dots + W_nX_n$ Ko'rib turganingizdek, **Machine Learningdagi har qanday chiziqli bashorat (Prediction) — bu shunchaki sizning ma'lumotlaringizning Chiziqli Kombinatsiyasidir!** Modelning asosiy ishi — o'sha to'g'ri keladigan c (ya'ni W) raqamlarini, o'ziga xos to'g'ri retseptni topishdan iborat.

2-QISM: Fazo Qamrovi (Span) - Imkoniyatlar Chegarasi

Biz qanday qilib kombinatsiyalar yasashni o'rgandik. Endi matematiklarda shunday savol tug'iladi: *"Agar men qo'limdagi mavjud vektorlarga (masalliq'larga) Ixtiyoriy, dunyodagi barcha mumkin bo'lgan barcha skalyar raqamlarni (minus cheksizlikdan plus cheksizlikkacha) qo'yib, hisoblab chiqaversam, men o'zi fazoning qaysi nuqtalariga yetib bora olaman?"*

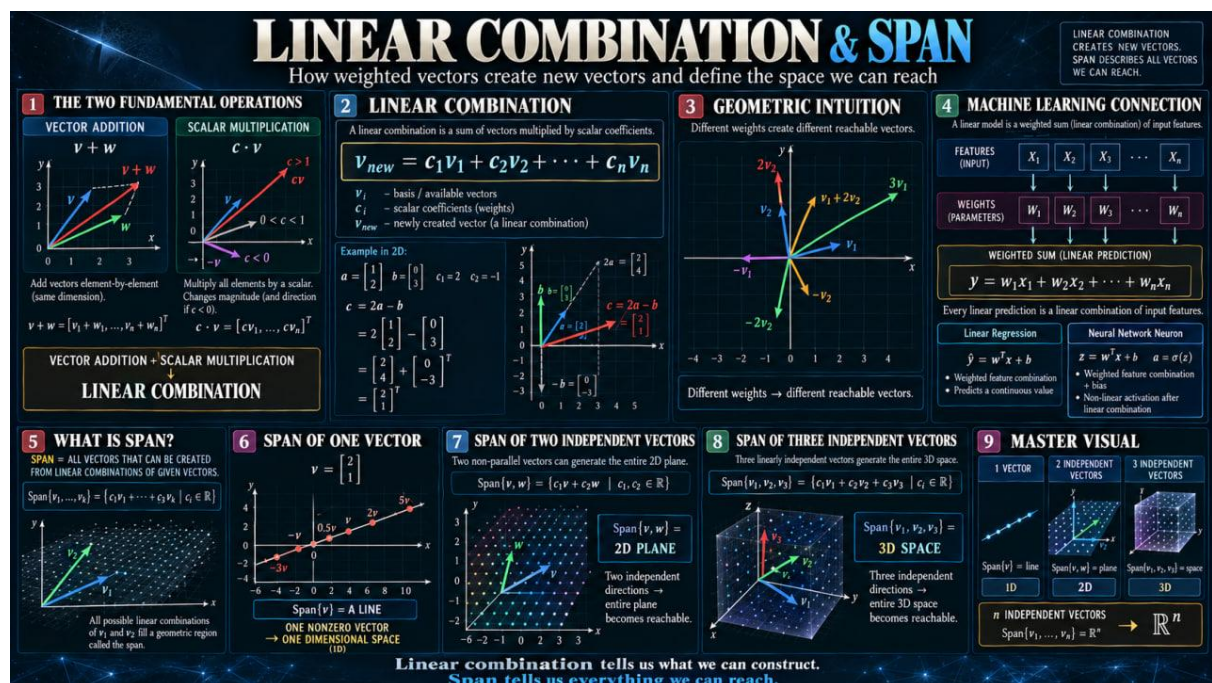
Aynan mana shu savolning javobi **Span (Fazo Qamrovi yoki Yoyilmasi)** deb ataladi.

Ta'rif: Berilgan vektorlarning Spani — bu ularning yordamida (chiziqli kombinatsiyasi orqali) yasash, yetib borish mumkin bo'lgan **BARCHA** ehtimoliy yangi vektorlar (nuqtalar) to'plamidir.

Keling, turli xil vaziyatlarni tasavvur qilamiz:

1. **Bitta vektorning Spani (Span of 1 vector):** Sizning 2D grafikingizda faqat bitta $\vec{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ vektor bor. Siz unga qanday raqam ko'paytirmang ($2\vec{v}, -5\vec{v}, 0.5\vec{v}$), u faqat o'zi yotgan chiziq bo'ylab cho'ziladi yoki teskari tomonga o'sadi, yoki nolga aylanadi. U hech qachon o'zining chizig'idan chiqib keta olmaydi (u yonga bura olmaydi). Demak, bitta nolmas vektorning Spani — bu fazodagi **Bitta To'g'ri Chiziqdır (Line)**.
2. **Ikkita vektorning Spani (Span of 2 vectors):** Endi sizda ikkita vektor bor: $\vec{v}$ va $\vec{w}$. Ular har xil tomonga qaragan (parallel emas). Agar siz ularni har xil skalyarlarga ko'paytirib qo'shsangiz, birinchisi orqali xohlagancha oldinga/orqaga yurasiz, ikkinchisi orqali xohlagancha o'ngga/chapga burilasiz. Natijada siz butun boshli, cheksiz tekislikning (qog'oz varag'i kabi) ISTALGAN nuqtasiga yetib bora olasiz! Demak, ikki o'lchamli koinotda 2 ta turli tomonga qaragan vektorning Spani — bu **Butun boshli 2D Fazo (Plane / Tekislik)** dir.
3. **Uchta vektorning Spani:** Agar uchinchi vektor $\vec{u}$ avvalgi ikkalasi yotgan qog'oz (tekislik) tekisligidan qiya qilib osmonga (yuqoriga) qarab chiqib ketgan bo'lsa, endi siz bu uchala vektor kombinatsiyasi orqali har qanday qavatga, har qanday balandlikka chiqa olasiz. Ularning Spani — **Barcha 3D Fazo (Volume)** dir.

Geometrik tushuncha: Span bu sizning qo'lingizdagi vektorlar "rul va motor" vazifasini bajarib, koinotning qanchalik katta qismini aylanib chiqa olishingiz mumkinligining o'lchovidir.



3-QISM: Chiziqli Qaramlik (Linear Dependence) - Keraksiz Takrorlanish

Ammo, bir muammo bor. Har doim ham qo'limizga yangi vektor (yangi masalliq) berilsa, bizning koinotimiz kengayib qolavermaydi! Ba'zan yangi vektor shunchaki "yolg'onchi" bo'ladi, u bizga hech qanday yangi manzil, yangi ma'lumot bermaydi.

Tasavvur qiling, siz 2D fazodasiz. Sizda bitta vektor bor: $\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$. Biz aytdik, uning Spani -

bu yagona chiziq. Kimdir kelib sizga zo'r yangilik bilan ikkinchi vektorni sovg'a qildi: $\vec{b} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$. Siz xursand bo'ldingiz: "Urta! Ikki vektorim bor, endi men butun boshli 2D tekislikni Span qila olaman (hamma joyga yetib boraman)!" Lekin shoshmang... Yaxshilab qarang: $\vec{b}$ aslida shunchaki $2 \cdot \vec{a}$ ning o'ziku! U ham xuddi $\vec{a}$ yotgan rels (chiziq) ustida yotibdi, shunchaki uzunroq. Agar siz ularni kombinatsiya qilsangiz ham, siz baribir shu bitta qattiq chiziq (rels) ichidan chiqib keta olmaysiz. $\vec{b}$ vektori sizga yangi "burilish" erkinligini bermadi. U bor-yo'g'i o'zingizda bor narsaning takrorlanishi xolos.

Ana shunday holat matematikada **Chiziqli Qaramlik (Linear Dependence)** deb ataladi.

Qat'iy Ta'rif: Agar guruhdagi vektorlardan kamida bittasini qolgan vektorlarning chiziqli kombinatsiyasi orqali yasab olish (yaratish) mumkin bo'lsa, bu vektorlar to'plami **Chiziqli Qaram (Linearly Dependent)** deyiladi. Buning ma'nosi shuki, agar siz shu qaram vektorni olib (o'chirib) tashlasangiz ham, sizning Spaningiz (imkoniyatlaringiz chegarasi) zarracha ham kichrayib qolmaydi. U guruhda ortiqcha yuk.

Matematik isboti: $\vec{v}_1, \vec{v}_2, \dots, \vec{v}_n$ vektorlar Chiziqli Qaram hisoblanadi, agarda quyidagi tenglamani qanoatlantiruvchi, barchasi NOLGA teng bo'lmagan ($c_1 = 0, c_2 = 0 \dots$ dan boshqqa) c skalyarlar to'plami topilsa:

$$c_1\vec{v}_1 + c_2\vec{v}_2 + \dots + c_n\vec{v}_n = \vec{0} \quad c_1\vec{v}_1 + c_2\vec{v}_2 + \dots + c_n\vec{v}_n = \vec{0}$$
(Ya'ni hech narsa qo'shmasdan nol qilish oson (hammasi 0 bo'lsa). Lekin biror nolmas miqdorda olib bir-biriga qo'shganda javob NOL ($\vec{0}$ vektor) ga qaytib kelsa, demak ular bir-birini yo'q qila oladigan o'zaro qarindosh, qaram vektorlardir).

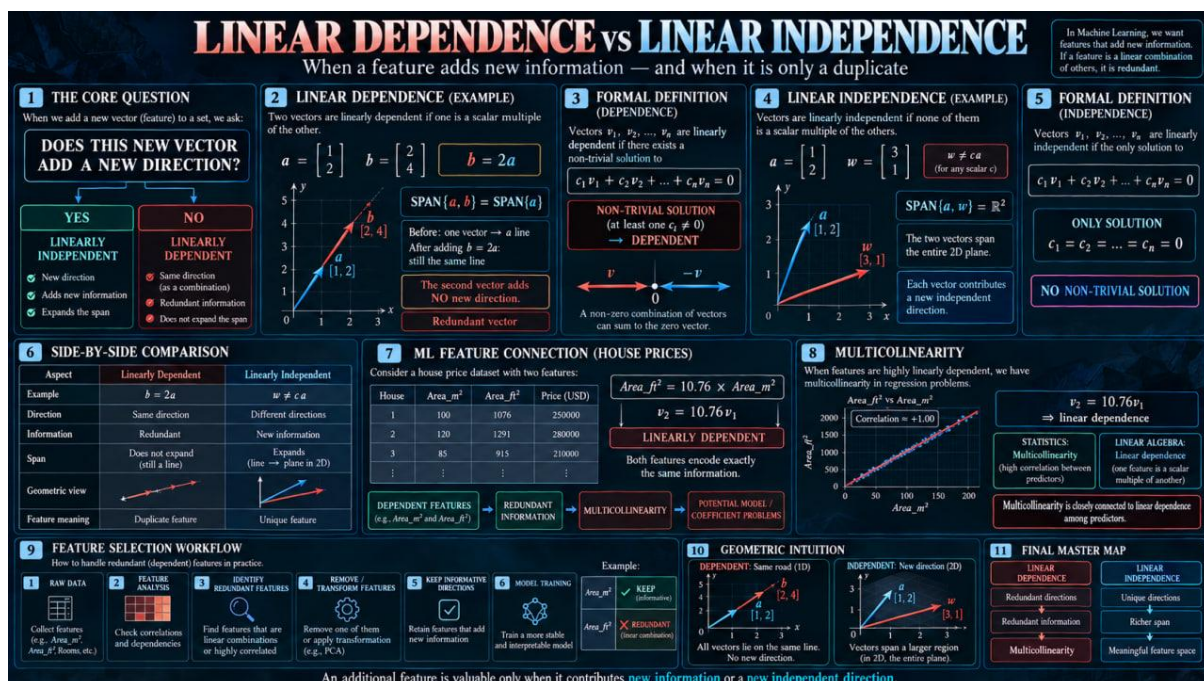
4-QISM: Chiziqli Mustaqillik (Linear Independence) - Sof Erkinlik

Endi tasavvur qiling, sizdagi ikkinchi vektor $\vec{w} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$ bo'lsin. U avvalgi $\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ ning chizig'ida (relsi) yotmagan, u butunlay boshqa burchakka qarab ketmoqda. Siz hech qachon $\vec{a}$ ni qandaydir songa ko'paytirish orqali $\vec{w}$ ni yasay olmaysiz. U mutlaqo yangi, original (betakror) yo'nalish. Bu ikkinchi vektor kelishi bilan sizning Span (imkoniyatingiz) birdaniga cheksiz tekislikka kengaydi!

Bunday guruh **Chiziqli Mustaqil (Linearly Independent)** vektorlar deb ataladi.

Qat'iy Ta'rif: Vektorlar to'plami Chiziqli Mustaqil deyiladi, agar ularning hech qaysi birini qolganlari orqali yasab (formula qilib) bo'lmasa. Ya'ni guruhdagi har bir vektor o'zining mutlaqo noyob, qolganlar yetib bora olmaydigan erkinlik yo'nalishini (dimensiyasini) qoshadi. Bu holatda:

$$c_1\vec{v}_1 + c_2\vec{v}_2 + \dots + c_n\vec{v}_n = \vec{0} \quad c_1\vec{v}_1 + c_2\vec{v}_2 + \dots + c_n\vec{v}_n = \vec{0}$$
tengligi **FAQATGINA** va **FAQATGINA** barcha skalyarlar $c_1 = c_2 = \dots = c_n = 0$ bo'lgandagina yechimga ega bo'ladi. Ular bir-birining ishiga aralashmaydigan butunlay mustaqil shaxslardir.



5-QISM: Bu tushunchalarning Machine Learning dagi O'rni (The ML Connection)

Sizda haqqli savol tug'ilishi mumkin: "Men nega relslar, tekisliklar va qaramliklar haqida bosh qotiryapman? Data Science da bu menga nima beradi?" Mana endi diqqatingizni qarating, bu barcha darsliklarning eng oltin qismidir.

Machine Learningda biz ma'lumotlar jadvallari (Dataset) bilan ishlaymiz. Har bir mijoz yozilgan **Qatorlar (Rows)** - bular fazodagi bitta nuqtalar bo'lsa, jadvaldagi har bir **Ustun / Xususiyat (Columns / Features)** — bular fazoni shakllantiruvchi VEKTORLARDIR!

Tasavvur qiling, siz uylar narxini (Target) bashorat qilish uchun ma'lumot yig'dingiz. Sizda ikkita Feature (ustun) bor:

1. Maydon_kv_metr ($\vec{v}_1$)
2. Maydon_kv_fut ($\vec{v}_2$) (1 kv.metr = 10.76 kv.fut).

Geometrik tomondan qarasangiz, $\vec{v}_2$ vektori aslida $\vec{v}_1$ vektorining aniq 10.76 ga ko'paytirilgan nusxasiku! Demak bu ikkala xususiyat (feature) **Chiziqli Qaram (Linearly Dependent)**. Ularning ikkalasini olib modelga solishning modelga hecham yangi ma'lumot (yangi yo'nalish, span) bermaydi, u bitta ustun bergan ma'lumot bilan bir xil ishni qila olardi.

Statistika tilida bu nima deyiladi? EDA darsini (1-dars) va Korrelyatsiya darsini (5-dars) eslang! Biz buni u yerda **Multikollinearlik (Multicollinearity)** deb atagan edik! Ha, Multikollinearlik — bu shunchaki Chiziqli Algebradagi "Chiziqli qaramlik" (Linear Dependence) ning Data Science dagi tarjimasidir! Agar siz ML (masalan Chiziqli Regressiya) modeliga chiziqli qaram bo'lgan (yoki o'ta kuchli korrelyatsiya qiladigan) ustunlarni bersangiz, matritsalar chalkashib ketadi (keyingi darslarda "Determinant nolga aylanadi" deb o'rganamiz) va model umuman yechim topa olmay xato (Error) beradi, yoki og'irliklari (weights) aqlbovar qilmas minglab sonlarga aylanib ketadi. Bunga **Dummy Variable Trap** ham deyiladi. Shuning uchun muhandislar modelni o'qitishdan oldin har doim ortiqcha,

qaram vektorlarni (ustunlarni) tashlab yuborishadi. Biz faqat **Chiziqli Mustaqil (Linearly Independent)** bo'lgan toza va sof ma'lumotlarni modelga kiritishimiz shart!

6-QISM: Baza (Basis) - Mukammal Fundamental To'plam

Biz ko'rdikki, koinotni (masalan 2D tekislikni) yaratish uchun bizga chiziqli mustaqil vektorlar kerak ekan. Xo'sh, fazoni to'liq ta'minlab berish uchun eng minimal (ortiqchasiz) va mukammal vektorlar to'plami nima deb ataladi? Bu to'plam geometriyada **Baza yoki Asos (Basis)** deb ataladi.

Ta'rif: Vektorlar to'plami qandaydir fazo (koinot) uchun **Baza (Basis)** hisoblanadi, agar ular **IKKITA** qat'iy shartni bir vaqtda bajarsa:

1. Ular butun fazoni qamrab olsa (**Span** sharti).
2. Ular bir-biriga mutlaqo chiziqli qaram bo'lmasa (**Linearly Independent** sharti).

Baza (Basis) - bu koinotning skeleti, uni ushlab turadigan eng muhim asosidir. Unda na biron ortiqcha (qaram) vektor bor, va na kamchilik (bo'sh qolgan joy) bor.

Standart Baza (Standard Basis): Maktab geometriyasidan biz eng mashhur bazani bilamiz.

U koordinata tizimidagi X va Y o'qlari. X o'qi (gorizontal) qadami: $\hat{i} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ Y o'qi (vertikal)

qadami: $\hat{j} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ Shu ikkita mo'jizaviy vektor $\hat{i}$ va $\hat{j}$ orqali biz tekislikdagi istalgan raqamni yasab bera olamiz (ular 2D ni to'liq span qiladi) va ularning bittasidan boshqasini aslo yasab bo'lmaydi (ular bir-biriga yuz foiz 90 gradus burchakda perpendikulyar va mustaqil). Ular 2D fazoning Standart Bazasidir.

Ammo, Baza faqat bitta bo'lmaydi! Biz ko'zimizni qisib, egrilangan butunlay boshqa ikkita vektorlar to'plamini tanlasak ham, agar ular mustaqil bo'lsa va tekislikni qoplasa, ular yordamida o'zimizning yangi, g'alati Baza koordinata sistemamizni yasashimiz mumkin. (Neyron tarmoqlari ma'lumotlarni tushunishi uchun ularni avvaliga tushunarsizdek ko'ringan o'zining maxsus "yashirin qatlam bazalariga" o'giradi).

7-QISM: O'lcham (Dimension) - Haqiqiy Murakkablik

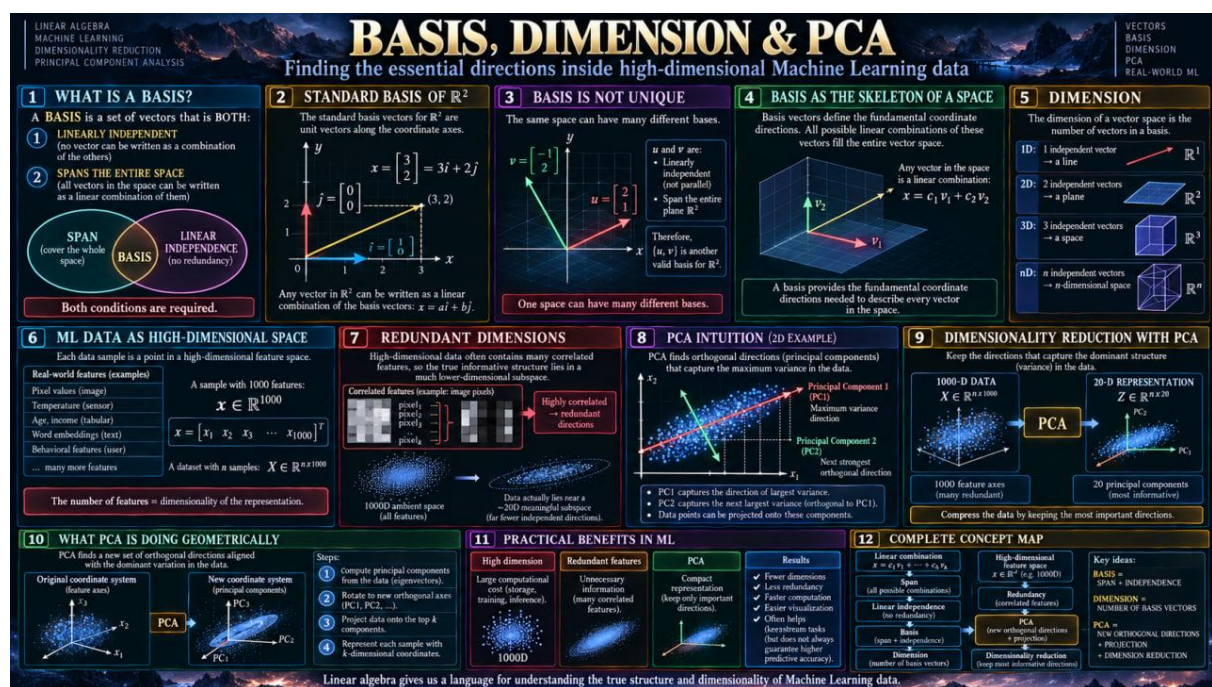
Agar biz qo'limizdagi bo'shliqning Bazasini (Basis) topgan bo'lsak, u yerda nechta vektor qatnashmoqda? Aynan mana shu BAZADAGI VEKTORLAR SONI fanda uning **O'lchami (Dimension)** deb ataladi.

- 1 ta mustaqil vektorli baza (chiziq) = **1D (Bir o'lchamli)**
- 2 ta mustaqil vektorli baza (tekislik) = **2D (Ikki o'lchamli)**
- n ta mustaqil vektorli baza = **n -Dimensional Fazo (R^n).**

Machine Learningdagi muhim tushuncha (Dimensionality Reduction - O'lchamni qisqartirish): Aytaylik, sizda 1000 ta ustundan iborat rasm (pixel) datasi bor. Demak, sizning ma'lumotingiz matematik jihatdan **1000 O'lchamli (1000 Dimensional)** fazoda joylashgan. Lekin tabiatda hamma narsa ham g'irt xaos (betartib) bo'lavermaydi. 1000 ta

piksellarning juda ko'pchiligi bir-biriga qattiq bog'liq (dependent) bo'ladi (masalan, osmon rasmidagi hamma piksellar bir xil ko'k rangda ketadi). Bu degani, garchi tashqaridan bizning fazomiz 1000-D ko'rinsada, uning aslida haqiqiy ma'noga ega bo'lgan (Independent) qismi qandaydir kichkina, balki bor-yo'g'i **20 O'lchamli (20D)** pastki fazoni (Subspace) hosil qilishi mumkin! Qolgan 980 tasi shunchaki takrorlanish (dependent).

Data Science dagi eng kuchli algoritmlardan biri — **PCA (Principal Component Analysis - Asosiy komponentlar tahlili)** qanday ishlashini bilasizmi? U aynan mana shu Chiziqli Algebra qonunlaridan foydalanib, 1000 o'lchamli katta fazo ichidan ma'lumotning asl tarqalishini ushlab turadigan yangi, kichkina (masalan 20 ta vektorli) mukammal **Baza (Basis)** o'qlarini qidirib topadi! Natijada biz 1000 o'lchamni 20 o'lchamga qisqartiramiz, lekin hech qanday foydali ma'lumotni yo'qotmaymiz (chunki span taxminan saqlanib qoladi). Modellarimiz yuzlab marotaba tezroq va aniqroq ishlashni boshlaydi.



Xulosa

Mana, aziz o'quvchim, biz chiziqli algebraning qurilish g'ishtlaridan uning me'morchiligiga (geometriyasiga) qadam qo'ydik. Bugun siz shunchaki raqamlar yig'indisini emas, balki qanday qilib **Chiziqli Kombinatsiya** yangi olamlarni kashf etishini ko'radingiz. Siz **Span** nima ekanini - u siz borishingiz mumkin bo'lgan maksimal koinot chegarasi ekanini anglab yetdingiz. Eng muhimi, nega Machine Learningda barcha ma'lumotlar to'plami ham foydali emasligi, qachon ma'lumot xato bo'lib ikkinchi ma'lumotni takrorlashi - **Chiziqli Qaramlik (Linear Dependence)** orqali tushuntirilishini bildingiz. Va buni ajoyib tarzda **O'lcham (Dimension)** bilan bog'lading.

Biz endi fazo va yo'nalishlar olamini tasavvur qila olamiz. Keyingi darsda (3-Dars) biz bularning barchasiga Matritsalarini qanday qilib bog'lashni ko'rib chiqamiz: Diagonal matritsa, Teskari (Inverse) matritsa nima va ular qanday qilib tenglamalar sistemasini yechib tashlashi, va kelajakdagi Gradient Descent optimizatsiyasi uchun qanday zamin yaratishini o'rganamiz. Shavqni pasaytirmang, koinot sirlarini ochishda davom etamiz!

ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)

3-Dars: Muhim Matritsa Turlari, Amallar va Chiziqli Tenglamalar Sistemasi Yechish

Assalomu alaykum, aziz va bo'lajak Sun'iy Intellect muhandisi! Bizning Chiziqli algebra koinotidagi sayohatimiz o'zining eng qiziqarli nuqtalaridan biriga yetib keldi.

Oldingi 2-darsimizda biz Vektorlarning chiziqli kombinatsiyasi, Fazo qamrovi (Span) va Chiziqli mustaqillik tushunchalarini o'rgangan edik. U yerda biz qanday qilib oddiy vektorlar butun boshli olamlarni (tekislik va fazolarni) yaratishi mumkinligini ko'rdik. Bugun esa e'tiborimizni to'liq **Matritsalar**ga qaratamiz. Data Science va Machine Learning olamida matritsalar shunchaki "raqamlar to'ldirilgan Excel jadvali" emas. Ular — fazoni buruvchi, cho'zuvchi, siquvchi va o'zgartiruvchi qudratli **Transformatorlar (Linear Transformations)** dir.

Neyron tarmoqlaridagi millionlab parametrlar, rasmlarni qisqartiruvchi algoritmlar va ma'lumotlardagi naqshlarni topuvchi barcha mexanizmlar matritsalar ustiga qurilgan. Bugungi darsimizda biz matritsalarining maxsus turlari bilan yaqindan tanishamiz, ularning o'ziga xos xususiyatlarini ko'ramiz va eng asosiysi — asrlar davomida odamlarni qiynab kelgan murakkab tenglamalar sistemalarini bitta nafis formula bilan qanday yechishni o'rganamiz.

Tayyor bo'lsangiz, miyangizni hisoblash rejimiga o'tkazing va chuqur matematik tahlilni boshlaymiz!

1-QISM: Kvadrat Matritsa (Square Matrix) - Mukammal O'lcham

Dunyoda juda ko'plab turdagi matritsalar bor. Ular uzunchoq (qatorlari ko'p) yoki yotiq (ustunlari ko'p) bo'lishi mumkin. Lekin ularning ichida eng elitasi, eng ko'p imtiyozlarga ega bo'lgani bu — **Kvadrat Matritsa (Square Matrix)** dir.

Ta'rif: Qatorlar soni (m) va ustunlar soni (n) bir-biriga teng bo'lgan matritsa Kvadrat matritsa deb ataladi. Ya'ni, uning o'lchami doimo $n \times n$ ko'rinishida bo'ladi. Masalan, 2×2 , 3×3 , yoki 1000×1000 o'lchamli matritsalar.

Nima uchun ular shunchalik muhim? Geometrik nuqtai nazardan, kvadrat matritsalar ma'lumotlarni **o'z fazosida saqlab qoladigan** operatsiyalardir. Agar siz 3 o'lchamli vektorni 3×3 kvadrat matritsaga ko'paytirsangiz, u yana 3 o'lchamli vektor bo'lib chiqadi. U ma'lumotni buzmaydi, o'lchamni qisqartirmaydi, shunchaki fazoni buradi yoki cho'zadi. Barcha eng qiziqarli matematik xususiyatlar (Determinant, Teskari matritsa, Xos qiymatlar - Eigenvalues) FAQAT kvadrat matritsalarda mavjud bo'ladi.

Raqamlardagi Misol (3×3 kvadrat matritsa):

$$A = \begin{bmatrix} 9 & 2 & 3 \\ 4 & 11 & 6 \\ 7 & 8 & 10 \end{bmatrix}$$

Asosiy Diagonal (Main Diagonal) va Iz (Trace): Kvadrat matritsalarining juda muhim bir qismi bor. Bu uning chap yuqori burchagidan o'ng pastki burchagigacha qiya tortilgan elementlar qatoridir. Yuqoridagi A matritsada 9,11,10 raqamlari **Asosiy diagonal** ni hosil qiladi. Matematik tilda a_{11}, a_{22}, a_{33} elementlar. Shu asosiy diagonalidagi barcha raqamlarning yig'indisi matritsaning **Izi (Trace - Tr)** deb ataladi. $Tr(A) = 9 + 11 + 10 = 30$. Trace xususiyati Kvant mexanikasida va murakkab ML optimizatsiyalarida saqlanib qoluvchi muhim invariant (o'zgarmas) miqdor hisoblanadi.

2-QISM: Diagonal Matritsa (Diagonal Matrix) - Sof O'lchov

Endi kvadrat matritsalarining ichidan eng oddiy va eng "yuvvosh" turini ajratib olamiz. Bu **Diagonal Matritsa** dir.

Ta'rifi: Agar kvadrat matritsaning **FAQAT asosiy diagonalida** noldan farqli raqamlar bo'lib, uning qolgan BARCHA (tepa va pastki) qismlari faqat Nollardan (0) iborat bo'lsa, bu Diagonal matritsa deyiladi. Matematik qoida: agar $i \neq j$ bo'lsa, $a_{ij} = 0$.

Misol:

$$D = \begin{bmatrix} 5 & 0 & 0 \\ 0 & -2 & 0 \\ 0 & 0 & 8 \end{bmatrix}$$

Uning qanday Super Kuchi bor?

1. **Amallar osonligi:** Diagonal matritsalar ustida amallar bajarish juda oson. Agar siz shunday ikkita matritsani bir-biriga qo'shsangiz yoki ko'paytirsangiz, siz faqat ularning diagonalidagi mos raqamlarni qo'shasiz yoki ko'paytirasiz. Hech qanday murakkab formula kerak emas.
2. **Darajaga ko'tarish:** Eng daxshatli kuchi shundaki, agar siz D matritsani 100-darajaga ko'tarmoqchi bo'lsangiz (D^{100}), kompyuter millionlab amallar qilib o'tirmaydi. U shunchaki diagonalidagi har bir raqamni alohida 100-darajaga ko'tarib qo'yadi xolos!

$$D^3 = \begin{bmatrix} 5^3 & 0 & 0 \\ 0 & (-2)^3 & 0 \\ 0 & 0 & 8^3 \end{bmatrix}$$

3. **Geometrik ma'nosi:** Oddiy matritsalar fazoni ham cho'zadi, ham qiyshaytiradi (buradi). Ammo Diagonal matritsa fazoni **UMUMAN burmaydi**. U X, Y, Z o'qlarini faqat o'z o'qi bo'ylab cho'zadi yoki siqadi (Scaling).

Machine Learningdagi o'rni: Biz keyinchalik PCA (Principal Component Analysis) yoki SVD (Singular Value Decomposition) kabi murakkab algoritmlarni o'tamiz. Bu algoritmlarning yagona maqsadi — juda murakkab, iflos, chalkash matritsalarini qanday qilib o'zgartirib, aynan shunday toza, oddiy va hisoblash oson bo'lgan **Diagonal Matritsaga (Diagonalization)** aylantirishdan iboratdir! Barcha kuch diagonalga yashiringan.

3-QISM: Birlik Matritsa (Identity Matrix) - Matritsalar Olamidagi "1"

Raqami

Keling, diagonal matritsaning yana-da maxsus, mutlaqo noyob bitta versiyasini ko'ramiz.

Ta'rifi: Asosiy diagonalidagi BARCHA elementlari roppa-rosa **1** ga teng bo'lgan va qolgan barcha elementlari **0** bo'lgan diagonal kvadrat matritsa — **Birlik Matritsa (Identity Matrix)** deb ataladi. U odatda I (Identity) yoki o'lchamini ko'rsatuvchi I_n bilan belgilanadi.

Misollar:

$$I_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

U nima vazifa bajaradi? Oddiy arifmetikada 1 raqami qanday mo'jizaga ega ekanini bilasiz: istalgan sonni 1 ga ko'paytirsangiz, hech narsa o'zgarmaydi ($5 \times 1 = 5$). Chiziqli algebrada **Birlik matritsa aynan 1 raqamining rolini o'ynaydi!** Har qanday A matritsani I matritsaga chapdan yoki o'ngdan ko'paytiring, natijada A ning o'zi o'zgarmasdan chiqaveradi:

$$A \cdot I = I \cdot A = A$$

Geometrik jihatdan, agar Birlik matritsa fazoga (vektorlarga) ta'sir qilsa, u ularni na cho'zadi, na buradi. Hamma narsani qanday bo'lsa, shunday joyida qoldiradi ("Do nothing" transformation). Biz buni hozir biroz pastroqda "Teskari matritsa"ni tushinishda eng asosiy tayanch nuqtasi sifatida ishlatamiz.

4-QISM: Transponirlash (Transpose) - Qatorlar va Ustunlar O'rnini

Almashtirish

Biz matritsalar ustida arifmetik amallarni bilamiz, lekin matritsaning strukturasi (shakli) bilan ham "o'ynashish" mumkin. Shunday jarayonlardan biri **Transponirlash (Transpose)** dir.

Ta'rifi: Matritsani Asosiy diagonali bo'ylab ko'zgu kabi ag'darish (yoki aylantirish) Transponirlash deyiladi. Bunda matritsaning **Barcha Qatorlari — Ustunlarga, barcha Ustunlari esa — Qatorlarga** aylanib qoladi. Belgilanishi: A matritsaning transponirlangani A^T (ba'zan A' yoki A^t) deb yoziladi.

Misol: Aytaylik, bizda 2×3 (ikki qator, uch ustun) o'lchamli to'g'ri to'rtburchak matritsa bor:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Agar biz uni Transponirlasak (A^T), u 3×2 o'lchamli (uch qator, ikki ustun) shaklga keladi:

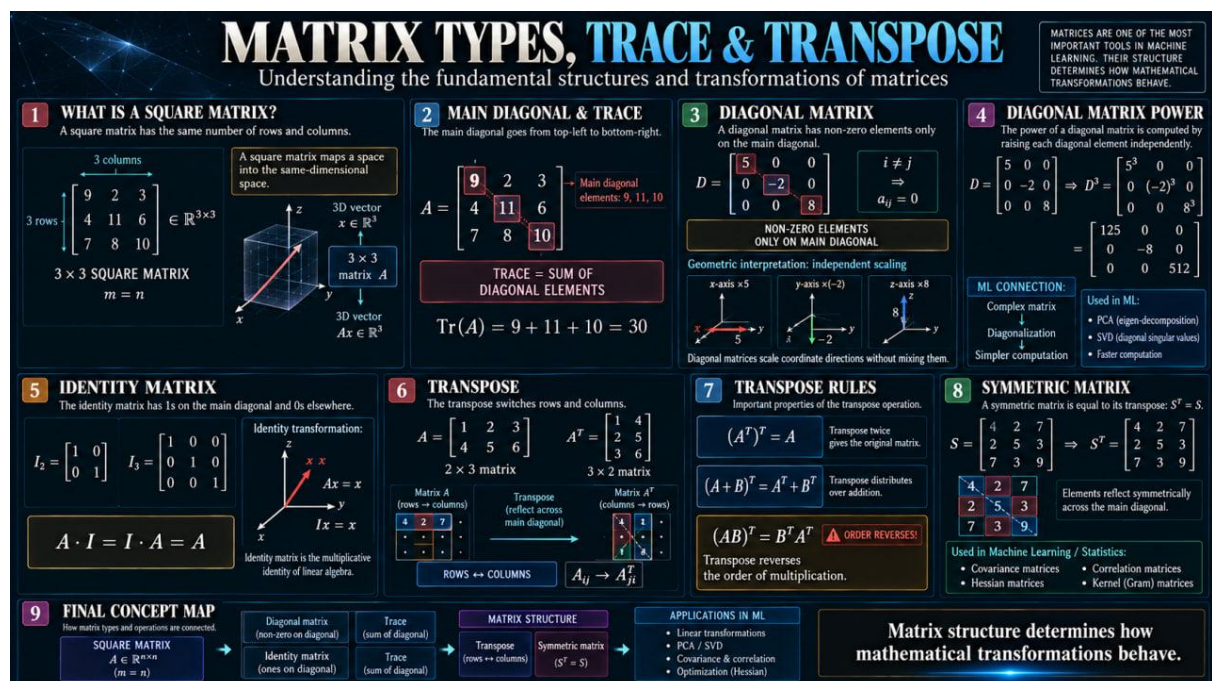
$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Ko'rib turganingizdek, A ning yotiq yotgan 1-qatori (**1, 2, 3**), A^T ning tikka turgan 1-ustuniga aylandi.

Transponirlashning Muhim Xususiyatlari:

1. **Ikki marta ag'darish:** Matritsani transponirlab, uni yana transponirlasangiz, asliga qaytadi. $(A^T)^T = A$.
2. **Qo'shish ustida:** $(A + B)^T = A^T + B^T$. O'zgarishsiz ishlaydi.
3. **Ko'paytirish ustida (DIQQAT - Oltin Qoida!):** $(AB)^T = B^T A^T$. E'tibor bering, matritsalar ko'paytmasini transponirlaganda, ularning **O'RNİ TESKARISIGA ALMASHIB QOLADI!** Bu chiziqli algebradagi eng ko'p xato qilinadigan va imtihonlarda eng ko'p so'raladigan joydir. Qoida teskarisiga ishlashiga sabab matritsalar o'lchamlari (dimensions) mos tushishi kerakligidir.

Simmetrik Matritsalar (Symmetric Matrices): Kvadrat matritsalar orasida shunday bir ajoyib tur borki, uni Transponirlaganingizda ham u o'zgarmaydi! Agar $S^T = S$ bo'lsa, bunday matritsa **Simmetrik** deb ataladi. Ya'ni diagonalning tepasidagi va pastidagi raqamlar xuddi kapalak qanotlaridek bir-biriga mukammal aks etib turadi. *Nega buni bilyapmiz?* Statistika va Machine Learningda barcha **Kovariatsiya matritsalar (Covariance matrices)**, **Korrelyatsiya matritsalar (Correlation matrices)** va Hessian matritsalar har doim va faqatgina Simmetrik matritsalar bo'ladi. Ular ma'lumotlar fani matematikasining yuragidir.



5-QISM: Matritsalarini Ko'paytirish (Matrix Multiplication) - Mantiqni Chuqurlashtiramiz

O'tgan 1-darsda biz Matritsalarini ko'paytirish qoidasini yuzaki ko'rib chiqqan edik ("Qatorni ustunga ko'paytirish - Dot product"). Endi keling, uning aslida nima ekanini va nega aynan shu g'alati qoida bilan ko'paytirilishini tushunib olamiz.

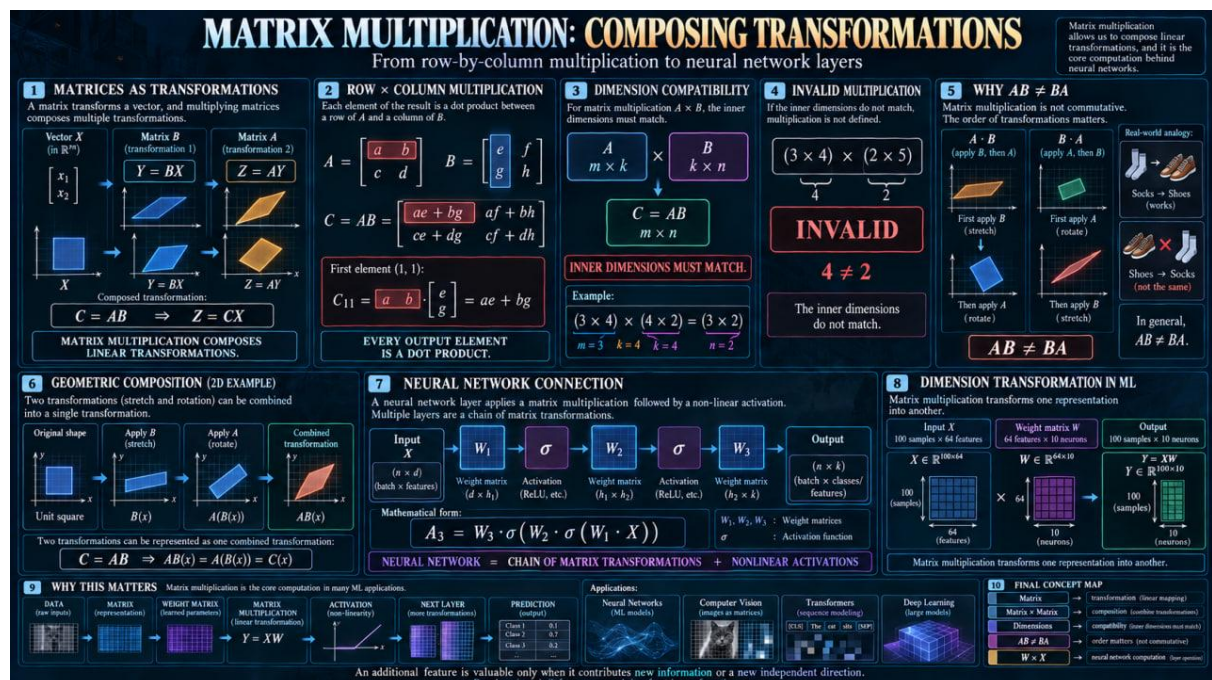
Nima uchun qatorni ustunga uramiz? Tasavvur qiling, siz qandaydir bir funksiyalar tizimini quryapsiz. X kompyuter kiritgan ma'lumot, Y oraliq natija, Z esa yakuniy natija bo'lsin. 1-qadamda $Y = B \cdot X$ qilamiz. (B matritsa X ni Y ga aylantirdi). 2-qadamda $Z = A \cdot Y$ qilamiz. (A matritsa Y ni Z ga aylantirdi). Agar biz X dan to'g'ridan-to'g'ri bir hatlab Z ga o'tmoqchi bo'lsak-chi? Matematiklar aniqlashganki, agar $C = A \cdot B$ qilib ikkita matritsani birinchi bo'lib bir-biriga (qatorni ustunga urish qoidasi orqali) ko'paytirsak, hosil bo'lgan yangi C matritsa

ikkala jarayonni o'zida mujassam etgan yagona "Super Transformator" ga aylanadi: $Z = C \cdot X$. Demak, Matritsalarini ko'paytirish — bu **Lineer transformatsiyalarni ketma-ket birlashtirish (Composition of linear transformations)** deganidir!

O'lchamlar qonuni (Esda saqlash shart!): A matritsa ($m \times k$) o'lchamda, B matritsa ($k \times n$) o'lchamda bo'lsa, ularni ko'paytirib bo'ladimi? HA! Chunki birinchisining **ustunlar soni (k)**, ikkinchisining **qatorlar soniga (k) teng**. Ular xuddi lego kabi bir-biriga kirishib ketadi. Hosil bo'lgan yangi $C = AB$ matritsasining o'lchami chekkada qolgan raqamlardan iborat bo'ladi: ($m \times n$).

Nima uchun $AB \neq BA$? (Kommutativlik qonunining yo'qligi) Oddiy raqamlarda $3 \times 4 = 12$ va $4 \times 3 = 12$. Lekin matritsalarida o'rinlarni almashtirsangiz umuman boshqa javob chiqadi yoki xatto ko'paytirib bo'lmay qoladi. Nima uchun? Hayotiy analogiya qilib ko'ramiz. "Paypoq kiyish" amali A matritsa bo'lsin, "Tufli kiyish" amali B matritsa bo'lsin. Agar siz oldin paypoq, keyin tufli kiysangiz (BA funksiyasi), hamma narsa to'g'ri bo'ladi. Lekin ularning o'rnini almashtirsangiz, ya'ni oldin tufli, uning ustidan paypoq kiysangiz (AB funksiyasi), ko'rinish umuman boshqacha va mantiqsiz bo'ladi. Transformatsiyalarda jarayonlarning ketma-ketligi o'ta muhim ahamiyatga ega!

(Neyron tarmoqlar (Neural Networks) da barcha qatlamlar bitta ulkan Matritsa ko'paytmasi formulasi zanjiridan iboratdir: $A_3 = W_3 \cdot \sigma(W_2 \cdot \sigma(W_1 \cdot X))$). Mana shu joyda W lar biz o'rgangan aynan shu matritsalaridir).



6-QISM: Teskari Matritsa (Inverse Matrix) - Vaqtni Orqaga Qaytarish

Keling, chiziqli algebraning eng buyuk sirlaridan birini ochamiz. Oddiy raqamlar dunyosida har bir amalning o'z teskarisi bor. Qo'shishning teskarisi ayirish. Ko'paytirishning teskarisi bo'lish. Agar siz sonni 5 ga ko'paytirsangiz ($x \cdot 5$), orqaga qaytish uchun uni yana teskari soniga ($5^{-1} = 1/5$) ko'paytirasiz va asl holatiga qaytasiz.

Matritsalar "bo'lish" degan tushuncha umuman yo'q! Hech qachon ikkita matritsani bir-biriga bo'lmaysiz (A/B degan narsa mavjud emas). Buning o'rniga bizda **Teskari Matritsa (Inverse Matrix - A^{-1})** tushunchasi mavjud.

Ta'rif: Agar shunday bitta A^{-1} matritsa topilsaki, uni bizning original A matritsamizga (xoh chapdan, xoh o'ngdan) ko'paytirganimizda, javob mutlaqo toza **Birlik Matritsaga (I)** aylanib qolsa, bu A^{-1} matritsa A ning Teskari Matritsasi deyiladi. Matematik qonun:

$$A \cdot A^{-1} = A^{-1} \cdot A = I$$

(Bu xuddi $5 \times (1/5) = 1$ deganidek gap. A o'zgarish qildi, A^{-1} esa o'sha o'zgarishni "bekor qildi", vaqtini orqaga qaytardi, natijada hamma narsa o'zgarishsiz holatga, I ga keldi).

Qachon Teskari matritsa mavjud bo'ladi?

1. U **faqatgina Kvadrat Matritsalar** uchun mavjud bo'lishi mumkin. To'g'ri to'rtburchak matritsalarining aniq teskarisi bo'lmaydi (ularda faqat pseudo-inverse bo'ladi).
2. Va eng asosiysi: **Uning Determinanti noldan farqli bo'lishi shart!** ($\det(A) \neq 0$).

Agar Determinant = 0 bo'lsachi? (Singular Matrix) Buni 2-darsimizdagi bilimlar orqali tushunamiz. Determinantning 0 bo'lishi — matritsani tashkil qiluvchi vektorlar **Chiziqli Qaram (Linearly Dependent)** ekanini bildiradi. Ya'ni transformatsiya fazoni qisib yuborgan (masalan 2D tekislikni bitta chiziqqa qisqartirib qisib yuborgan). Tasavvur qiling, siz 3D modelni g'ijimlab 2D varaq holatiga keltirdingiz. Siz ma'lumotni yo'qotdingiz. Endi o'sha varaqdan qanday qilib eski hajmli 3D modelni qayta yasab olasiz? Iloji yo'q! Nima yo'qolganini bilmaysiz. Aynan shuning uchun, agar determinant 0 bo'lsa fazo siqilgan bo'ladi va **Teskari matritsa umuman mavjud bo'lmaydi!** Bunday qaytarib bo'lmas, ma'lumotni o'ldiradigan matritsalar **Singular Matritsalar (Singular Matrices)** deyiladi. ML da bunday muammoga tushib qolsangiz (masalan multikollinearlik tufayli), kompyuteringiz xato (Error: Singular Matrix) berib to'xtab qoladi.

2×2 Matritsa uchun Teskarisini topish formulasi: Kattaroq matritsalarini kompyuter Gauss-Jordan usuli bilan topadi, lekin 2×2 uchun oson mo'jizaviy formula bor. Agar

$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ bo'lsa, Uning determinanti: $\det(A) = (a \cdot d) - (b \cdot c)$ Teskari matritsa shunday yasaladi:

1. Asosiy diagonalda a va d ning o'rnini almashtiramiz.
2. Qolgan (ikkinchi) diagonalda b va c ning ishorasini minusga o'zgartiramiz.
3. Hammasini oldiga chiqqan Determinantga bo'lib chiqamiz ($1/\det$).

$$A^{-1} = \frac{1}{(ad-bc)} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} A^{-1} = \frac{1}{(ad-bc)} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

Solishtirish: Tasavvur qiling siz A orqali barcha rasmlardagi piksellarni kattalashtirdingiz. Endi rasmlarni yana o'z holiga qaytarmoqchi bo'lsangiz, uni yana bitta qilib bo'lib o'tirmaysiz. Shunchaki oldindan hisoblangan A^{-1} ni piksellarga ko'paytirib yuborasiz va hammasi o'z joyiga qaytadi.

7-QISM: Chiziqli Tenglamalar Sistemasi Yechish (Teskari Matritsaning G'alabasi)

Nima uchun biz bu qadar g'alati qoidalar bilan Teskari matritsa izlaymiz? Sababi, chiziqli algebra olimlari asrlar davomida odamlarni qiynab kelgan bitta ulkan muammoni — yuzlab, minglab o'zgaruvchilari bor murakkab Tenglamalar Sistemalarini osongina yechishni maqsad qilishgan edi.

Maktab darsligidan misol: Keling, eng oddiy 2 ta noma'lumli tenglamani yodga olamiz:

1. $3x + 4y = 10$
2. $2x + 3y = 7$

Odatda biz buni yechish uchun ikkinchi tenglamani biror songa ko'paytirib, birinchisidan ayirib, x ni yo'qotib, keyin y ni topib, yana orqaga qo'yib azob chekardik. Bu "O'rniga qo'yish usuli" deb ataladi. Ikkita tenglama uchun oson, lekin agar 100 ta noma'lumli 100 ta tenglama bo'lsachi? Inson umri yetmaydi.

Matritsa Usulida Yechish (Sehr shu yerda boshlanadi): Biz yuqoridagi tenglamalar sistemasini 3 ta alohida bloklarga (matritsalariga) ajratib yozib olamiz.

1. Koeffitsientlar matritsasi (A): Noma'lumlar oldidagi raqamlar
2. Noma'lumlar vektori (x): Biz izlayotgan x va y
3. Natijalar vektori (b): Tenglikdan keyingi javoblar.

Shunda bizning tizim bunday ko'rinishga keladi:

$$\underbrace{\begin{bmatrix} 3 & 4 \\ 2 & 3 \end{bmatrix}}_A \cdot \underbrace{\begin{bmatrix} x \\ y \end{bmatrix}}_x = \underbrace{\begin{bmatrix} 10 \\ 7 \end{bmatrix}}_b \underbrace{\begin{bmatrix} 3 & 4 \\ 2 & 3 \end{bmatrix}}_A \cdot \underbrace{\begin{bmatrix} x \\ y \end{bmatrix}}_x = \underbrace{\begin{bmatrix} 10 \\ 7 \end{bmatrix}}_b$$

Ko'rib turibsizmi? Biz murakkab tizimni qisqagina $A \cdot x = b$ ko'rinishidagi eng oddiy tenglamaga aylantirdik!

Endi buni qanday yechamiz? Biz izlayotgan narsa — x vektori. Uni yolg'iz qoldirishimiz kerak. Buning uchun tenglikning IKKALA tomonini ham A ning Teskari matritsasi (A^{-1}) ga **chap tomondan** ko'paytiramiz (esingizda bo'lsin, matritsalarida tomonlar muhim).

$$A^{-1} \cdot (A \cdot x) = A^{-1} \cdot b$$

Biz bilamizki, $A^{-1} \cdot A = I$ (Birlik matritsa).

$$I \cdot x = A^{-1} \cdot b$$

Birlik matritsani (I) har qanday narsaga ko'paytirsangiz yana o'zi qoladi. Demak, yakuniy mo'jizaviy YECHIM formulasi tayyor:

$$x = A^{-1} \cdot b$$

Hech qanday qo'shish, ayirish, o'rninga qo'yish shart emas! Shunchaki A ning teskarisini topib, b ga ko'paytirib, bo'ldi! Hamma noma'lumlar bir zumda topiladi.

Sonlarda isbotlab ko'ramiz: Tepadagi $A = \begin{bmatrix} 3 & 4 \\ 2 & 3 \end{bmatrix}$ ning teskarisini topamiz. Determinant $\det(A) = (3 \cdot 3) - (4 \cdot 2) = 9 - 8 = 1$. (Ajoyib, determinant 1 ga teng ekan, nol emas, yechim bor!).

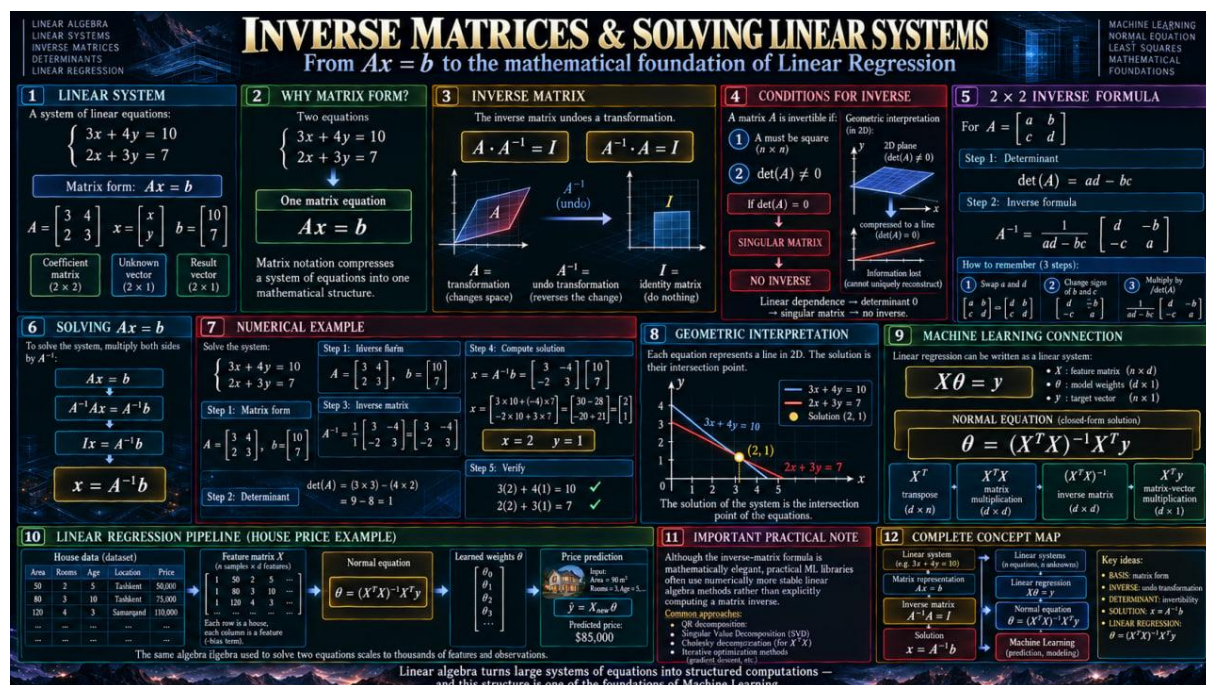
$$A^{-1} = \frac{1}{1} \begin{bmatrix} 3 & -4 \\ -2 & 3 \end{bmatrix} = \begin{bmatrix} 3 & -4 \\ -2 & 3 \end{bmatrix}$$

Endi x ni topish uchun uni natijalar vektori b ga $([10, 7]^T)$ ko'paytiramiz:

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 3 & -4 \\ -2 & 3 \end{bmatrix} \cdot \begin{bmatrix} 10 \\ 7 \end{bmatrix} = \begin{bmatrix} 3 & -4 \\ -2 & 3 \end{bmatrix} \cdot \begin{bmatrix} 10 \\ 7 \end{bmatrix}$$

Qatorni ustunga uramiz: 1-qator: $(3 \cdot 10) + (-4 \cdot 7) = 30 - 28 = 2$ 2-qator: $(-2 \cdot 10) + (3 \cdot 7) = -20 + 21 = 1$

Javob chiqdi! $x = 2, y = 1$. Tekshirib ko'ramiz: $3(2) + 4(1) = 6 + 4 = 10$. Mukammal ishlaromqda!



Machine Learning bilan Bog'liqligi (The Ultimate ML Connection): Nega biz buni o'rgandik? Data Sciencedagi eng birinchi va mashhur algoritim bu — **Chiziqli Regressiya (Linear Regression)** dir. Uylarning narxini bashorat qilish kabi masalalar. U yerda sizda minglab qatorli Data (X matritsa) va Minglab narxlar javobi (y vektor) bo'ladi. Siz esa modelning Og'irliklarini (Weights, bizdagi noma'lum x kabi θ vektorini) topishingiz kerak bo'ladi. Chiziqli regressiyaning matematik olamdagi 100% aniq va rasmiy yechimi (Normal Equation deb ataladi) xuddi yuqoridagi kabi yoziladi:

$$\theta = (X^T \cdot X)^{-1} \cdot X^T \cdot y$$

E'tibor berdingizmi? Formulasi har xil ko'ringani bilan, uning ichida aynan biz hozir o'rgangan **Transponirlash** (X^T), **Matritsalar ko'paytmasi** ($X^T \cdot X$) va eng asosiysi shularning **Teskari Matritsasi** ($^{-1}$) yotibdi! ML dagi barcha modellar ortida mana shu oddiy tenglamalar sistemasini, shunchaki juda ulkan o'lchamlarda ekanligiga qaramay kompyuter yordamida izchil va tezkor hal etish san'ati yotadi.

Xulosa

Mana aziz do'stim, siz yana bir muhim dovondan o'tdingiz! Bugungi darsda siz matritsalar faqat raqamlar qutisi emasligini, ular haqiqiy hayotdagi chiziqli o'zgarishlarning tili ekanligini anglab yetdingiz. Siz diagonal va birlik matritsalarining vazifasini bildingiz. Transponirlash qanday qilib o'lchamlarni burchakka burishini ko'rdingiz. Eng katta yutug'ingiz esa — siz endi dunyodagi har qanday murakkablikdagi Chiziqli Tenglamalar tizimini nega **Teskari Matritsa** orqali bitta xamla bilan yechish mumkinligini, va bu kompyuterlarga, Machine Learning modellariga nima uchun kerakligini chin dildan tushunasiz.

Chiziqli algebra arxitekturamiz tayyor. Keyingi bekatlarimizda biz vektorlar o'zgarishlarga duch kelganda o'z yo'nalishini yo'qotmaydigan, ML ni tebratib turgan asosiy tushunchalar: xos qiymatlar (Eigenvalues) va xos vektorlar (Eigenvectors) qatorlariga qadam qo'yamiz. Hozircha esa bugungi matritsa turlarini qog'ozda bir bor o'zingiz sinab ko'ring va ajoyib ishingizdan zavqlaning!

ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)

4-Dars: Determinant, Xos Qiymatlar (Eigenvalues) va Xos Vektorlar (Eigenvectors)

Assalomu alaykum, bilimdon va tinimsiz izlanuvchi Data Scientist! Bizning chiziqli algebra olamidagi sayohatimiz o'zining eng apogeyiga (cho'qqisiga) yetib keldi. O'tgan darslarda siz matritsalar faqatgina "Raqamlar yozilgan katakchalar" emasligini, balki ular butun boshli koinotni (fazo va vektorlarni) cho'zuvchi, buruvchi, qisuvchi va aylantiruvchi **Lineer Transformatsiyalar (Linear Transformations)** ekanligini anglab yetdingiz.

Bugun biz siz bilan o'sha transformatsiyalarning "Xarakterini", ya'ni ularning ichki qalbini o'lchaydigan ajoyib matematik asboblardan tanishamiz. Agar siz ML olamida ma'lumotlarni qanday qilib "siqishni" (qisqartirishni), shovqinlardan tozalashni yoki Google qidiruv tizimi (PageRank algoritmi) qanday qilib eng muhim veb-sahifalarni topishini tushunmoqchi bo'lsangiz, bugungi dars siz uchun kashfiyot bo'ladi.

Tayyor bo'lsangiz, tasavvuringizni ishga tushiring, biz boshladik!

1-QISM: Determinant - Kengayish yoki Qisqarish O'lchovi

Maktab yoki universitet darslarida "Determinant" deganda ko'pchiligimizning ko'z o'ngimizga uzun, zerikarli ko'paytirish va ayirish formulalari ($ad - bc$) keladi. Ammo u aslida nimani anglatadi? Nima uchun u hamma joyda qatnashaveradi?

Geometrik Ma'nosi (The Geometric Intuition): Tasavvur qiling, siz 2D (ikki o'lchamli)

tekislikdasiz. Sizda ikkita asosiy baza vektorlari bor: $\hat{i} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ (X o'qi bo'ylab 1 qadam) va

$\hat{j} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ (Y o'qi bo'ylab 1 qadam). Agar shu ikkita vektorni birlashtirsangiz, ularning o'rtasida tomonlari 1 ga teng bo'lgan mukammal **Kvadrat** yasaladi. Uning yuzasi (Area) $1 \times 1 = 1$ ga teng.

Endi bu tekislikka qandaydir A matritsasi transformatsiya sifatida ta'sir qildi:

$$A = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} A = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}$$

Bu matritsa nimani anglatadi? U birinchi o'qni (X ni) 3 marta cho'zadi, ikkinchi o'qni (Y ni) esa 2 marta cho'zadi. Endi boyagi eski kichkina kvadratimiz nima bo'ldi? Uning tomonlari 3 va 2 bo'lgan to'g'ri to'rtburchakka aylandi. Uning yangi yuzasi $3 \times 2 = 6$ ga teng! Bizning fazomiz (yuzamiz) dastlabki holatidan roppa-rosa **6 barobar** kattalashdi. Aynan mana shu "Kattalashish yoki Kichrayish Koeffitsienti" — **Determinant** deb ataladi! Biz yozamiz: $\det(A) = 6$.

- Agar $\det(A) = 1$ bo'lsa, demak transformatsiya fazoning yuzasini/hajmini o'zgartirmagan (masalan, shunchaki rasmni 90 gradusga burgan xolos).
- Agar u 3D (uch o'lchamli) matritsa bo'lsa, determinant uning Yuzasini emas, balki **Hajmining (Volume)** qanchalik o'zgarganini ko'rsatadi.

2x2 Matritsa Formulali ($ad - bc$ qayerdan kelgan?): Oddiy to'g'ri to'rtburchak matritsa

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \text{ ni olaylik. Bunda birinchi ustun } \begin{bmatrix} a \\ c \end{bmatrix} \text{ yangi o'zgartirilgan } \hat{i} \text{ vektori qayerga}$$

tushganini ko'rsatadi. Ikkinchi ustun $\begin{bmatrix} b \\ d \end{bmatrix}$ esa yangi $\hat{j}$ qayerga tushganini ko'rsatadi. Bu ikki vektor birlashib parallelogramm hosil qiladi. Maktab geometriyasi orqali isbotlanganki, ana shu parallelogrammning yuzasi doim aynan $(a \cdot d) - (b \cdot c)$ formulasi bilan topiladi! Mo'jiza!

2-QISM: Manfiy (Negative) va Nol (Zero) Determinantlar

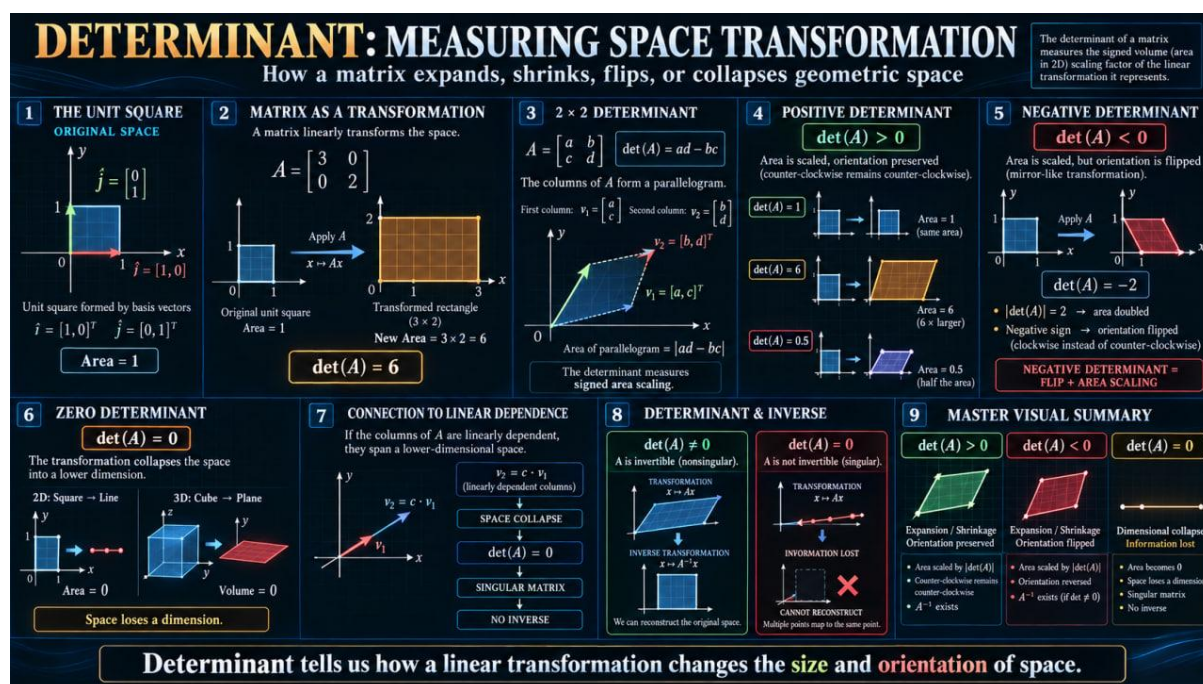
Determinant har doim ham musbat son chiqavermaydi. Uning manfiy yoki aniq nol chiqishining ortida juda katta falsafa yotadi.

A. Manfiy Determinant (Fazo To'ntarilishi): Agar $\det(A)$ manfiy son chiqsa (masalan $\det(A) = -2$), bu maydon kattalashganini, LAKIN fazoning "Ichi tashqarisiga to'ntarilib (ag'darilib) qolganini" bildiradi! Tasavvur qiling, siz qog'ozda o'qiyapsiz. Agar men qog'ozni orqa tomoniga o'girsam, yozuvlar chapdan o'ngga emas, teskari (ko'zgu effekti) o'qila boshlaydi. Geometriyada bu $\hat{i}$ va $\hat{j}$ vektorlari o'zaro o'rin almashib, fazoning orientatsiyasi (Orientation) o'zgarganligini bildiradi. Ammo hudud baribir 2 barobar kattalashgan (mutlaq qiymati).

B. Nol Determinant ($\det(A) = 0$) - Qutqarib bo'lmas halokat! Agar transformatsiyadan keyin determinant 0 ga teng bo'lib qolsa, bu nima degani? Bu degani, bizning yuzamiz yoki hajmimiz mutlaqo yo'q bo'lib, **Nolga aylandi**. Qanday qilib 2D qog'ozning yuzasi nol bo'lishi

mumkin? Faqat bitta yo'l bilan — agar siz u qog'ozni g'ijimlab, bitta oddiy 1D chiziqqa (line) aylantirib qo'ysangiz! Yoki 3D kubikni qisib, yassilab tekislikka tushirsangiz (yoki undan ham yomoni bitta nuqtaga abjag'ini chiqarsangiz). *Eslatma:* Bu holatni 2-darsda biz nima degan edik? **Chiziqli Qaramlik (Linear Dependence)**! Ha, agar matritsa ustunlari bir-biriga qaram bo'lsa (bittasi ikkinchisining ustida yotsa), ular fazoni kengaytira olmaydi, natijada fazo qisiladi va $\det(A) = 0$ chiqadi.

Teskari Matritsa va Determinant Aloqasi (Oltin Qoida): Nima uchun o'tgan (3-dars)da "Determinant nolga teng bo'lsa, teskari matritsa mavjud bo'lmaydi" degan edik? Chunki, Teskari Matritsa (A^{-1}) — bu vaqtni orqaga qaytarish amalidir. Agar matritsa fazoni qisib yubormagan bo'lsa, biz orqaga qayta olamiz. Ammo $\det(A) = 0$ bo'lib, siz 2D rasmni bitta to'g'ri chiziqqa majburlab qisib yuborsangiz (informatsiyani o'ldirsangiz), endi o'sha bitta chiziqqa qarab eski chiroyli rasmning piksellari qayerda bo'lganini bilib, qayta tiklab ololasizmi? YO'Q! Ma'lumot yo'qoldi. Shuning uchun **Singular Matrices** ($\det = 0$) ning teskarisi umuman mavjud bo'lmaydi. Ular ma'lumotlar qotilidir.



3-QISM: Xos Vektorlar (Eigenvectors) va Xos Qiymatlar (Eigenvalues) - Fazoning Ustunlari

Determinant butun boshli fazoning hajmi qanday o'zgarishini ko'rsatsa, **Eigenvalues / Eigenvectors** (Talaffuzi: Aygen-valyuz) fazoning aynan qaysi yo'nalishlari asosiy ta'sirga uchrashini ko'rsatuvchi mikroskopik vositalardir. Nemis tilida "Eigen" so'zi "O'ziga xos, maxsus, xususiy" degan ma'nolarni anglatadi.

Muammoning qo'yilishi: Tasavvur qiling, bizda bitta kvadrat matritsa A bor. U qandaydir Transformatsiyani amalga oshiradi (buradi va cho'zadi). Biz bu fazoga yuzlab turli xil vektorlarni (strelkalarni) chizib qo'ydik. Transformatsiya boshlanganda, matritsa bu vektorlarning hammasini o'z o'rnidan qo'zg'atadi: ularning ba'zilari o'ngga qiyshayadi, ba'zilari chapga aylanib ketadi, uzunliklari o'zgaradi. Xullas, deyarli hamma vektorlar

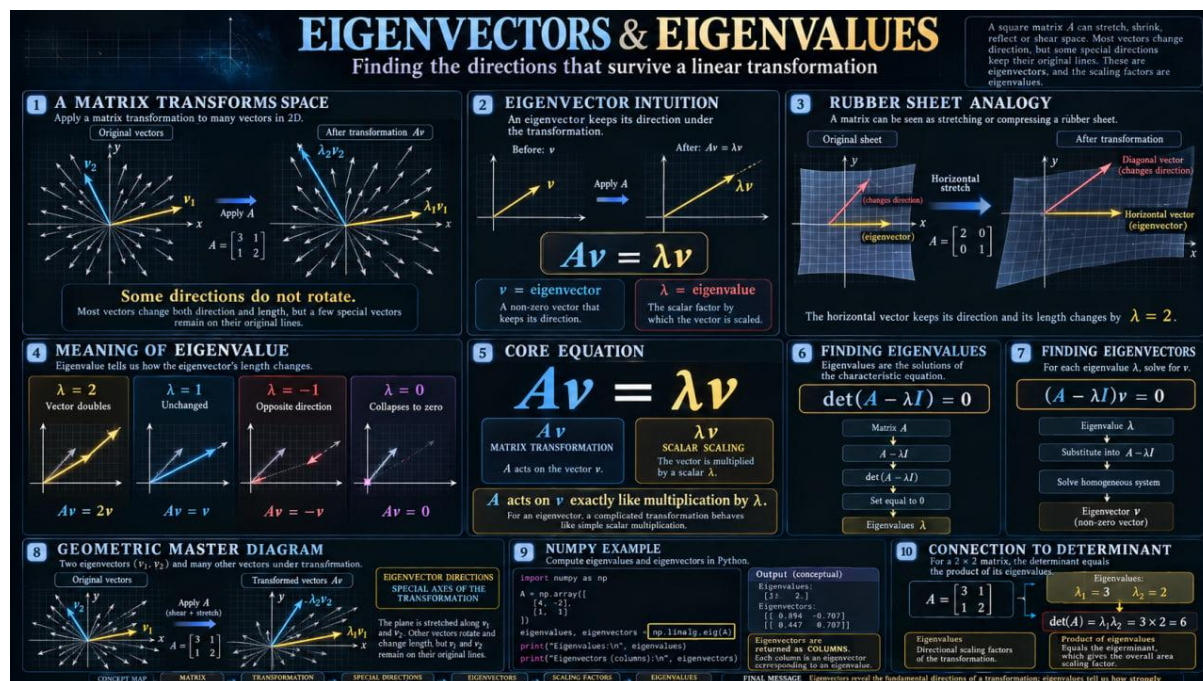
o'zining dastlabki yo'nalishidan (Span chizig'idan) adashib, boshqa yo'nalishga burilib ketadi.

Lekin... Bu g'ala-g'ovur ichida shunday bir **Sehrli Vektorlar** topiladiki, transformatsiya ularni qanchalik qiynamasin, ular o'z yo'nalishidan bir millimetr ham burilmaydi! Matritsa ularni yo shunchaki uzunroq qilib cho'zadi, yoki qisqartiradi, yoki teskari tomonga o'stiradi, LAKIN ular o'zlari yotgan to'g'ri chiziqdan aslo tashqariga burilmaydi. Aynan mana shu o'z mustaqilligini, o'z chizig'ini (yo'nalishini) yo'qotmaydigan maxsus vektorlar — **Xos Vektorlar (Eigenvectors)** deb ataladi!

Va matritsa ularni necha barobarga cho'zgani (yoki qisqartirgani) ko'rsatkichi — bu uning **Xos Qiymati (Eigenvalue, λ - Lambda)** deb ataladi.

Hayotiy Analogiya (Kauchuk plyonka): Tasavvur qiling, siz rezina (kauchuk) varaqni ikkita chetidan ushlab faqat Gorizontal yo'nalishda tortib cho'zyapsiz.

- Agar siz shu varaqqa qiya, diagonal chiziq tortgan bo'lsangiz, varaqni cho'zganingizda o'sha qiya chiziqning burchagi o'zgarib ketadi, u gorizontal tomonga og'adi (Bu oddiy vektor, burilib ketdi).
- Lekin, agar siz to'g'ri X o'qi bo'ylab gorizontal chiziq tortgan bo'lsangiz, siz qancha tortmang, u chiziq burilmaydi, u faqat o'sha o'zining chizig'i bo'ylab uzunlashadi xolos. (Mana shu rezina varaqning **Xos Vektori!**).



4-QISM: Matematik Tenglama va Yechim Mantiqi

Yuqoridagi butun boshli falsafani matematika bittagina nafis, oddiy va shohona tenglamaga joylashtirgan:

$$A \cdot \vec{v} = \lambda \cdot \vec{v}$$

Keling, bu mo'jizaviy tenglamani birgalikda o'qiymiz:

- **Chap tomon ($A \cdot \vec{v}$):** Bu biz bilgan Murakkab Matritsalar olami. Biz qandaydir A matritsani bir $\vec{v}$ vektoriga ko'paytiraymiz (Transformatsiya qilyapmiz). Odatda matritsa vektorni qiyshaytirishi, aylantirishi kerak.
- **O'ng tomon ($\lambda \cdot \vec{v}$):** Bu esa Skalyarlar (Oddiy sonlar) olami! Biz o'sha bir xil $\vec{v}$ vektorini shunchaki bitta yakka raqamga (λ) ko'paytiraymiz (Bu amal vektorni burmaydi, faqat o'stirib-qisadi).

Tenglikning ma'nosi: Shunday maxsus $\vec{v}$ vektorlar borki, ular uchun A degan daxshatli matritsaning ta'siri bilan bitta oddiy λ (Lambda) raqamining ta'siri mutlaqo **bir xil natija beradi!** Transformatsiya bu vektorlar uchun shunchaki skalyar ko'paytirish darajasiga tushib qoladi.

- Bu yerdagi $\vec{v}$ — bu bizning **Eigenvector**.
- Bu yerdagi λ (Lambda) — bu bizning **Eigenvalue**.

Agar $\lambda = 2$ bo'lsa, xos vektor o'z yo'nalishi bo'ylab 2 marta cho'ziladi. Agar $\lambda = 1$ bo'lsa, xos vektor umuman o'zgarmaydi. Agar $\lambda = -1$ bo'lsa, xos vektor xuddi o'sha chiziqda, lekin orqaga (180 gradusga) qarab o'zgaradi. Agar $\lambda = 0$ bo'lsa, bu vektor butunlay yeb yuboriladi (nolga aylanadi). (Aytgancha, agar hech bo'lmasa bitta $\lambda = 0$ bo'lsa, demak faza qisilgan va uning Determinanti ham 0 ga teng bo'ladi!).

Matematikada ularni qanday topishadi? Odatda bu $\det(A - \lambda I) = 0$ ko'rinishidagi Xarakteristik Tenglama orqali qo'lda yechiladi. Siz undan kvadratik yoki kubik tenglama hosil qilib, λ ning ildizlarini topasiz. Lekin ML mutaxassisi sifatida siz buni qo'lda yechmaysiz. Buni kompyuter yashin tezligida yechadi.

5-QISM: Pythonda Eigenvalues va Eigenvectors (NumPy mo'jizasi)

Pythonda **numpy** kutubxonasining kichik qismi bo'lgan **linalg** (Linear Algebra) orqali buni bir qatorda yechish mumkin. Keling, buni sinab ko'ramiz.

Aytaylik, bizda fazoni siljituvchi A matritsasi bor:

$$A = \begin{bmatrix} 4 & -2 \\ 1 & 1 \end{bmatrix}$$

```
import numpy as np
```

```
# Matritsani yaratamiz
```

```
A = np.array([[4, -2],
               [1, 1]])
```

```
# Sehrli kod: eig() funksiyasi bizga ham Lambda, ham Vektorlarni qaytaradi
eigenvalues, eigenvectors = np.linalg.eig(A)
```

```
print("Xos Qiymatlar (Eigenvalues / Lambda):")
print(eigenvalues)
```

```
print("\nXos Vektorlar (Eigenvectors):")
print(eigenvectors)
```

Natija tahlili: Sizga Python ikkita xos qiymat qaytaradi: masalan $\lambda_1 = 3$ va $\lambda_2 = 2$. Va **eigenvectors** degan yangi matritsa qaytaradi. Uning *ustunlari* (qatorlari emas) — o'sha biz izlagan mustaqil $\vec{v}_1$ va $\vec{v}_2$ o'qlari bo'ladi. Demak, bu A matritsa fazoni shunday ezyaptiki, u fazodagi ikkita pinhona o'qlarni topib olib, birini 3 marta, ikkinchisini esa 2 marta cho'zmoqda. Boshqa hamma yo'nalishdagi vektorlar shu o'qlarga moslashib qiyshayib ketadi.

6-QISM: Data Science uchun eng katta Sovg'a - PCA (Principal Component Analysis)

Hozir miyangizda "Xo'p, shunday o'zgarmas chiziqlar va ularni cho'zuvchi raqamlar bor ekan, ML dagi uylar narxi yoki kasalliklarni aniqlash kabi datalar orasida buni menga nima keragi bor?" degan haqli savol aylanyapti. Eslaysizmi, Statistika bo'limining 5-darsida "Kovariatsiya Matritsasi" degan narsa haqida gaplashgan edik?

Tasavvur qiling, sizda 10,000 ta ustundan iborat rasm (yoki matn) dataseti bor. Uning shakli 10,000 O'lchamli fazodagi bulut (nuqtalar to'plami) kabi. Ularning ichida keraksiz, bir-birini takrorlaydigan ustunlar (feature'lar) juda ko'p. Siz ularni ML modelga bersangiz, kompyuteringiz qotib qoladi (Curse of Dimensionality). Biz ma'lumotning asl mohiyatini saqlab qolgan holda, o'lchamni 10,000 dan 50 ga tushirishimiz kerak.

Bu muammoni **PCA (Asosiy Komponentlar Tahlili)** deb nomlangan super-algoritm yechadi. U qanday ishlaydi?

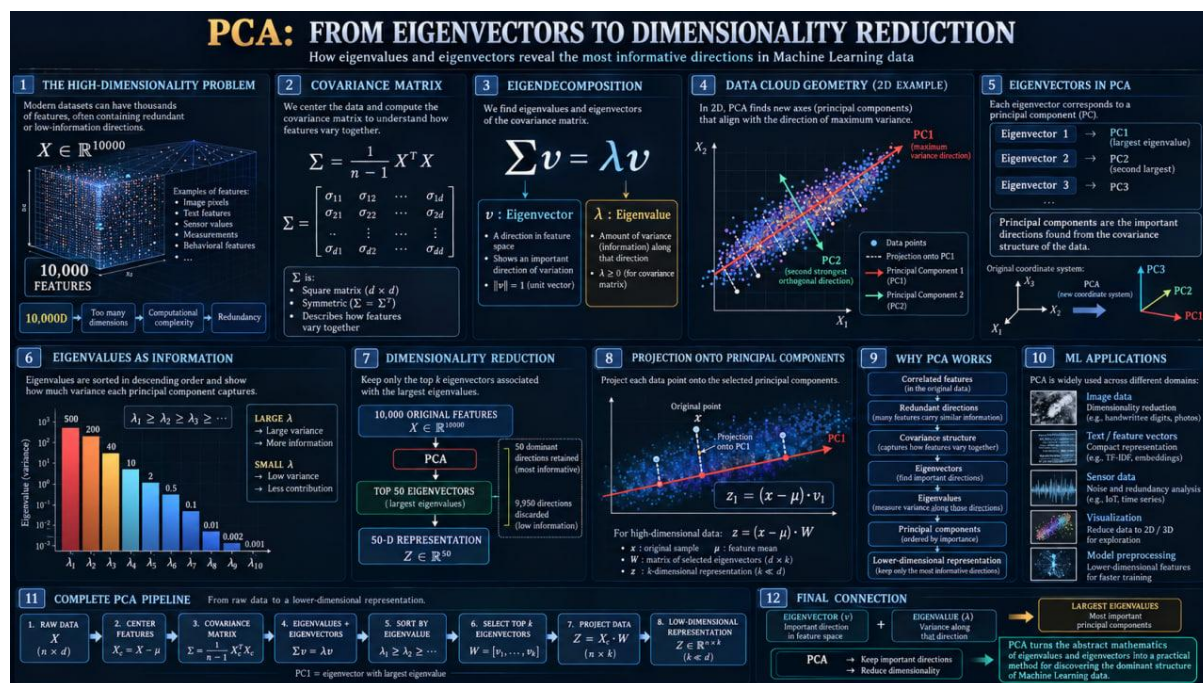
1. Avval siz barcha 10,000 ta ustuningiz qanday qilib o'zaro bir-biri bilan birgalikda o'zgarishini ko'rsatuvchi **Kovariatsiya Matritsasini (Covariance Matrix - Σ)** hisoblaysiz. (U kvadrat va simmetrik matritsa bo'ladi).
2. Keyin siz O'SHA Kovariatsiya Matritsasining **Eigenvalue va Eigenvectorlarini** ([np.linalg.eig](#) orqali) hisoblaysiz! Mo'jiza shu yerda yuz beradi!

PCA dagi Xos Vektorlarning (Eigenvectors) ma'nosi: Data bulutingiz qandaydir g'alati qiya shaklda cho'zilib tarqalgan bo'ladi. Kovariatsiya matritsasining Xos Vektorlari — bu dataning shunchaki o'qlari emas, ular ma'lumot eng ko'p tarqalgan, **eng ko'p ma'lumot (Variance) yashiringan haqiqiy markaziy "Skelet" o'qlaridir!** Birinchi Xos Vektor (PC1) - ma'lumot bulutini eng uzunlik boylab yorib o'tuvchi o'q. Ikkinchi Xos Vektor (PC2) - unga perpendikulyar bo'lgan ikkinchi eng muhim o'q, va hokazo.

PCA dagi Xos Qiymatlarning (Eigenvalues) ma'nosi: Xos qiymatlar (λ) esa, o'sha har bir yorib o'tuvchi o'qda (Xos vektorda) **bizning ma'lumotimizning jami Variatsiyasidan necha foizi yotganligini** (axborotning qadrini) o'lchab beruvchi tarozi hisoblanadi! Agar 1-o'qning xos qiymati $\lambda_1 = 500$, ikkinchisini $\lambda_2 = 200$, 10-chisini $\lambda_{10} = 0.001$ chiqsa nima qilasz? Siz zudlik bilan tushunasizki: "Aha! Jami axborotning deyarli hammasi eng katta Xos

Qiyomatga ega bo'lgan dastlabki bir nechta Xos Vektorlarga yig'ilgan ekan! Qolgan minglab Xos qiymati nollarga teng bo'lgan o'qlar bular shunchaki ma'lumotsiz shovqin (Noise) va takrorlanish ekan!".

Siz shunchaki eng kattadan kichikgacha **eng katta Eigenvalue ga ega bo'lgan 50 ta Eigenvector ni ajratib olasiz** va ulardan o'zingizning yangi, toza, ixcham Baza koordinatangizni (Basis) yasaysiz. Qolgan 9,950 ta o'qni axlatga tashlab yuborasiz! Natijada modelingiz asl (eng muhim) ma'lumotni hech qanday ortiqcha shovqinsiz va yuzlab marta tezroq o'rganadi! Bularning bari shu oddiy $Av = \lambda v$ tenglamasining kuchi orqali ishlaydi.



Xulosa

Mana do'stim, bugun siz shunchaki matritsalarini hisoblash emas, balki ularning "ruhini" ko'rishni o'rgandingiz.

- Determinant** — bu transformatsiya natijasida butun boshli fazoning hajmi necha marta kattalashgani yoki qisilganini (yoki nolgacha ezilganini) bildiruvchi universal o'lchov ekanligini kashf etdingiz.
- Eigenvectors (Xos vektorlar)** — koinot qanday chayqalmasin, o'zining tayanch chizig'ida qattiq turadigan mustahkam o'qlar (ko'prik ustunlari) ekanligini anglab yetdingiz.
- Eigenvalues (Xos qiymatlar)** — o'sha ustunlarning qanchalik cho'zilishi yoki qisqarishini o'lchaydigan kuchaytirgichlar ekanini bildingiz.
- Eng muhimi, bular shunchaki quruq fan emas, balki sun'iy intellektga **PCA algoritmi** orqali trillionlab megabayt ma'lumotlar ichidan faqat "eng muhim" mag'zini topib olishiga imkon beruvchi yagona usul ekanligini dildan tushundingiz.

Chiziqli algebraning eng abstrakt va fundamental darslaridan birini muvaffaqiyatli yakunladingiz. Keyingi darsda (5-Dars) biz bu vektorlar, transformatsiyalar kompyuter

ekranidagi Haqiqiy Rasmlarni (Images) qanday qilib o'zgartirishi (Image Augmentation) ni ko'rib chiqamiz. O'z kashfiyotlaringizdan zavqlaning, biz yanada yuqoriga parvoz qilamiz!

ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)

5-Dars: Vektor Transformatsiyalari va Rasmlar (Vector Transformations and Images)

Assalomu alaykum, aziz Data Scientist va Kelajak Sun'iy Intellect Muhandisi! Bizning Chiziqli Algebra olamidagi sayohatimiz o'zining eng go'zal va eng amaliy bekatlaridan biriga yetib keldi.

Shu paytgacha biz Vektorlar va Matritsalar haqida gaplashdik. Ularni qo'shdik, ko'paytirdik, determinantini topdik. Lekin ular aslida ko'zga qanday ko'rinadi? Bugungi darsda biz matritsalar shunchaki "raqamlar qutisi" emas, balki fazoni harakatga keltiruvchi, uni o'zgartiruvchi "**Harakat dvigatellari**" (**Transformations**) ekanligini vizual tarzda ko'ramiz. Va eng muhimi, bugun biz matematika qanday qilib **Computer Vision (Kompyuterli Ko'rish)** texnologiyasiga aylanishini, kompyuter ekranidagi rasmlar aslida nima ekanligini kashf etamiz.

Agar siz yuzni taniydigan (Face Recognition), xavfli o'smalarni rentgen rasmlaridan topadigan yoki avtopilot (Self-driving) avtomobillarini yaratmoqchi bo'lsangiz, bugungi dars sizning eng yaqin do'stingizga aylanadi. Tayyor bo'lsangiz, tasavvur ko'zlaringizni oching, biz boshladik!

1-QISM: Chiziqli Transformatsiya nima? (Linear Transformation)

"Transformatsiya" so'zi inglizchadan tarjima qilinganda "O'zgartirish, shaklini almashtirish" degan ma'noni bildiradi. Chiziqli algebrada **Transformatsiya** bu aslida biz maktabdan biladigan oddiy **Funksiya (Function)** ning o'zginasidir.

Esingizdami, $f(x) = 2x$ degan funksiya bo'lardi. Biz unga qandaydir $x = 3$ sonini kiritar edik, u esa bizga yangi 6 sonini chiqarib berardi (son o'zgarardi). Transformatsiya (kengroq qilib aytsak, Chiziqli Transformatsiya) xuddi shunday ishlaydi, faqat u yakkama-yakka sonlarni emas, balki butun boshli **Vektorlarni** qabul qiladi va ularni yangi **Vektorlarga** o'zgartirib chiqarib beradi!

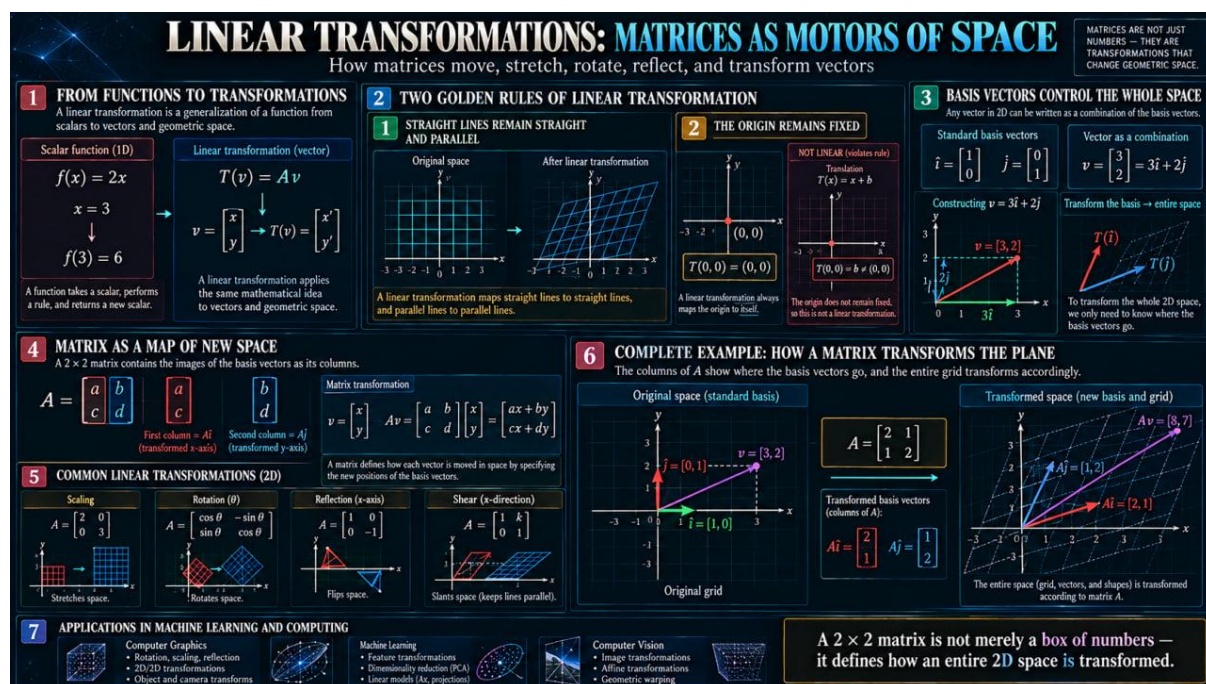
Biz buni matematikada $T(\vec{v})$ deb yozamiz. Ya'ni transformatsiya qilingan vektor.

Chiziqli Transformatsiyaning 2 ta qat'iy oltin qoidasi bor: Qandaydir o'zgarish "Chiziqli (Linear)" bo'lishi uchun, u fazoni (koinotni) qanchalik tortib, cho'zib, burmasin, quyidagi ikki qoida aslo buzilmasligi shart:

1. **Barcha to'g'ri chiziqlar to'g'riligicha qolishi kerak.** (Grid lines remain straight and parallel). Ular qiyshayib, yoy yoki to'lqin shakliga kirib qolmasligi kerak.

2. **Koordinata boshi (Origin - 0,0 nuqta) joyidan jilmasligi kerak.** Hamma narsa o'zgarsa ham, 0 nuqtasi o'z o'rnida qotib turishi shart.

Agar shu ikki shart bajarilsa, bunday harakatni (transformatsiyani) har doim bitta **Matritsa** orqali yozib, ifodalash mumkin!



2-QISM: Transformatsiyaning Matritsali Ifodasi (Matrix Representation)

Qanday qilib bitta oddiy 2×2 matritsa butun boshli cheksiz koinotni (2D tekislikni) burishi yoki qisishi mumkin? Buni tushunish uchun bizga yana o'sha qadrdon baza vektorlarimiz — $\hat{i}$

(X o'qi, $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$) va $\hat{j}$ (Y o'qi, $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$) yordamga keladi.

Tekislikdagi har qanday vektor (masalan $\vec{v} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$), aynan shu ikkita kichkina tayanch vektorning chiziqli kombinatsiyasidan yasaladi ($3\hat{i} + 2\hat{j}$). Shuning uchun, koinotni o'zgartirish uchun hamma nuqtalarni birma-bir o'zgartirib chiqish shart emas! Shunchaki, transformatsiyadan keyin o'sha ikkita $\hat{i}$ va $\hat{j}$ **qayerga borib tushganini** bilsak kifoya! Qolgan hamma vektorlar ularga ergashib avtomatik ravishda to'g'ri joylashib oladi.

Kashfiyot: Har qanday 2×2 matritsa, masalan $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$, aslida transformatsiyadan keyingi yangi koinotning xaritasidir!

- Birinchi ustun $\begin{bmatrix} a \\ c \end{bmatrix}$ — bu o'zgartirilgan (yangi) $\hat{i}$ vektorning qayerga tushganini ko'rsatadi.

- Ikkinchi ustun $\begin{bmatrix} b \\ d \end{bmatrix}$ — bu o'zgartirilgan (yangi) $\hat{j}$ vektorning qayerga tushganini ko'rsatadi.

Mana nima uchun biz Vektorni Matritsaga ko'paytiramiz ($A \cdot \vec{v}$)! Bu ko'paytirish aslida: *"Ey vektor, sen eski koinotda eding, endi men senga yangi A koinotning koordinatalarini beraman, o'sha yoqqa ko'chib o't!"* degan buyruqdir.

3-QISM: 4 ta Asosiy Transformatsiya Turlari (The Big Four)

Machine Learning va Kompyuter Grafikasida asosan 4 turdagi bazaviy harakatlar (transformatsiyalar) keng ishlatiladi. Keling, ularning har biri qanday matritsa orqali yasashini ko'rib chiqamiz.

1. Scaling (Cho'zish va Qisish)

Scaling — bu fazoni X yoki Y o'qlari (yoki ikkalasi) bo'ylab tortib kattalashtirish yoki qisib kichraytirishdir. U har doim **Diagonal Matritsa** orqali amalga oshiriladi (3-darsda biz Diagonal matritsaning yagona ishi o'lchovni o'zgartirish ekanligini aytgandik).

Agar biz X o'qini s_x marta, Y o'qini esa s_y marta kattalashtirmoqchi bo'lsak, scaling matritsasi bunday bo'ladi:

$$S = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} S = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

Misol: Agar rasmni eniga (X bo'ylab) 2 marta kattalashtirib, bo'yini (Y) esa o'zgarishsiz (1)

qoldirmoqchi bo'lsak: $S = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$. Bu rasmni yassilab, yon tomonga qiya semirtirib yuboradi. Agar ikkala raqam ham bir xil bo'lsa (masalan 2 va 2), rasm mutanosib ravishda hamma tomondan kattalashadi (Uniform scaling).

2. Reflection (Akslantirish / Ko'zgu effekti)

Reflection — bu fazoni ma'lum bir o'qqa nisbatan huddi ko'zgu kabi ag'darishdir. Masalan, siz telefoningizda selfi tushsangiz, u odatda Y o'qiga nisbatan o'zgaradi (chap o'ngga, o'ng chapga aylanadi).

- **X o'qi bo'ylab akslantirish (tepasi pastga tushadi):** X o'qi joyida qoladi ($\hat{i}$ o'zgarmaydi: 1,0), lekin Y o'qi ($\hat{j}$) pastga qarab, manfiy tomonga o'giriladi (0,-1).

$$Ref_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} Ref_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

- **Y o'qi bo'ylab akslantirish (chap o'ngga almashadi):**

$$Ref_y = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

(Bu operatsiyani bajarganda, 4-darsda aytganimizdek Matritsaning Determinanti manfiy bo'lib qoladi, chunki olam "ichkaridan tashqariga to'ntarildi!").

$$Ref_y = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

(Bu operatsiyani bajarganda, 4-darsda aytganimizdek Matritsaning Determinanti manfiy bo'lib qoladi, chunki olam "ichkaridan tashqariga to'ntarildi!").

3. Rotation (Burish / Aylantirish)

Rasmni yoki fazoni markaz atrofida ma'lum bir θ (Teta) burchakka aylantirish. Bu eng chiroyli matritsalaridan biridir. Maktabdagi trigonometriyani eslaysizmi? Sinus va Kosinuslar aynan aylana harakatlarini tavsiflash uchun ishlatiladi.

θ burchakka (soat miliga qarshi) buruvchi matritsa:

$$R(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} R(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

Misol: Butun fazoni 90 gradusga (90°) burmoqchimiz. Bizamizki, $\cos(90^\circ) = 0$ va $\sin(90^\circ) = 1$

$$R(90^\circ) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

1. Formulaga qo'yamiz: Bu matritsa har qanday vektorni olib, uni aniq 90 gradusga tikka holatga (chapga) burib qo'yadi.

4. Shear (Siljitish / Qiya qilish)

Siz qalin kitobni stolga qo'yib, uning ustki muqovasidan ushlab yonga sekin itarsangiz nima bo'ladi? Kitobning qatlami qiya shaklga kiradi (parallelogramm bo'lib qoladi). Lekin uning balandligi o'zgarmaydi. Aynan mana shu harakat Geometriyada **Shear (Qiya qilish)** deyiladi.

X o'qi bo'ylab k koeffitsientga qiya qiluvchi matritsa:

$$Sh_x = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix} Sh_x = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$$

Bu holatda, X o'qi (i) joyida turadi, lekin Y o'qining (j) uchi asta-sekin X o'qi tomonga k qadam og'a boshlaydi.

THE BIG FOUR: FUNDAMENTAL VECTOR TRANSFORMATIONS
Scaling, reflection, rotation, and shear — the geometric operations behind computer vision

These four linear transformations are the building blocks of more complex transformations used in computer graphics, computer vision, and machine learning.

1 SCALING — Change the size along each axis

Scaling matrix (2D): $S = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$

Original (unit square)

Scale X by 2 (stretch horizontally): $S = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$

Uniform scaling by 2 (enlarge both axes): $S = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$

Diagonal matrices control independent axis scaling.

$X \rightarrow 2x$ $Y \rightarrow \text{unchanged}$

$X \rightarrow 2x$ $Y \rightarrow 2y$

VECTOR / IMAGE

A 2D vector (or image) can be transformed using linear operations.

$v = \begin{bmatrix} x \\ y \end{bmatrix}$

2 REFLECTION — Mirror across an axis (reverse orientation)

Reflection across x-axis (flip vertically): $Ref_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

Reflection across y-axis (flip horizontally): $Ref_y = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$

Properties:

- $\det(Ref_x) = -1 < 0$
- $\det(Ref_y) = -1 < 0$

Reflection reverses orientation (changes the "handedness").

Original

Reflected

$\det(\text{Reflection}) < 0$

3 ROTATION — Rotate around the origin by an angle θ

Rotation matrix (2D): $R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$

Geometric view (counter-clockwise rotation)

$\hat{j} = [0, 1]$

$\hat{i} = [1, 0]$

90° counter-clockwise rotation: $R(90^\circ) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$

$\hat{i} = [1, 0] \rightarrow [0, 1]$

$\hat{j} = [0, 1] \rightarrow [-1, 0]$

90° COUNTER-CLOCKWISE

$\hat{j} = [0, 1] \rightarrow [1, 0]$

Preserves length and angles. Only changes the direction.

$\det(R(\theta)) = 1$

4 SHEAR — Slant the shape (change the angle)

x-shear matrix (horizontal shear): $Sh_x = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$

Example: $k = 1$

General form: $Sh_x = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$

$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x + ky \\ y \end{bmatrix}$

Shear changes the angle/shape while preserving the vertical height (in this x-shear example).

$\det(Sh_x) = 1$

Top edge shifts horizontally

Bottom edge remains fixed

5 TRANSFORMATION COMPOSITION — Apply multiple transformations in sequence

Original vector

1. Rotate (R)

2. Scale (S)

Final vector

$v_i = R \cdot v$

$v_{new} = S \cdot v_i$

$v_{new} = S \cdot R \cdot v$

Operations are applied from RIGHT to LEFT. Operations closest to the vector happen first.

6 SUPER MATRIX — Combine transformations

Multiple transformations can be combined into a single matrix.

$C = S \cdot R$

Then apply: $v_{new} = C \cdot v = S \cdot R \cdot v$

Composition is matrix multiplication. The result is still a linear transformation.

7 ORDER MATTERS — Matrix multiplication is not commutative

Path A: Rotate $\rightarrow$ Scale ($R \rightarrow S$)

Path B: Scale $\rightarrow$ Rotate ($S \rightarrow R$)

$S \cdot R \neq R \cdot S$

Matrix multiplication is generally non-commutative. Different orders produce different final transformations.

$v_A = S \cdot R \cdot v$

$v_B = R \cdot S \cdot v$

8 FINAL CONCEPT MAP

- SCALING $\rightarrow$ size (stretch/shrink)
- REFLECTION $\rightarrow$ orientation (mirror)
- ROTATION $\rightarrow$ angle (direction)
- SHEAR $\rightarrow$ slant (change shape)
- COMPOSITION $\rightarrow$ multiple transformations
- MATRIX MULTIPLICATION $\rightarrow$ transformation pipeline

The Big Four transformations are the fundamental building blocks for computer vision. Complex transformations, image augmentations, and neural network geometric operations are built from combinations of scaling, reflection, rotation, and shear.

4-QISM: Transformatsiyalarni Birlashtirish (Composition of Transformations)

Hayotda biz bitta rasmni bitta amalda ham burib, ham kattalashtirib, ham qiya qilib qo'yishimiz mumkin. Chiziqli algebrada ko'plab transformatsiyalarni ketma-ket qo'llash — **Matritsalarini Ko'paytirish (Matrix Multiplication)** orqali amalga oshiriladi.

Tasavvur qiling, sizda $\vec{v}$ vektor bor. Siz uni avval 90° ga burmoqchisiz (R matritsa). Keyin uni 3 marta kattalashtirmoqchisiz (S matritsa).

Biz buni ketma-ket yozamiz. **DIQQAT:** Matritsalar doim o'ngdan chapga qarab o'qiladi (amallar eng o'ngdan, vektorga eng yaqin joydan boshlanadi)! **Yakuniy amal:** $\vec{v}_{yangi} = S \cdot R \cdot \vec{v}$ Agar siz birinchi o'rinda S va R matritsalarini bir-biriga ko'paytirib, bitta yangi (Super) Matritsa ($C = S \cdot R$) yasab olsangiz, o'sha bitta C matritsasi sizga IKKALA ishni bitta harakatda (bir soniyada) bajarib beradi! Bu kompyuter grafikasida trillionlab hisob-kitoblarni tejaydigan oltin qoidadir. GPU larning butunlay asosi mana shu ustiga qurilgan.

Ogohlantirish: Biz 3-darsda aytgan qoidani eslang! $S \cdot R \neq R \cdot S$. Agar siz avval aylantirib keyin cho'zsangiz, boshqa shakl hosil bo'ladi. Agar avval cho'zib, cho'zilgan narsani aylantirsangiz, mutlaqo boshqa va g'alati shakl kelib chiqadi. Ketma-ketlik koinot qonunlarida juda muhim.

5-QISM: Rasm - bu Matritsa va Pikel Qiymatlari (Image as a Matrix)

Hozirgacha biz abstrakt nuqtalar haqida gapirdik. Xo'sh, bu transformatsiyalar qanday qilib siz hozir o'qib turgan kompyuter ekranidagi Rasmlarga ulanadi? Bir narsani qat'iy anglab oling: **Kompyuter uchun Rasm degan narsa yo'q. Rasm — bu shunchaki yuz minglab raqamlar to'ldirilgan juda ulkan Matritsadir!**

Oq-qora rasmlar (Grayscale): Tasavvur qiling sizda qora va oq rangli 10×10 piksellik kichkina itning rasmi bor. Pikel — bu ekrandagi eng kichik nuqta (katakcha). Kompyuter bu rasmni xuddi 10 qator va 10 ustundan iborat kvadrat Matritsa kabi saqlaydi. Har bir katakchadagi raqam o'sha nuqtaning qanchalik yorug' ekanligini anglatadi. Odatda bu raqamlar 1 Bayt xotira egallaydi, ya'ni ularning qiymati **0 dan 255 gacha** bo'lishi mumkin.

- **0** = Mutlaqo qop-qora (Black).
- **255** = Mutlaqo oppoq (White).
- **128** = O'rtacha kulrang (Gray).

Rangli rasmlar (RGB Images): Sizning telefoningizdagi 1920×1080 o'lchamli rangli rasm qanday saqlanadi? U uchta rangdan yasaladi: Qizil (Red), Yashil (Green), va Ko'k (Blue) — qisqacha RGB. Shuning uchun rangli rasm endi shunchaki 2D Matritsa emas, balki uch qavatli Matritsa bo'ladi. Uchinchi o'lcham (Depth/Channels = 3) paydo bo'ladi. Biz bunday ko'p qavatli va ko'p o'lchamli matritsalarini fanda **Tensorlar (Tensors)** deb ataymiz. (Google ning sun'iy intellekt kutubxonasi nima uchun "TensorFlow" deb atalganini endi tushungandirsiz?).

6-QISM: Image Augmentation (Rasmlarni Ko'paytirish san'ati)

Tasavvur qiling, sizga Bosh direktoringiz "Mushuk va Itlarni ajratadigan Zo'r Neyron tarmog'i" (Deep Learning model) yaratishni buyurdi. Siz Google'dan 1000 ta mushukning rasmini yuklab oldingiz. Hamma rasmlarda mushuklar chiroyli qilib tekis kameraga qarab turibdi. Siz modelni o'qitdingiz. U 100% aniqlik bilan ishlayapti.

Keyin loyihani ishga tushirdingiz. Bir foydalanuvchi o'zining mushugini suratga oldi, lekin u suratni telefonda *teskari* holatda yoki *qiyshayib* qolgan holatda modelga kiritdi. Sizning "mukammal" modelingiz "Bu mushuk emas, bu nima o'zi?" deb xato javob berdi. Nega? Chunki Neyron tarmoqlari o'ta sodda, ular biz kabi mantiqiy fikrlamaydi. Ular uchun 30 gradusga qiyshaygan piksellar to'plami mutlaqo yangi ko'rilmagan ma'lumotdek ko'rinadi (Overfitting muammosi).

Yechim: Modeldan ko'proq aqlli bo'lish uchun sizga **Image Augmentation (Rasmlarni Sun'iy Kattalashtirish)** yordamga keladi! Siz minglab yangi rasmlarni internetdan qidirib yurmaysiz. Siz qo'lingizdagi o'sha 1000 ta rasmni olasiz va kompyuterga buyruq berasiz: *"Har bir mushukning rasmini ol. Uni Linear Transformatsiya orqali:*

- 15 gradusga o'ngga bur (Rotation).
- O'ngdan chapga akslantir (Reflection / Horizontal Flip).
- Ozroq yonga siljit (Shear).
- Ozroq kattalashtirib kesib ol (Zoom/Scaling).

Ana shunda sizning 1000 ta rasmingiz bir lahzada, hech qanday pulsiz va mexanik mehnatsiz **50,000 ta turli xil pozadagi mushuklar** datasetiga aylanadi! Modelingiz endi *teskari* turgan mushukni ham, *yotgan* mushukni ham taniy oladigan "Sog'lom va Bardoshli" (Robust) modelga aylanadi.

Koddagi ko'rinishi: PyTorch yoki TensorFlow kabi kutubxonalarda bu shunday yoziladi:

```
import torchvision.transforms as transforms
```

```
# Matritsa transformatsiyalarini zanjir qilib birlashtirish (Composition)
data_augmentation = transforms.Compose([
    transforms.RandomRotation(degrees=30),      # Rotation Matrix
    transforms.RandomHorizontalFlip(p=0.5),    # Reflection Matrix
    transforms.RandomResizedCrop(size=(224,224)) # Scaling Matrix
])
```

Ko'rib turganingizdek, siz shunchaki tayyor funksiyalarni ishlatyapsiz, ammo ularning *qornida* (*under the hood*) aynan bugun biz o'rgangan $\cos(\theta)$ va $\sin(\theta)$ li Matritsa Ko'paytmalari ishlab, millionlab piksellarni yangi koordinatalarga ko'chirib o'tkazmoqda.

7-QISM: ML va Computer Visionning Yakuniy Bog'liqligi (CNN)

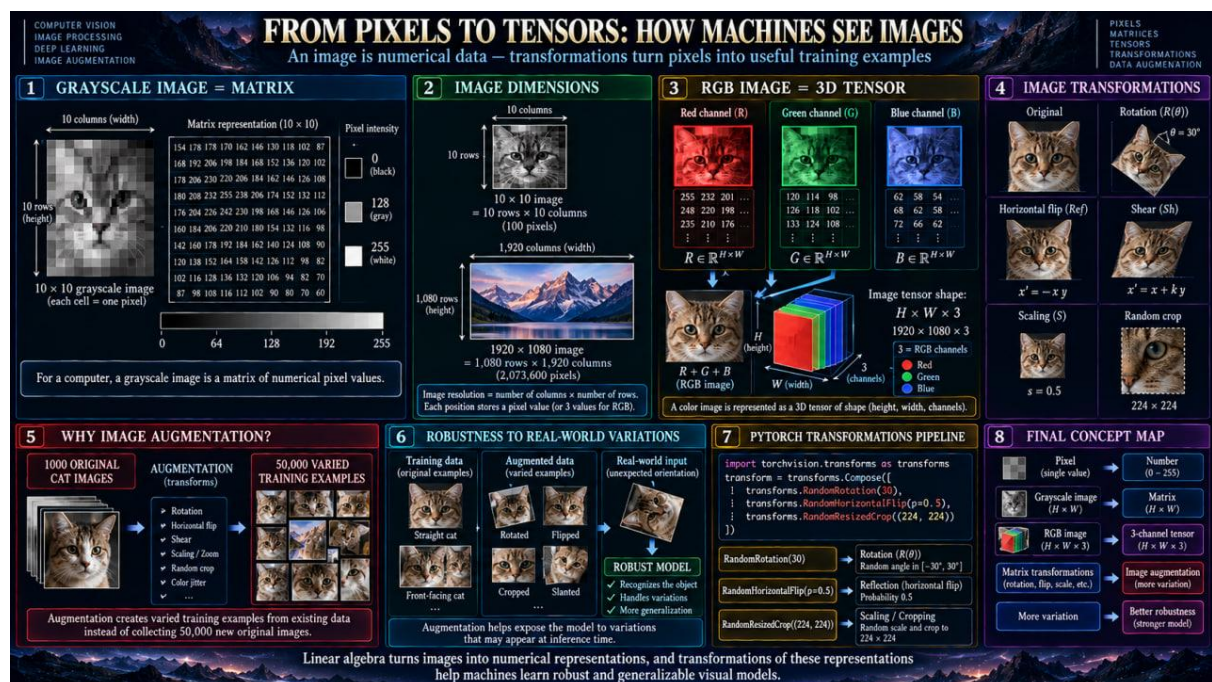
Xo'sh, rasm matritsa ekan. Uni o'zgartirishni ham bildik. Lekin model rasmdagi narsa mushuk ekanligini o'zi qanday qilib fosh qiladi? Qanday qilib quloqlarni, ko'zlarni va yuz shakllarini bilib oladi? Buning siri — **Convolutional Neural Networks (CNN - Konvolutsion Neyron Tarmoqlari)** da yashiringan. (Biz ularni Deep Learning kurslarida batafsil yechamiz, ammo uning yuragi hozir siz bilgan Chiziqli Algebradir).

Konvolutsiya amali nima? Bu ham Aslida Matritsani Matritsaga ko'paytirishdir! Kompyuterda qandaydir maxsus, kichkina 3×3 matritsalar bor. Ularni fanda **Kernels** yoki **Filters (Filtrlar)** deb atashadi.

- Ba'zi filtrlar shunday raqamlardan tuzilganki, u rasmni aylanib chiqqanda, faqatgina rasmning Ichidagi **Vertikal (Tikka) chiziqlarni** topib, yonib qoladi (qolgan joylarni qop-qora qilib qo'yadi).
- Boshqa matritsa faqat **Gorizontal (Yotiq) chiziqlarni** aniqlaydi.
- Yana boshqasi rasmni xiralashtiradi (Blur), boshqasi o'tkirlashtiradi (Sharpen).

Deep Learning modeli o'zining "O'rganish" (Training) jarayonida aynan mana shu 3×3 Filtr Matritsalarining Ichidagi *raqamlarni (weightlarni)* qanday bo'lishi kerakligini o'zi mustaqil qidirib topadi! U millionlab yuzlarni ko'radi, ularni to'g'ri xatolar orqali moslashtiradi va oxirida ko'z, burun, quloqni izlab topadigan o'nlab super-Filtr Matritsalarini yaratadi.

Sizning telefoningizdagi Instagram filtrlari, Yuzingizni ochadigan FaceID, xavfsizlik kameralaridagi mashina raqamini o'qiydigan tizimlar — barchasining qon tomirida **Bitta katta Matritsaning o'zi kabi ko'plab kichkina Matritsalariga skalyar ko'paytirilishi (Dot Product va Matrix Multiplication)** tinimsiz oqib yotibdi.



Xulosa

Mana, aziz bilim izlovchi! Siz bugun Chiziqli Algebra olamining naqadar go'zal va jonli ekanligiga guvoh bo'ldingiz. Biz mavhum bo'lib ko'ringan Matritsalarini — bo'shliqni buruvchi, cho'zuvchi va akslantiruvchi Xarita (Transformatsiya) ekanligini angladik. Biz Rasmlar qanday qilib RGB (0 dan 255 gacha) matritsa to'plamiga aylanishini va bu matritsalarini geometrik transformatsiyalar orqali o'zgartirish (Image Augmentation) Neyron tarmoqlarini "aqlli" va "bardoshli" qilishning yagona yo'li ekanligini kashf qildik.

Shu nuqta bilan biz **Machine Learning uchun Chiziqli Algebra** blokini fundamental darajada muvaffaqiyatli yakunladik. Siz endi ma'lumotlar bilan fazoviy ishlashni bilasiz. Biroq, Sun'iy Intellect yaratishda ikkinchi qanot ham kerak. Model o'zining xatolaridan qanday xulosa chiqarib, tobora aqlli bo'lib borishni o'rganishi uchun unga "O'zgarishlar va Hosilalar" fani kerak. Keyingi, **6-Darsimizdan boshlab**, biz sirli va hayajonli **Kalkulyus (Calculus -**

Matematik Analiz olamiga (Limitlar va Uzluksizlik) sho'ng'iymiz. Hozircha bugungi bilimlaringiz bilan o'z kompyuteringizdagi rasmlarga oddiy rasm emas, balki ajoyib raqamlar dengizi deb qarang. Keyingi darsda ko'rishguncha!

ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)

6-Dars: Limitlar va Uzluksizlik (Limits and Continuity)

Assalomu alaykum, aziz va tolmas Data Scientist!

Siz bilan Chiziqli Algebra bo'limini muvaffaqiyatli yakunlab, Machine Learningning ikkinchi qudratli qanoti — **Kalkulyus (Calculus, ya'ni Matematik Analiz)** olamiga qadam qo'ymoqdamiz. Agar Chiziqli Algebra "Ma'lumotlarni qanday qilib to'g'ri joylashtirish va ko'paytirish mumkin?" degan savolga javob bersa, Kalkulyus "Shu ma'lumotlar o'zgarganda nima bo'ladi? Eng yaxshi (optimal) natijani qanday topamiz?" degan savolga javob beradi.

Sun'iy intellekt (masalan, Neyron tarmoqlari) qanday qilib "o'rganadi"? U o'zining xatolarini (Loss function) hisoblaydi va bu xatoni doimiy ravishda nolga qarab **kamaytirib boradi (pasayadi)**. Aynan shu "nolga qarab intilish", "sekin-asta o'zgarish" va "eng pastki nuqtani topish" jarayonlarining barchasi Kalkulyus qonunlariga asoslangan.

Kalkulyusning barcha mo'jizalari (Hosila va Integrallar) yagona, mitti, lekin o'ta qudratli bitta tushuncha ustiga qurilgan. Bu — **Limit (Chegara / Intilish)** dir. Bugungi darsda biz Limitlar nima ekanligini, Uzluksizlik (Continuity) nega ML modellar o'lishi yoki yashab qolishini hal qilishini batafsil, har bir ipidan tortib o'rganamiz. Qahvangizni hozirlang, biz harakat (dinamika) olamiga kirdik!

1-QISM: Limit nima o'zi? (The Concept of a Limit)

Limit so'zi hayotda "Chegara" degan ma'noni bildiradi. Ammo matematikada u biroz boshqacharoq ma'noga ega: **"Limit — bu qandaydir nuqtaga o'ta yaqinlashganimizda (lekin uning ustiga aniq chiqmaganimizda) nima yuz berishini kuzatishdir"**.

Nima uchun biz nuqtaning o'zini o'rganmaymiz-u, unga yaqinlashishni o'rganamiz? Buni bitta klassik muammo orqali tushuntiraman. Aytaylik, bizda quyidagi funksiya bor:

$$f(x) = \frac{x^2 - 4}{x - 2}$$

Biz $x = 2$ bo'lganda funksiyaning qiymati (javobi) necha bo'lishini bilmogchimiz. Keling, x ning o'rniga 2 ni qo'yib ko'ramiz: Tepasi (Surati): $2^2 - 4 = 4 - 4 = 0$. Pasti (Maxraji): $2 - 2 = 0$. Javob: $0/0$. Matematikada nolni nolga bo'lish (yoki umuman har qanday sonni nolga bo'lish) aniqlanmagan, mumkin bo'lmagan amaldir. Kompyuter dasturida buni qilsangiz **ZeroDivisionError** xatoligini berib, dastur qotib qoladi (crash bo'ladi).

Demak, biz $x = 2$ nuqtada ko'r ekanmiz. Funksiya u yerda nima ish qilayotganini bilmaymiz. Ammo, biz matematiklar osonlikcha taslim bo'lmaymiz! Agar biz 2 ning ustiga chiqa olmasak, unga sekin-asta **juda ham yaqin borib** qarab ko'rsak-chi?

Keling, jadval tuzamiz va x ga 2 ga juda yaqin bo'lgan raqamlarni (kalkulyator orqali) berib ko'ramiz:

Chap tomondan (2 dan kichkina sonlar orqali) yaqinlashamiz:

- Agar $x = 1.9$ bo'lsa, $f(x) = 3.9$
- Agar $x = 1.99$ bo'lsa, $f(x) = 3.99$
- Agar $x = 1.999$ bo'lsa, $f(x) = 3.999$

O'ng tomondan (2 dan kattaroq sonlar orqali) yaqinlashamiz:

- Agar $x = 2.1$ bo'lsa, $f(x) = 4.1$
- Agar $x = 2.01$ bo'lsa, $f(x) = 4.01$
- Agar $x = 2.001$ bo'lsa, $f(x) = 4.001$

Jadvalga qarab qanday xulosa qilasiz? Garchi x aynan 2 ga teng bo'lganda nima bo'lishini biz hech qachon hisoblay olmasak-da, uning javobi ikki tomondan ham **4 ga qarab ketayotganini (intilayotganini)** aniq ko'rib turibmiz!

Ana shu 4 raqami — funksiyaning Limitidir! Biz buni shunday yozamiz:

$$\lim_{x \rightarrow 2} \frac{x^2 - 4}{x - 2} = 4 \lim_{x \rightarrow 2} \frac{x^2 - 4}{x - 2} = 4$$

O'qilishi: x o'zgaruvchi 2 ga intilganda (qarab harakatlanganda), funksiyaning limiti (javobi) 4 ga intiladi.

(Aytgancha, algebraik jihatdan $x^2 - 4$ ni $(x - 2)(x + 2)$ deb yozib, $(x - 2)$ larni qisqartirib yuborsak, $f(x) = x + 2$ qoladi. Endi 2 ni qo'ysak, $2 + 2 = 4$ chiqadi. Limitlar aslini olganda shunday ajoyib algebrik fokuslar mantig'idir).

Asosiy Qoida: Limit nuqtaning o'zida nima bo'layotgani bilan umuman qiziqmaydi. Limit faqat nuqtaning *atrofida* nima bo'layotgani (tendensiya) haqida xabar beradi.

2-QISM: Bir Tomonlama Limitlar (One-sided Limits)

Biz tepada funksiya ham chapdan, ham o'ngdan yaqinlashdik va ikkalasi ham chiroyli qilib bitta joyda — 4 da uchrashdi. Lekin hayot (va funksiyalar) doim ham bunchalik mukammal emas. Ba'zan chap tomondan kelsangiz boshqa joyga, o'ng tomondan kelsangiz mutlaqo boshqa joyga borib qolishingiz mumkin.

Shuning uchun matematikada limitlarni ikkiga bo'lib tahlil qilishadi:

1. **Chap Limit (Left-hand limit):** $x \rightarrow a^-$ Bu yerda "minus" belgisi a sonining manfiy ekanligini bildirmaydi. U " a raqamiga o'zidan kichikroq bo'lgan tomondan (sonlar o'qining chap tomonidan) yaqinlashish" degan ma'noni beradi. Masalan:
 $\lim_{x \rightarrow 2^-} f(x)$. Ya'ni 1.9, 1.99, 1.999 ... orqali kelish.

2. **O'ng Limit (Right-hand limit):** $x \rightarrow a^+$ Bu esa "a raqamiga o'zidan kattaroq bo'lgan tomondan (o'ng tomondan) yaqinlashish" deganidir. Masalan: $\lim_{x \rightarrow 2^+} f(x)$. Ya'ni 2.1, 2.01, 2.001 . . . orqali kelish.

Hayotiy misol: Avtoturargoh (Parking) narxlari. Agar avtomobilingizni 1 soatgacha qo'ysangiz — 5000 so'm. Agar 1 soatdan o'tib, 2 soatgacha qo'ysangiz — 10000 so'm. Tasavvur qiling, siz roppa-rosa 1 soatda chiqib ketmoqchisiz ($x = 1$).

- Agar vaqtga chapdan yaqinlashsak (masalan, 59 minut, 59 minut-u 59 soniya bo'lsa) — siz hali ham 5000 so'm berasiz. Demak, Chap Limit = 5000.
- Agar vaqtga o'ngdan yaqinlashsak (ya'ni 1 soat-u 1 daqiqa, 1 soat-u 1 soniya o'tgan bo'lsa) — tizim sizni ikkinchi soatga o'tdi deb hisoblaydi va 10000 so'm so'raydi. Demak, O'ng Limit = 10000.

Oltin Teorema (Limitning mavjudligi): Qandaydir bitta nuqtada haqiqiy To'laqonli Limit ($\lim_{x \rightarrow a} f(x)$) mavjud bo'lishi uchun bitta qat'iy shart bajarilishi shart: **Chap limit va O'ng limit bir-biriga TENG bo'lishi kerak!** Agar parking misoliga o'xshab chap tomon 5000 ga, o'ng tomon 10000 ga teng bo'lsa, unda $x = 1$ nuqtada Limit **Mavjud Emas (Does Not Exist - DNE)** deb e'lon qilinadi. Chunki kompyuter qaysi javobni tanlashni (qaysi tomonga ishonishni) bilmay chalkashib qoladi.

3-QISM: Cheksiz Limitlar va Asimptotalar (Infinite Limits)

Ba'zan, limitlarni hisoblaganimizda javob ma'lum bir aniq songa (4 ga yoki 5000 ga) qarab emas, balki koinotning eng uzoq chekkalariga — **Cheksizlikka** (∞) qarab uchib ketadi. Yoki aksincha, biz x ni o'zini cheksizlikka jo'natib, funksiya qanday muvozanatga kelishini bilmoqchi bo'lamiz.

Bularni ikki xil holatda ko'ramiz:

A. Limitning javobi Cheksizlikka aylanganda (Asymptotes)

Mashhur bitta funksiyaning ko'raylik: $f(x) = \frac{1}{x}$. Biz x nolga (0 ga) qarab intilganda nima bo'lishini bilmoqchimiz. O'ng tomondan (musbat tomondan) yaqinlashamiz:

- $x = 0.1 \implies 1/0.1 = 10$
- $x = 0.01 \implies 1/0.01 = 100$
- $x = 0.0001 \implies 1/0.0001 = 10,000$

Ko'rib turganingizdek, maxraj qanchalik nolga yaqinlashib kichrayib borsa, jami javob shunchalik dahshatli tezlik bilan Osmonlarga qarab (Plyus Cheksizlikka, $+\infty$) otilib chiqib ketyapti! Bunday paytda grafikda (y o'qiga parallel ravishda) hech qachon kesib o'tib bo'lmaydigan ko'rinmas devor paydo bo'ladi. Bu devor geometriyada **Vertikal Asimptota (Vertical Asymptote)** deb ataladi. (Eslatma: Bu yerda ham o'ng limit $+\infty$, chap limit esa (manfiy sonlar hisobiga) $-\infty$ ga ketadi, shuning uchun 0 nuqtada aniq limit yo'q).

B. x ning o'zi Cheksizlikka intilganda (Limits at Infinity)

Machine Learningda biz ko'pincha modelimiz minglab, millionlab marta takrorlanganda (iteratsiya qilinganda) nima bo'lishini, xatolik muvozanatlashadimi yoki yo'qmi bilmoqchi bo'lamiz. Bu $\lim_{x \rightarrow \infty} f(x)$ deb yoziladi.

Yana o'sha $f(x) = 1/x$ ni olaylik. Endi x ni tobora kattalashtirib boramiz:

- $x = 100 \Rightarrow 1/100 = 0.01$
- $x = 10,000 \Rightarrow 1/10,000 = 0.0001$
- $x = 1,000,000 \Rightarrow 0.000001$

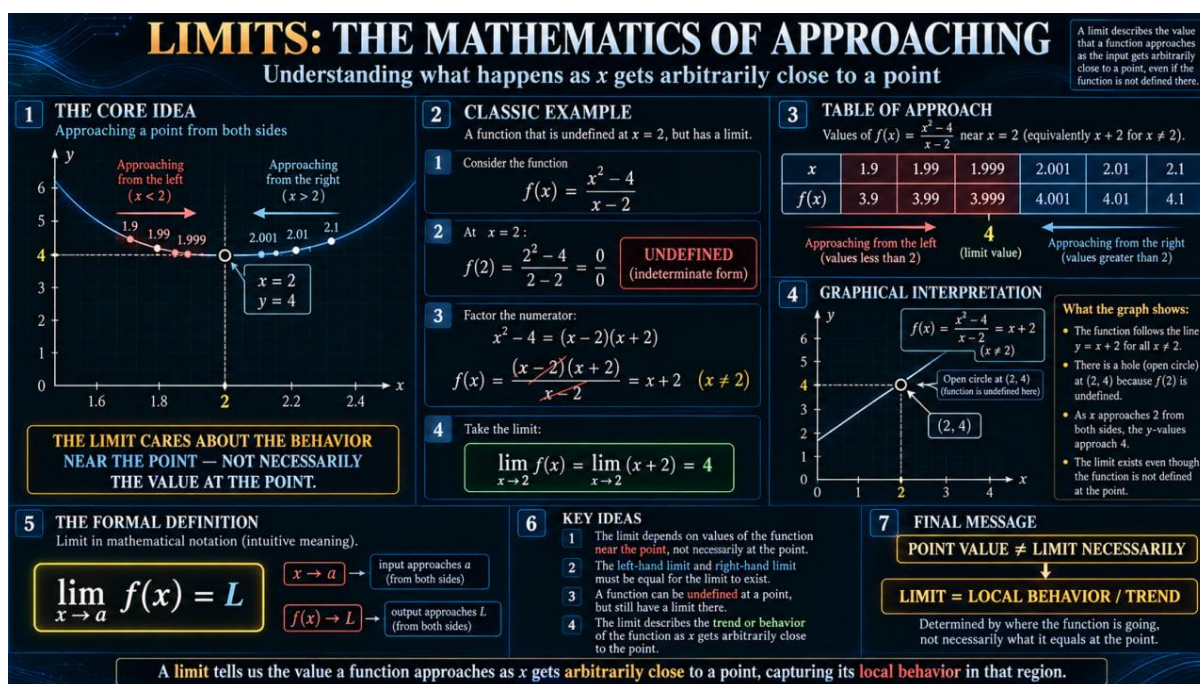
x ni qanchalik cheksizlikka qarab uzoqlashtirmang, funksiya javobi sekin-asta, lekin qat'iy ravishda **Nolga** (0) qarab bormoqda va u hech qachon noldan o'tib, manfiy bo'lib ketmaydi. Bu degani, grafik x o'qiga tobora yopishib borib, muvozanatlashadi. Bu **Horizontal Asimptota** deyiladi.

ML da qo'llanilishi (Sigmoid Funksiyasi): Classification (Masalan, Mushuk yoki Kuchukligini topuvchi Logistic Regression) modellarida eng mashhur bo'lgan Sigmoid funksiyasi bor:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Ushbu funksiya nima nega bunchalik mashhur? Chunki uning Limitlari unga ajoyib "ehtimollik qutisi" (Probability box) vazifasini beradi:

- Agar $x \rightarrow \infty$ (plyus cheksizlik) ga intilsa, funksiyaning limiti 1 ga intiladi.
- Agar $x \rightarrow -\infty$ (minus cheksizlik) ga intilsa, funksiyaning limiti 0 ga intiladi. Natijada, neyron tarmog'i qanchalik katta yoki qanchalik manfiy raqam bermasin, Sigmoid uni doimo chiroyli qilib o'zining chegaralari (0 va 1) orasiga qamab beradi (Bu ehtimollikning 0% dan 100% gacha bo'lish qoidasiga mukammal tushadi!).



4-QISM: Uzlüksizlik (Continuity)

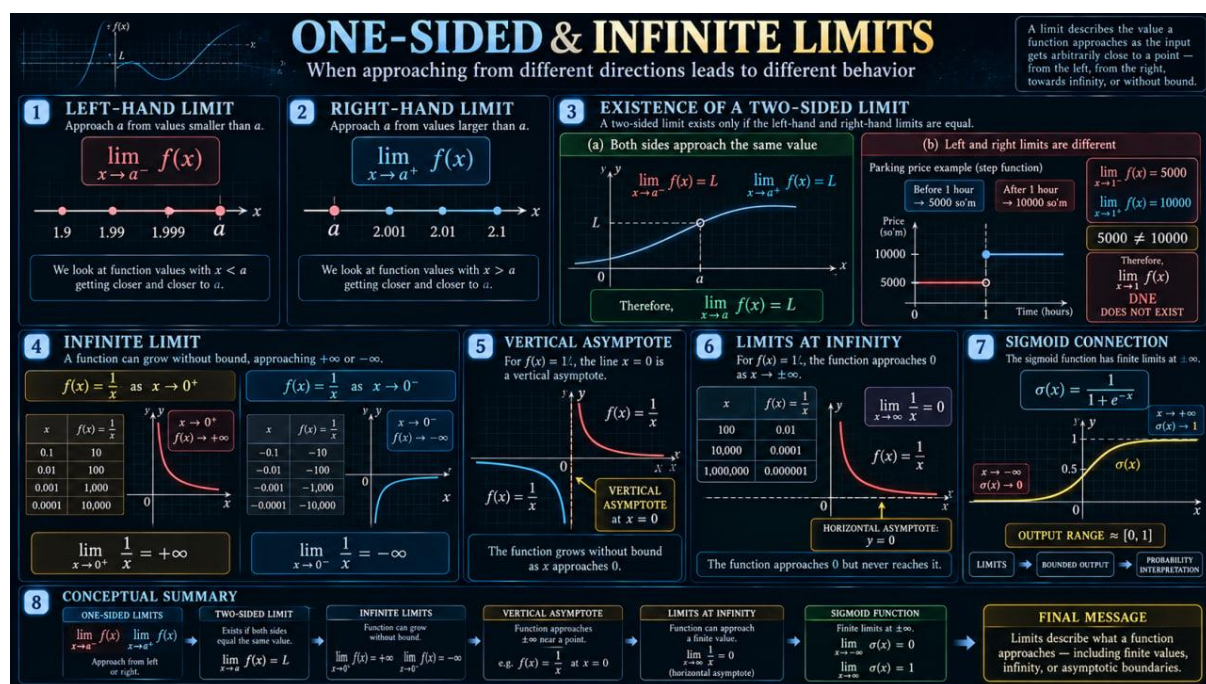
Biz hozirgina limitlar dunyosini (nuqtaga yaqinlashishni) ko'rdik. Ammo bitta funksiya rostdan ham hamma joyda ravon va ishonchli ishlashini qayerdan bilamiz? Bu fanda **Uzlüksizlik (Continuity)** deb ataladi.

Sodda intuitsiya: Qog'oz oling va qalam bilan qandaydir bitta to'lqinsimon grafik (chiziq) chizing. Agar siz butun grafikni qalamning uchini qog'ozdan **UMUMAN KO'TARMASDAN (uzmasdan)** chiza olsangiz, bu funksiya **Uzlüksiz funksiya (Continuous Function)** deyiladi! Agar qayerdadir teshik bo'lsa, qalamni ko'tarib teshikdan o'tib tushishga yoki grafikning boshqa qavatiga sakrashga to'g'ri kelsa, funksiya o'sha joyda **Uzilishga ega (Discontinuous)** bo'ladi.

Matematik ta'rifi (Uzlüksizlikning 3 ta Qat'iy qoidasi): Funksiya $x = a$ nuqtada uzluksiz bo'lishi uchun, u xuddi hujjat topshirayotgandek quyidagi 3 ta imtihondan ketma-ket o'tishi shart:

1. $f(a)$ **aniqlangan bo'lishi kerak.** (Ya'ni, agar o'sha nuqtaga xuddi uyni izlagandek borsangiz, u yerda haqiqatan ham "uy" — aniq bitta qiymat mavjud bo'lishi kerak. Maxraj nol bo'lib xato bermasligi kerak).
2. $\lim_{x \rightarrow a} f(x)$ **mavjud bo'lishi kerak.** (Yuqorida o'rganganimizdek, chapdan kelganda ham, o'ngdan kelganda ham bitta markazga yaqinlashish kerak. Nuqtalar turli tomonga sochilib ketmasligi lozim).
3. **Eng asosiysi: Limitning javobi va Nuqtaning o'zidagi javob aynan bir xil bo'lishi kerak!** Ya'ni, $\lim_{x \rightarrow a} f(x) = f(a)$. (Siz chap va o'ng uzoqdan qarab taxmin qilgan uyingiz, eshigini ochib kirganingizda ichkarida turgan narsa bilan aynan bir xil bo'lishi shart).

Agar mana shu uchta shartdan aqalli bittasi buzilsa, u holda funksiyaning grafik zanjiri uzilgan (Discontinuous) deb topiladi.



5-QISM: Uzilish Turlari (Types of Discontinuity)

Qalamni qog'ozdan uzib olishning har xil sabablari bo'ladi. Matematikada grafiklarda asosan 3 xil uzilish turi yuz beradi. Ularni bilish ML muhandisi uchun juda muhim, chunki modellar bu nuqtalarga kelib qolsa turlicha qotib qolishlari mumkin.

1. Yo'qotish mumkin bo'lgan uzilish (Removable Discontinuity / "Teshik")

Buni ko'pincha **"Point Discontinuity (Nuqtaviy uzilish)"** deb ham aytishadi. Grafik ketyapti, ketyapti, chiroyli silliq... lekin birdaniga o'rtada bitta kichkina nuqta yo'q (teshikcha qolgan). Limit (ikki tomondan yaqinlashish) bor, lekin nuqtaning o'zi yoki o'z joyida emas, yoki grafikning tepasida (alohida nuqta bo'lib) yashaydi.

- **Sababi:** 3-qoidaning buzilishi (Limit mavjud, lekin u funksiyaning o'sha yerdagi asil qiymatiga teng emas).
- **Davosi:** Bu uzilishni davolash oson. Shunchaki o'sha teshikni yamoq qilib o'zimiz xohlagan raqam bilan to'ldirib qo'ysak, yana ravon uzluksiz chiziq paydo bo'ladi. Shuning uchun "Removable" (yo'qotsa bo'ladigan) deyiladi. (Biz 1-qismda ishlagan

$f(x) = \frac{x^2-4}{x-2}$ funksiyasi 2 nuqtasida aynan shunday teshikka ega edi).

2. Sakrashli uzilish (Jump Discontinuity)

Grafik bir darajada (masalan $y=10$) kelib, birdaniga uzilib qoladi-da, mutlaqo boshqa qavatdan (masalan $y=20$) davom etib ketadi. Xuddi pillapoyaga, zinaga o'xshaydi.

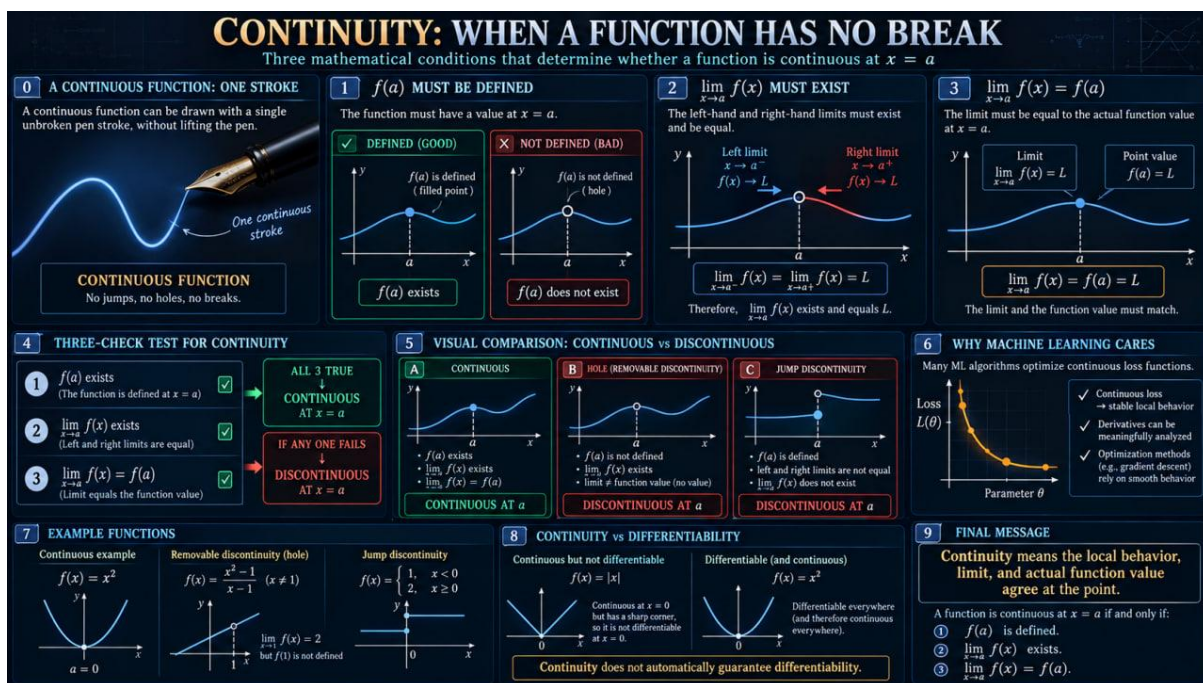
- **Sababi:** 2-qoidaning buzilishi. Chap limit bor, o'ng limit ham bor, lekin ular **bir-biriga teng emas!** Biri 10 da, ikkinchisi 20 da tugadi. (Biz yuqorida ko'rgan Parking misolimiz aynan Jump uzilish edi).

Nima uchun Machine Learningda Jump Discontinuity yomon ko'riladi? Dastlabki Sun'iy neyronlar (1950-yillardagi **Perceptronlar**) aynan "Step Function" (Zinasimon funksiya) ga asoslangan edi. U funksiya $-x$ lar uchun 0 javobni, $+x$ lar uchun esa birdan sakrab 1 javobni berardi. Bu Jump uzilishiga ega funksiya. Lekin muammo shundaki, Gradient Descent (Neyron tarmoqlarning o'rganish mexanizmi) uzilishli joyda hisob-kitob (hosila) ololmaydi! Shuning uchun Olimlar zinasimon funksiyadan voz kechib, uni uzluksiz, silliq ko'tariladigan "Sigmoid" funksiyasiga almashtirishgan. Aynan shu bitta kashfiyot Deep Learningni butunlay inqilob qilib yuborgan!

3. Cheksiz uzilish (Infinite Discontinuity)

Bu yerda qandaydir sakrash yoki teshik emas, umuman jarlik (qora tuynuk) mavjud. Grafik ketyapti va bitta raqamga kelganda birdan osmonga (yoki pastki tubsizlikka) tikka otilib qochib ketadi.

- **Sababi:** Limitning o'zi $+$ yoki $-$ cheksizlikka aylanadi (Asimptota holati). Bu yerda hech qanday raqam haqida gap bo'lishi mumkin emas. (Biz ko'rgan $1/x$ funksiyasining 0 dagi holati aynan Infinite Discontinuity'dir). ML modelni o'qitishda o'zgaruvchilar bunday joyga kelsa (Gradient Explosion), kompyuter **NaN** (Not a Number) degan xato berib, modelni "o'ldiradi". Biz buni oldini olish uchun (masalan Regularization yoki Gradient Clipping) amallardan foydalanamiz.



6-QISM: Limit va Hosila (Derivative) ning Bog'liqligi

Xo'sh, nima uchun biz Limitlarni bunchalik chuqur o'rgandik? Buning siri bitta — Limitlarsiz fanda **Hosila (Derivative)** degan narsa bo'lmas edi! Hosila Kalkulyusning 50% qismini tashkil qiladi. U biron bir funksiyaning (yoki mashina modelining) **O'zgarish Tezligi (Rate of change) va Qiyaligini (Slope)** topib beradi.

Mashinada Toshkentdan Samarqandga (300 km) yo'lga chiqdingiz va manzilga 3 soatda yetib keldingiz. Sizning o'rtacha tezligingiz qancha bo'ldi? $Tezlik = \frac{Masofa}{Vaqt} = \frac{300}{3} = 100 \text{ km/soat}$. Lekin bu "O'rtacha" tezlik! Siz hamma joyda aniq 100 bilan yurmadingiz. Qayerdadir radar bor edi 70 ga tushdingiz, qayerdadir 120 gacha chiqdingiz.

Savol: Aynan Jizzaxdan o'tayotganingizda, soat roppa-rosa 14:15 da spidometringiz ko'rsatgan **Ayni vaqtdagi aniq tezligingiz (Instantaneous rate of change)** qanday edi? Buni topish uchun sizga "O'rtacha" emas, Hosila kerak bo'ladi! Va hosila Limit mantiqiga asoslanadi:

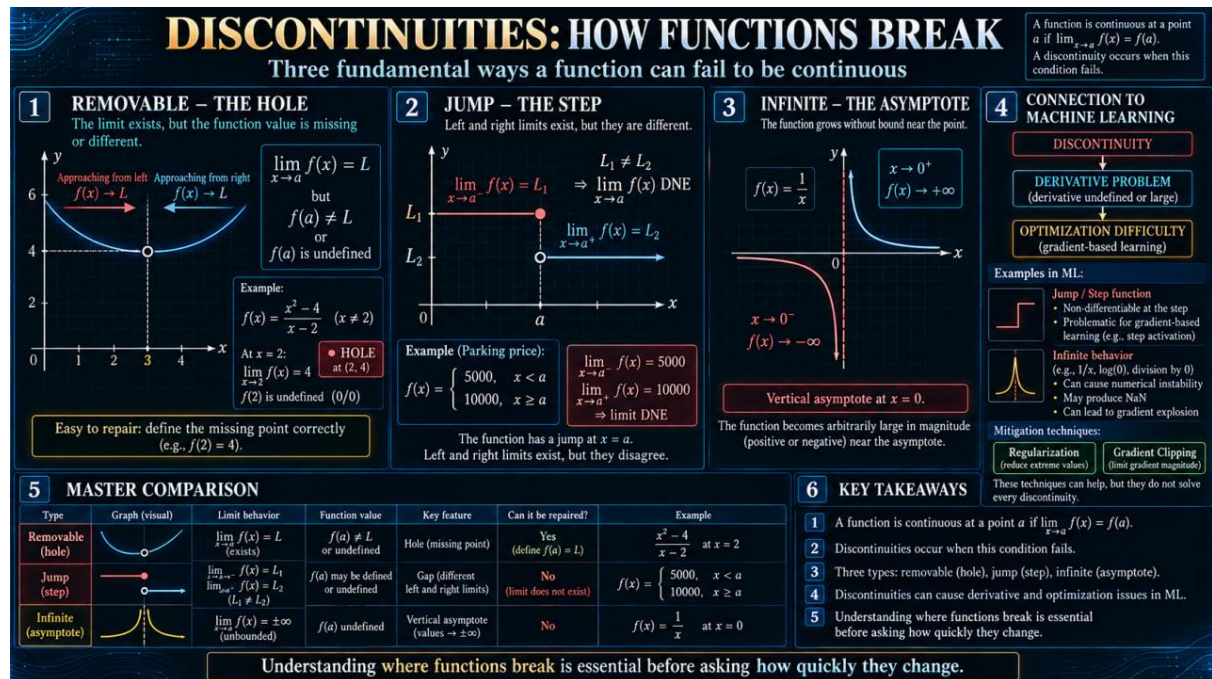
Siz vaqt oralig'ini qisqartirib borasiz.

- Dastlab Jizzaxdan chiqish va kirishdagi 1 soat oraliqqa bo'lasiz.
- Keyin 1 daqiqa oraliqdagi tezlikni hisoblaysiz.
- Keyin 1 soniyadagi.
- Oxiri, o'sha vaqt oralig'ini (uni h harfi bilan belgilaymiz) **nolga qarab intiltirasiz (Limit olasiz)!**

Ana shunda paydo bo'ladigan sehrli formula (Hosilaning tug'ilishi) mana bunday ko'rinadi:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \quad f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Bu tenglamani eslab qoling. Bu barcha kashfiyotlarning, jumladan bizning Neyron Tarmoqlarimiz xatoni qanday qilib o'lchashini belgilab beruvchi universal qonundir. h bu ikki nuqta orasidagi masofa bo'lib, u Limit yordamida 0 ga intilganda (ya'ni 2 ta nuqta deyarli 1 ta nuqtaga yopishganda), biz aynan bitta joydagi qiyalikni, tezlikni aniq topib olamiz. Bu keyingi 7-darsimizning mutlaq bosh qahramoni bo'ladi.



7-QISM: ML Intuition (Machine Learningda bularning nima aloqasi bor?)

Agar siz "Men matematik emasman, menga limitlar va uzluksizlik kompyuterda qachon kerak bo'ladi?" degan savolni berayotgan bo'lsangiz, demak e'tiboringizni to'plash vaqti keldi. Kalkulyus — Deep Learningning dvigatelidir. Keling, uning uzluksizlik (Continuity) va Limitlar qismi AI ni qanday boshqarishini ko'ramiz.

1. Optimization and Loss Landscape (Gradient Descentning sayohati)

Siz qandaydir Neyron tarmog'ini o'rgatmoqchisiz (Train qilyapsiz). U dastlab tasodifiy raqamlar bilan "ahmoqona" javoblar beradi, uning xatolik miqdori (Loss function - L) juda baland bo'ladi. Sizning vazifangiz shu baland "Tog'" (Loss Landscape) ning eng pastidagi chuqurligini, vodiysini (Global Minimum) topib, kompyuterni o'sha yerga olib borishdir.

Bu sayohat algoritmi **Gradient Descent (Gradient ushlab tushish)** deb ataladi. U bitta kichkina qadam tashlaydi, atrofga qaraydi, yana qadam tashlaydi va nolga qarab pastlab (limit olib) boradi. **Ammo diqqat qiling:** Gradient Descent ishlashi (ya'ni biz tepada ko'rgan Hosilani hisoblay olishi) uchun, biz yurib borayotgan Xatolik Tog'i **MUKAMMAL DARAJADA UZLUKSIZ (Continuous) va Ravon (Smooth)** bo'lishi qat'iy talab qilinadi!

- Agar bizning Loss funksiyamiz (Tog'imiz) ning qayeridadir Teshik (Removable discontinuity) bo'lsa, sharikli kompyuterimiz o'sha teshikka tushib g'oyib bo'ladi.
- Agar tog'da Jarlik va Sakrash (Jump discontinuity) bo'lsa, kompyuter qaysi qiyalikka (hosilaga) ishonishini bilmay aylanib qoladi. Aynan shuning uchun biz Machine

Learningda yozadigan barcha Loss funksiyalar (MSE, Cross-Entropy) matematik jihatdan uzluksiz bo'lishi shart deb qoida qilib belgilangan.

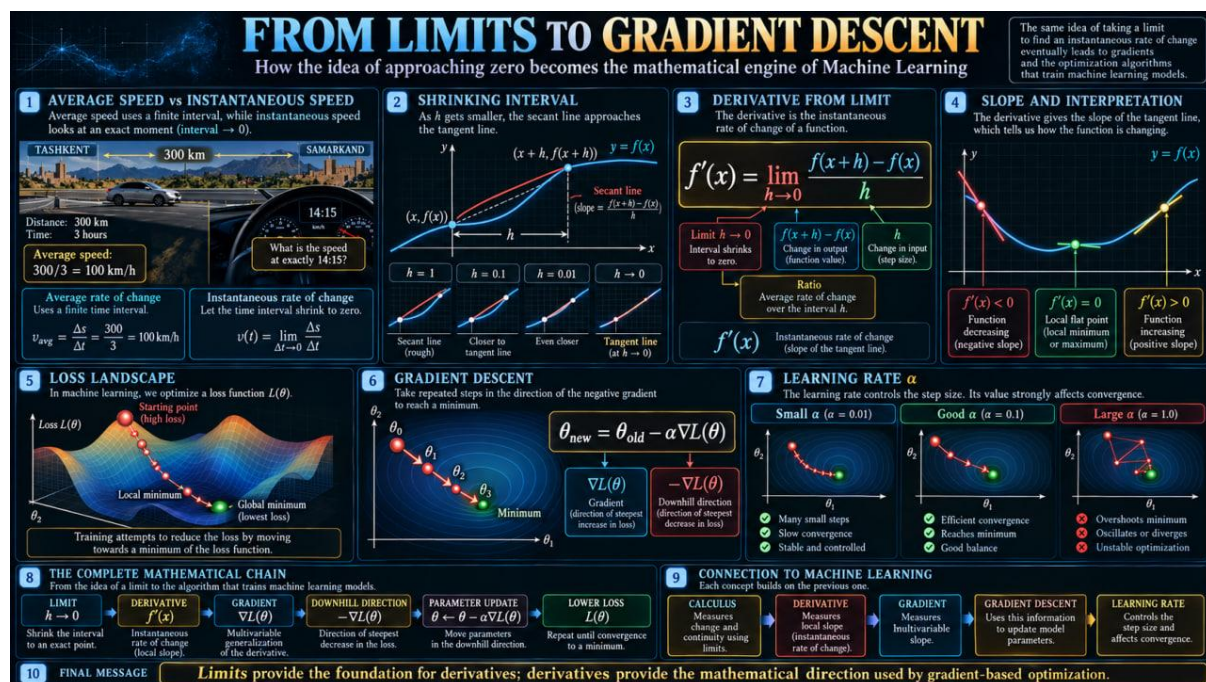
2. Activation Functions (Aktivatsiya funksiyalari - ReLU mo'jizasi)

Deep Learningning eng asosiy komponenti bo'lgan "Aktivatsiya funksiyasi" degan narsa bor. U kelgan signallarni o'tkazish yoki o'tkazmaslikni hal qiladi. Olimlar avval Jump (Zinasimon) funksiyani ishlatishib foyda topmagach, Sigmoid degan chiroyli uzluksiz (lekin sekin ishlaydigan) funksiyaga o'tishdi.

Lekin bugungi kunda dunyodagi (GPT dan tortib hamma joyda) eng ko'p ishlatiladigan aktivatsiya funksiyasi bor. Bu **ReLU (Rectified Linear Unit)** deb ataladi. Uning formulasi juda oddiy: $f(x) = \max(0, x)$.

- Agar raqam manfiy bo'lsa ($x < 0$), nolga aylantiradi.
- Agar musbat bo'lsa ($x \geq 0$), o'zini qoldiradi.

Bu grafik xuddi burchak ostida singan temir tayoqqa o'xshaydi. Nol nuqtasigacha yassilaniib keladi, noldan keyin 45° burchak ostida yuqoriga o'sadi. **Bu funksiya uzluksizmi?** Matematik analiz qilamiz: $\lim_{x \rightarrow 0^-} (0) = 0$, $\lim_{x \rightarrow 0^+} (x) = 0$. Ikkala limit ham markazda bitta nol (0) nuqtada birlashyapti, hech qanaqa teshik yoki sakrash (Jump) yo'q. Demak, **HA, ReLU mutlaqo Uzluksiz funksiya (Continuous Function)!** Lekin bu nuqtada qalam bilan "singan" joy hosil bo'lyapti, u tekis silliq (Smooth) aylanib o'tmayapti. (Bunday singan nuqtalarda hosila olish - differentiability muammosi tug'iladi, u yerda qiyalik yo'q. Ammo ML freymvorklari bu nuqtani majburan qandaydir bitta o'ng tomonga biriktirib amallab o'tib ketishadi). Eng muhimi — funksiyaning o'zi uzilmasdan saqlanib qolgan. Aynan uzluksiz bo'lgani uchun u zamonaviy Sun'iy Intelлектning mutlaq qiroliga aylangan!



3. Learning Rate (O'rganish tezligi - kichik qadamlar siri)

Yuqorida Hosila formulasi ($\lim_{h \rightarrow 0}$) ga qarasangiz, h (oraliq qadam) nolga qarab juda sekin mitti qadamlar bilan intiladi deb yozilgan. Machine learningda aynan mana shu h ning amaliy ko'rinishi **Learning Rate** (α , **Alfa**) deb ataladi. Agar biz o'qitish jarayonida Learning rateni judayam katta qilib qo'ysak (katta qadam tashlasak), Limit mantiqiga putur yetadi, va bizning modelimiz chuqur (vodiyni) aniq nuqtasini topolmay, devordan devorga sakrab, hech qachon optimal (eng zo'r) nuqtaga tusha olmaydi (Divergence). Limit qonuniyatlari bizga "Muvaffaqiyatga doim mayda, uzluksiz qadamlar bilangina (Limitni e'tiborga olib) erishish mumkin" ekanligini o'rgatadi.

Xulosa (Conclusion)

Qadrli intiluvchan ML muhandisi, tabriklayman! Siz bugun Chiziqli Algebra nomli abstrakt matematikaning statik (qotib qolgan) dunyosidan chiqib, olamni harakatda (dinamik) kuzata oladigan qatlam — **Kalkulyus** fani poydevorini qurib bitirdingiz.

- Siz **Limitlar** shunchaki nuqtaga erishish emas, balki "unga har ikki tomondan intilish va tendensiyaning o'rganish" ekanligini kashf qildingiz.
- Siz Chap va O'ng limitlar tortishuvi orqali funksiyada nuqson bor yo'qligini, agar ular birlashmasa funksiya "Sakrashiga (Jump)" tushunib yetdingiz.
- **Uzluksizlik (Continuity)** shunchaki qalamni uzmasdan chizish emas, balki butun boshli Neyron Tarmoqlari xatolik tepaligidan qulab tushmay pastga tushib kela olishi uchun kafolat (Garant) beruvchi matematik talab ekanligini anglab yetdingiz.

Biz dars yakunida **Hosila (Derivative)** degan juda qiziq va o'ta muhim tushunchaga eshik ochdik. Kalkulyusning birinchi harfi Limitlar bo'lsa, uning davomi Hosiladir. Keyingi, navbatdagi darsimizda (7-Dars: Derivatives and Gradient), biz aynan o'sha $h \rightarrow 0$ formulasi orqali modellar o'zgarishlar qiyaligini (Slopeni) qanday aniqlashini, Gradient tushunchasi nega bizni to'ppa-to'g'ri nishonga (Minimumga) olib borishini o'rganamiz. Hozircha esa bugungi Limitlar falsafasini aqlingizda qayta pishiring, biz kelajak sun'iy intellekt arxitekturasini quryapmiz! Olg'a!

ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)

7-Dars: Hosila va Gradient (Derivatives and Gradient)

Assalomu alaykum, bilimga chanqoq va kelajakni o'zgartirishga tayyor bo'lgan Sun'iy Intellekt muhandisi! O'tgan darsimizda biz Kalkulyusning poydevori bo'lgan **Limitlar (Limits)** va **Uzluksizlik (Continuity)** haqida chuqur suhbatlashdik. U yerda biz qanday qilib nuqtaga o'ta yaqin kelish mumkinligini o'rgandik. Bugun biz o'sha bilimlarni to'plab, insoniyat tarixidagi eng ulug'vor matematik tushunchalardan biri — Hosila (Derivative)ni va uning ko'p o'lchamli "akasi" bo'lmish **Gradient**ni kashf etamiz.

Agar siz mendan: "Machine Learning modellarini nima aqlli qiladi?" deb so'rasangiz, men hech ikkilanmasdan "Gradientlar!" deb javob bergan bo'lardim. Chiziqli algebra (Matritsalar) yordamida biz ma'lumotlarni kompyuterga kiritamiz. Ehtimollar nazariyasi (Probability) yordamida biz modelga noaniqlikni bashorat qilishni o'rgatamiz. Ammo faqat va faqat **Hosila**

va Gradient yordamida biz modelga o'z xatosini tushunishni va uning ustida ishlashni (Optimization) o'rgata olamiz.

Ushbu darsda biz hosila qanday qilib o'zgarish tezligini o'lchashini, geometrik nuqtai nazardan qiyalik (Slope) nima ekanligini, Xususiy hosilalar (Partial derivatives) orqali koinot qanday kengayishini va nima uchun barcha Deep Learning tizimlari Gradient Descent algoritmligiga qaram ekanligini eng mayda detallargacha ko'rib chiqamiz. Qahvangizni hozirlang, biz matematika olamida parvozni boshladik!

1-QISM: Hosila Nima? O'zgarish Tezligi (Rate of Change)

Hayotda hamma narsa o'zgaradi. Mashina joyidan siljiydi, havo harorati ko'tariladi, kompaniya aksiyalari narxi o'zgaradi, bizning ML modelimizning xatoligi (Loss) kamayib boradi. Matematika bu o'zgarishlarni o'lchashni xohlaydi. Va bu o'lchov asbobi fanda **Hosila (Derivative)** deb ataladi.

Tasavvur qiling, siz avtomobilda Toshkentdan Samarqandga qarab yo'lga chiqdingiz. Oraliq masofa 300 km. Siz u yerga 3 soatda yetib bordingiz. Sizning **o'rtacha tezligingiz** qancha bo'ldi? Buni topish oson: $O'rtachatezlik = \frac{Masofao'zgarishi}{Vaqto'zgarishi} = \frac{\Delta y}{\Delta x} = \frac{300}{3} = 100km/soat$.

Matematikada qandaydir ikkita qiymat o'rtasidagi farq Δ (Delta) harfi bilan belgilanadi. Ya'ni $\Delta y = y_2 - y_1$.

Lekin o'ylab ko'ring, siz butun yo'l davomida roppa-rosa 100 km/soat tezlikda qotib yurmadingiz-ku, to'g'rimi? Qayerdadir 120 ga chiqdingiz, qayerdadir (radar bor joyda) 70 ga tushdingiz, choyxonada umuman 0 tezlikda to'xtab turdingiz. Menga "O'rtacha" tezlik kerak emas. Menda shunday bir daxshatli savol bor: **Siz Jizzaxdan o'tayotganingizda, soat roppa-rosa 14:15 da sizning ayni shu soniyadagi (Instantaneous) aniq tezligingiz qanday edi?** Mashinangiz spidometri aynan o'sha millisoniyada qaysi raqamni ko'rsatib turgan edi?

Biz bu aniq "Laxzali" tezlikni qanday hisoblaymiz? Agar biz formulamiz $\frac{\Delta y}{\Delta x}$ ni ishlatsak, vaqt oralig'imiz (biz uni h yoki Δx deymiz) roppa-rosa nolga teng bo'lib qoladi. Chunki 14:15 va 14:15 orasi nol soat. Nol vaqtda nol masofa bosib o'tiladi. Javob 0/0 bo'lib qoladi! 6-Darsda o'rganganimizni eslaysizmi? 0/0 bu biz taslim bo'lishimiz kerak degani emas. Bunga **Limit** yordamida yechim topamiz! Biz vaqt oralig'ini (h) nolga qarab juda sekin intiltiramiz: 1 minut, keyin 1 soniya, keyin 1 millisoniya...

Mana shu mo'jizaviy Limit operatsiyasi orqali biz biron bir nuqtadagi **Laxzali O'zgarish Tezligini (Instantaneous Rate of Change)** topamiz. Ana shu topilgan narsa **HOSILA** deb ataladi!

Hosilaning Fundamental Formulasi: Agar bizda $f(x)$ funksiya bo'lsa, uning x nuqtadagi hosilasi odatda ustiga shtrix qo'yib $f'(x)$ (ef shtrix x deb o'qiladi) yoki $\frac{df}{dx}$ (df bo'lingan dx) ko'rinishida belgilanadi. Uning ta'rifi (Limit yordamida) bunday:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \quad f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- Bu yerdagi $f(x + h) - f(x)$ bu Masofaning (yoki javobning) ozgina o'zgarishi, ya'ni Δy .
- h esa o'sha biz tanlagan o'ta kichik oraliq qadam (vaqt o'zgarishi, Δx). Biz uni chegarada nolga ($h \rightarrow 0$) intiltiramiz.

Machine Learningda hosila degani: "Mening modelimdagi vaznlarni (Weights - w) bir millimetrga o'zgartirsam, mening xatoligim (Loss - L) qancha foizga va qaysi tomonga o'zgaradi?" degan savolning mutlaq aniq matematik javobidir.

2-QISM: Hosilaning Geometrik Ma'nosi (Slope va Tangent Line)

Fizikada hosila bu "Tezlik" degani. Geometriyada esa hosila nima degani? Keling buni chizmalarsiz tasavvur qilishga urinamiz.

Secant Line (Kesuvchi chiziq): Qog'ozga bitta chuqur kosa shaklidagi parabolani chizing ($f(x) = x^2$). Shu kosa ustida istalgan ikkita nuqtani tanlang (masalan $x = 1$ va $x = 3$). Va bu ikki nuqtani bitta to'g'ri chiziq bilan tutashtiring. Bu chiziq parabolani ikkita joydan kesib o'tgani uchun uni *Kesuvchi chiziq (Secant Line)* deymiz. Bu chiziqning qiyaligi (Slope) — boyagi biz gaplashgan "O'rtacha tezlik" dir.

Tangent Line (Urinma chiziq): Endi boyagi ikkinchi nuqtani ($x = 3$) sekin-asta, huddi relsda yurgandek, parabola bo'ylab birinchi nuqtaga ($x = 1$) qarab surib (Limit qilib) yaqinlashtirib kelavering. Ikkita nuqta bir-biriga shunchalik yaqinlashib, deyarli ustma-ust tushib qolganda (oraliq $h \rightarrow 0$ bo'lganda), haligi Kesuvchi chiziq endi faqat va faqat **bitta nuqtadan** tegib o'tadigan chiziqqa aylanadi. Bu chiziq geometriyada **Urinma Chiziq (Tangent Line)** deb ataladi!

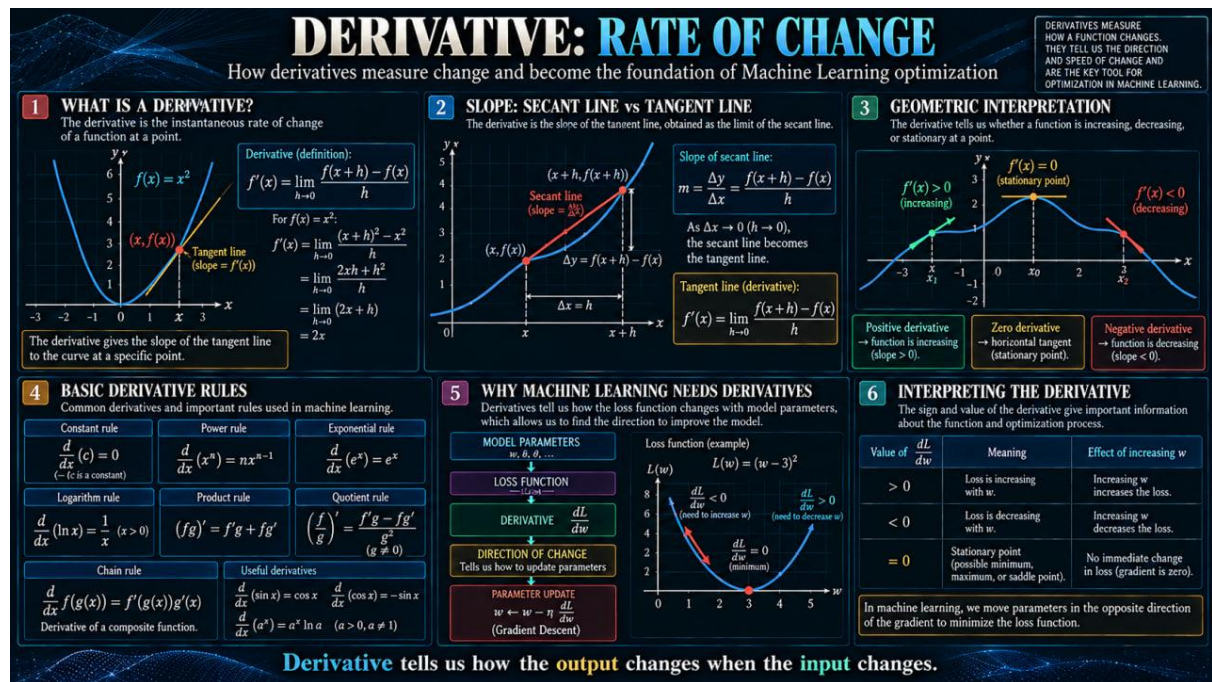
Qiyalik (Slope - m): Maktab geometriyasini eslang, har qanday to'g'ri chiziqning tenglamasi $y = mx + b$ formulasi orqali yozilardi. Bu yerdagi m soni — **Qiyalik (Slope)** deb ataladi. U chiziqning qanchalik tik yoki yassi ekanligini, ya'ni agar x o'qi bo'ylab 1 qadam onga yursak, chiziq bizni necha qadam yuqoriga yoki pastga olib borishini ko'rsatadi.

SHOHONA XULOSA: Biron bir funksiyaning Hosila olish ($f'(x)$) degani — u grafikning ayni o'sha nuqtasiga tegib o'tuvchi **Urinma Chiziqning Qiyaligini (Slope ni) topish deganidir!**

- Agar $f'(x)$ ning javobi **Musbat (Positive)** son bo'lsa: Demak urinma chiziq yuqoriga qarab ketyapti. Grafik o'syapti (x o'shsa, y ham oshadi).
- Agar $f'(x)$ ning javobi **Manfiy (Negative)** son bo'lsa: Demak urinma chiziq pastga qarab sho'ng'iyapti. Grafik kamayyapti (x o'shsa, y tushadi).
- Agar $f'(x)$ ning javobi roppa-rosa **NOL (0)** ga teng bo'lsa: Demak urinma chiziq yerga mutlaqo parallel (yotik, gorizontal) bo'lib qolgan. Grafik o'smayapti ham, tushmayapti ham. Bu yer ko'pincha "Tog'ning eng uchi" (Maximum) yoki **"Chuqurlikning eng tubi" (Minimum)** nuqta hisoblanadi!

(Esingizdan chiqarmang: Machine Learningdagi Gradient Descent algoritmining yakka-yu yagona eng oily maqsadi — Aynan xatoliklar kosasining eng tubini, ya'ni Hosilasi Nolga teng

bo'lgan joyni qidirib topishdir! Biz tepalikdan boshlaymiz, nishablik (qiyalik) yo'nalishi bo'ylab to hosila nol bo'lgunicha pastga qarab yuraveramiz).



3-QISM: Hosila Olish Qoidalari (Derivative Rules)

Yuqorida biz Hosilani topishning fundamental "Limit" formulasini yozdik. Lekin agar biz doim limit bilan hisoblasak, bitta oddiy funksiyaning hosilasini topish uchun yuzlab qator hisob-kitob qilishimizga to'g'ri kelardi. Matematiklar buning yechimini allaqachon topishgan. Ular ko'plab qonuniyatlarni kashf qilishgan va biz shunchaki tayyor qoidalar (Rules) yordamida ko'zni yumib 1 soniyada hosila olishimiz mumkin.

ML muhandisi sifatida siz bu tayyor qoidalarni yoddan bilishingiz shart. Keling, eng mashhurlarini birma-bir tahlil qilamiz:

1. O'zgarmas Son Qoidasi (Constant Rule)

Agar $f(x)$ qandaydir o'zgarmas songa (C) teng bo'lsa (masalan $f(x) = 5$ yoki $f(x) = -100$), unda x qanchalik o'zgarmasin, javob baribir 5 bo'lib turaveradi. Grafikda bu yotiq (gorizontal) chiziq degani. Yotiq chiziqning qiyaligi (o'zgarishi) yo'q. Demak, har qanday o'zgarmas (yakkaxon) sonning hosilasi **Nolga** (0) teng!

$$\frac{d}{dx}[C] = 0$$

2. Daraja Qoidasi (Power Rule) - Eng ko'p ishlatiladigani!

Agar x qandaydir darajaga (n) ega bo'lsa, masalan $f(x) = x^n$. Hosilani topish uchun biz shunchaki o'sha tepasidagi darajani oldinga ko'paytuvchi qilib tushiramiz, va tepada qolgan darajadan bitta (1) ayirib qo'yamiz.

$$\frac{d}{dx}[x^n] = n \cdot x^{n-1}$$

Misollar:

- $f(x) = x^3 \Rightarrow f'(x) = 3x^{3-1} = 3x^2$
- $f(x) = x^2 \Rightarrow f'(x) = 2x^{2-1} = 2x$ (Kosaning hosilasi to'g'ri chiziq ekan!).
- $f(x) = x \Rightarrow$ Aslida x^1 degani. $f'(x) = 1 \cdot x^0 = 1 \cdot 1 = 1$. (Shunchaki oddiy x ning hosilasi doim 1 bo'ladi).

3. O'zgarmas songa ko'paytirish (Constant Multiple Rule)

Agar o'zgarmas son (C) yakkaxon emas, balki x ga ko'paytirilgan bo'lsa (masalan $f(x) = 5x^3$), unda u son nolga aylanib ketmaydi! U o'z joyida kutaveradi, biz faqat x ning hosilasini olamiz va unga ko'paytiramiz.

$$\frac{d}{dx}[C \cdot f(x)] = C \cdot f'(x)$$

Misol: $f(x) = 5x^3 \Rightarrow f'(x) = 5 \cdot (3x^2) = 15x^2$.

4. Qo'shish va Ayirish Qoidasi (Sum and Difference Rule)

Agar funksiyalar bir-biriga qo'shilgan yoki ayirilgan bo'lsa, ularning hosilalari ham alohida-alohida olinib, shunchaki o'rtadagi ishora o'z joyida qoladi.

$$\frac{d}{dx}[f(x) \pm g(x)] = f'(x) \pm g'(x)$$

Misol: $f(x) = 3x^4 - 2x^2 + 10x - 7$. Hosilasini olamiz: $f'(x) = (4 \cdot 3x^3) - (2 \cdot 2x) + (10 \cdot 1) - 0 = 12x^3 - 4x + 10$. (Qanday oson, a?).

5. Ko'paytirish Qoidasi (Product Rule)

Agar ikkita har xil x li funksiyalar bir-biriga ko'paytirilgan bo'lsa, masalan $h(x) = f(x) \cdot g(x)$. Bunda shunchaki $f'(x) \cdot g'(x)$ qilib yuborib bo'lmaydi! Bu eng ko'p qilinadigan qo'pol xatodir. To'g'ri qoida bunday: "Birinchisining hosilasi ko'paytirilgan ikkinchisining o'zi, plus, birinchisining o'zi ko'paytirilgan ikkinchisining hosilasi".

$$\frac{d}{dx}[f(x) \cdot g(x)] = f'(x)g(x) + f(x)g'(x)$$

6. Bo'lish Qoidasi (Quotient Rule)

Agar $h(x) = \frac{f(x)}{g(x)}$ bo'lsa. Qoida quyidagicha:

$$\frac{d}{dx}\left[\frac{f(x)}{g(x)}\right] = \frac{f'(x)g(x) - f(x)g'(x)}{[g(x)]^2}$$

7. Zanjir Qoidasi (Chain Rule) - Asoslarning asosi!

Agar funksiyaning ichida yana boshqa bitta funksiya yashiringan bo'lsa (Composite functions), masalan ruslarning Matryoshka qo'g'irchog'i kabi: $h(x) = f(g(x))$. Biz buni qanday ochamiz? Zanjir qoidasiga ko'ra, biz avval ustki, tashqi funksiya (f) hosila olamiz

(ichidagi $g(x)$ ga tegmasdan), keyin unga ko'paytiruvchi qilib ichki funksiyaning (g) hosilasini yozamiz.

$$\frac{d}{dx}[f(g(x))] = f'(g(x)) \cdot g'(x)$$

*Eslatma: Biz Zanjir qoidasini va uning Deep Learningdagi nomi bo'lmish **Backpropagation** (Orqaga tarqalish) ni kelgusi maxsus darsimizda (11-dars) eng mayda ipigacha tahlil qilamiz.*

8. Maxsus funksiyalar (Eksponenta, Logarifm, Trigonometriya)

Neyron tarmoqlari o'zgaruvchilarni "ezish" (activation) uchun shu funksiyalarni ko'p ishlatadi:

- **Eksponenta (e^x):** Matematikadagi yagona mo'jizaviy funksiya. Uning o'zgarish tezligi uning o'ziga teng! $\frac{d}{dx}[e^x] = e^x$.
- **Natural Logarifm ($\ln(x)$):** $\frac{d}{dx}[\ln(x)] = \frac{1}{x}$. (Bu qoida Cross-Entropy loss funksiyasini hisoblashda ishlaydi).
- **Sinus va Kosinus:** $\frac{d}{dx}[\sin(x)] = \cos(x)$ va $\frac{d}{dx}[\cos(x)] = -\sin(x)$.

4-QISM: Pythonda SymPy bilan Hosila Olish (Derivative with SymPy)

ML muhandisi sifatida siz murakkab funksiyalarning hosilasini qog'ozda o'tirib soatlab hisoblamaysiz. Kompyuterlar bu ishini vizual va simvolik tarzda a'lo darajada qila oladi. Pythonda buning uchun **SymPy** (Symbolic Python) degan ajoyib kutubxona bor. U shunchaki raqamli natija bermaydi, balki biz tepadagi qoidalarni ishlatib topgan **algebraik formulamizni** keltirib chiqaradi.

Keling, buni amalda kod orqali yozamiz: Funksiya: $f(x) = 3x^4 - 2x^2 + 10x - 7$. Biz bu funksiya x ga nisbatan hosila topmoqchimiz va so'ng $x = 2$ bo'lgandagi qiyalikni (slope ni) hisoblamoqchimiz.

```
import sympy as sp
```

```
# 1. 'x' ni o'zgaruvchi (symbol) sifatida e'lon qilamiz
x = sp.Symbol('x')
```

```
# 2. Funksiyani yozamiz (Python sintaksisida daraja '**' bilan yoziladi)
f = 3*x**4 - 2*x**2 + 10*x - 7
```

```
# 3. Funksiyadan hosila olish (diff - differentiate degani)
f_prime = sp.diff(f, x)
```

```
print("Asil Funksiya: ", f)
print("Hosilasi (Formulasi): ", f_prime)
# Natija: Hosilasi (Formulasi): 12*x**3 - 4*x + 10
```

```
# 4. Hosilaga aniq qiymat (masalan x = 2) qo'yib hisoblash
slope_at_2 = f_prime.subs(x, 2)
print("x = 2 nuqtadagi Qiyalik (Slope): ", slope_at_2)
```

Natija: $x = 2$ nuqtadagi Qiyalik (Slope): $12 \cdot (8) - 4 \cdot (2) + 10 = 96 - 8 + 10 = 98$

Ko'rib turganingizdek, **SymPy** yordamida biz har qanday dahshatli funktsiyaning hosilasini bitta `sp.diff()` buyrug'i bilan aniq matematik ko'rinishda olamiz. Bu kutubxona matematik tadqiqotlar va yangi Machine Learning me'morchiligini qurishda (Gradient formalarini topishda) juda keng qo'llaniladi.

5-QISM: Xususiy Hosilalar (Partial Derivatives) - Koinotni Kengaytiramiz

Shu joygacha biz faqat bitta o'zgaruvchili (x) funktsiyalar bilan ishladik ($f(x)$). Bu koinot shunchaki bitta ikki o'lchamli qog'ozdagi chiziqdan iborat edi xolos. Lekin hayot murakkab. Siz bashorat qilayotgan uyning narxi faqat uning **maydoni (x)** ga emas, balki uning **qavati (y)** ga, **xonalar soni (z)** ga va yuzlab boshqa feature'larga bog'liq bo'ladi. Ya'ni, ML da bizning funktsiyalarimiz Ko'p o'zgaruvchili bo'ladi: $f(x, y, z, \dots)$. Biz buni geometrik qilib $f(x, y)$ ya'ni 3D (uch o'lchamli) adirlar, tog'lar manzarasi deb tasavvur qilishimiz mumkin.

Bu yerda daxshatli savol tug'iladi: Men shu qabariq "Tog'" ning biron bir nuqtasida turibman. Agar bitta qadam oldinga yursam balandlik o'zgaradimi? Xo'sh, qaysi tomonga yursam? Shimolga (X o'qi) yursam qiyalik (slope) balandroq bo'lishi mumkin, Sharqqa (Y o'qi) yursam pastlikka olib tushishi mumkin!

Bizda bitta emas, har bir yo'nalish uchun alohida qiyalik (hosila) mavjud bo'lib qoldi! Ana shu holatda fanda **Xususiy Hosila (Partial Derivative)** tushunchasi kiritiladi. Biz to'g'ri turgan d (derivation) belgisini biroz egib, ∂ (Partial - del) belgisiga o'tkazamiz.

- $\frac{\partial f}{\partial x}$: Bu f funktsiyaning FAQAT x ga nisbatan hosilasi degani.
- $\frac{\partial f}{\partial y}$: Bu f funktsiyaning FAQAT y ga nisbatan hosilasi degani.

Xususiy Hosila Olishning Oltin Qoidasi: Bu qoida shunchalik oson va chiroyliki, siz hech qachon qiynalmaysiz. *Qachonki siz qaysidir (masalan x) o'zgaruvchiga nisbatan hosila olayotgan bo'lsangiz, qolgan BARCHA o'zgaruvchilarni (masalan y va z larni) huddi oddiy o'zgarmas son (Constant) kabi qotirib qo'yasiz!* O'zgarmas sonning hosilasi esa boya ko'rganimizdek 0 ga teng yoki oldinga ko'paytuvchi bo'lib tushadi.

Amaliy Misol: Bizda mana bunday Tog' (Funktsiya) bor:

$$f(x, y) = x^2 \cdot y + 3y^2 + 5x$$

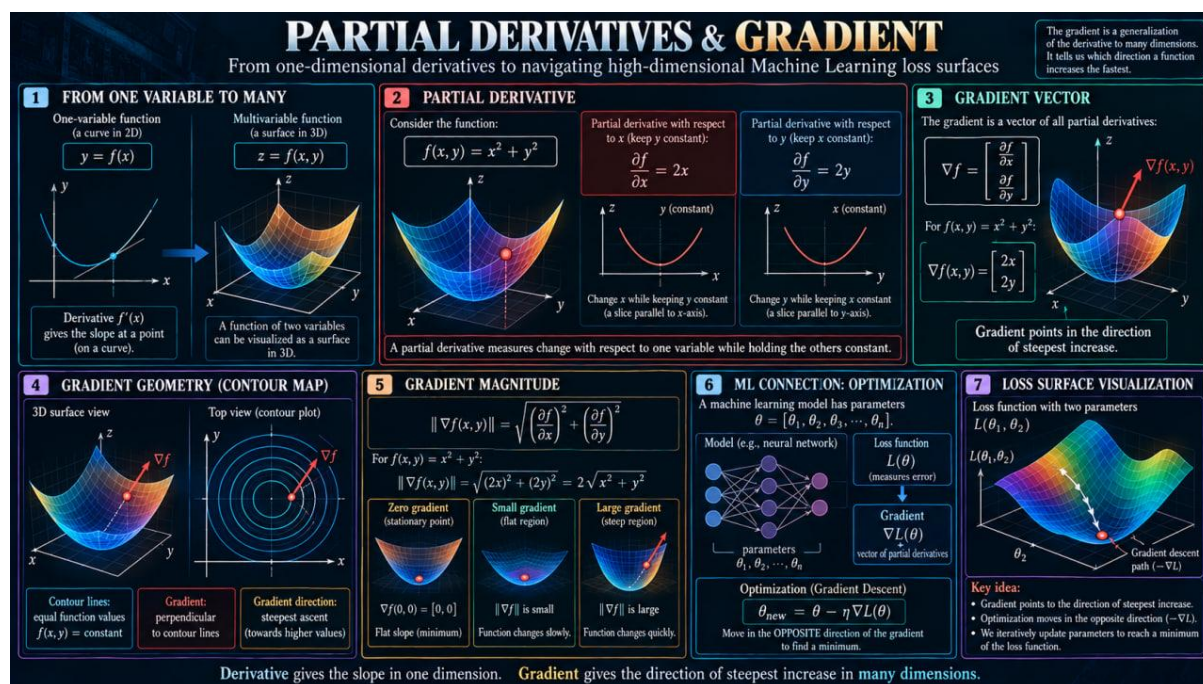
1. **X ga nisbatan xususiy hosila olamiz ($\frac{\partial f}{\partial x}$):** Biz bu yerda y ni huddi "3" yoki "10" kabi oddiy raqam deb faraz qilamiz.
 - Birinchi had ($x^2 \cdot y$): y raqam, shuning uchun x^2 dan hosila $2x$, va y yonida qolaveradi $= 2xy$.

- Ikkinchi had ($3y^2$): Bu yerda umuman x yo'q! Bu butunlay o'zgarmas son (constant). Uning hosilasi = 0.
- Uchinchi had ($5x$): Hosilasi = 5. **Javob:** $\frac{\partial f}{\partial x} = 2xy + 5$. (Bu Shimolga yurganingizdagi qiyalik!)

2. **Y ga nisbatan xususiy hosila olamiz** ($\frac{\partial f}{\partial y}$): Endi x ni o'zgarmas son (qotirilgan raqam) deb olamiz.

- Birinchi had ($x^2 \cdot y$): x^2 raqam kabi qoladi, y ning hosilasi 1. Javob = x^2 .
- Ikkinchi had ($3y^2$): Hosilasi = $6y$.
- Uchinchi had ($5x$): Bu yerda y yo'q! Constant. Hosilasi = 0. **Javob:** $\frac{\partial f}{\partial y} = x^2 + 6y$. (Bu Sharqqa yurganingizdagi qiyalik!)

Siz tasavvur ham qila olmaysiz, butun boshli Sun'iy Neyron Tarmoqlari (Deep Learning) ana shu juda oddiy qoida evaziga milliardlab o'zgaruvchilar (Weights) ustidan xususiy hosilalar olib, qiyaliklarni soniyaning mingdan bir bo'lagida topib, aqlli bo'lib bormoqda!



6-QISM: Gradient Vektori (Gradient Vector - ∇f)

Mana, biz shuncha vaqt atrofida aylangan so'zimizning aniq va shohona ma'nosiga keldik. Yuqorida biz har bir yo'nalish uchun (x uchun, y uchun, z uchun) alohida-alohida xususiy hosilalar (qiyaliklar) topdik. Geometriya va fizika uzoq yillar davomida bu alohida raqamlarni bitta kuchga, bitta ko'rsatkichga birlashtirish yo'lini izladi. Va ular ajoyib yechimni topishdi: **Barcha xususiy hosilalarni yig'ib, bitta Vektor (Quticha) ichiga joylaymiz!**

Ana shu barcha xususiy hosilalar jamlanmasidan iborat bo'lgan sehrli vektor fanda **Gradient (Gradient Vektori)** deb ataladi! U fanda yunoncha Nabla harfi — teskari uchburchak (∇) bilan belgilanadi.

Agar $f(x, y)$ funksiyamiz bo'lsa, uning Gradienti quyidagi vektor hisoblanadi:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} \quad \nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

Agar uylar narxini 100 xil xususiyat bo'yicha bashorat qilsak (100-D fazo), unda Gradient Vektori ham o'z ichiga 100 ta qatordan iborat xususiy hosilalarni olgan juda uzun, yagona Vektor bo'ladi. Vektorlar bizga o'q yo'nalishini ko'rsatishini biz 1-darsdan bilamiz. Gradient vektori shunchaki raqamlar ro'yxati emas, u eng aqlli o'ziga xos kompasdir! Xo'sh, u qayerga qarab yo'naltirilgan?

7-QISM: Gradient Yo'nalishi (Gradient Direction - Mantiqiy cho'qqi)

Tasavvur qiling, siz tun qorong'usida ko'zingizni bog'lagan holda tog' (Tog' bu bizning Xatolik (Loss) Funktsiyamiz, $J(w)$) ning qayeridadir turibsiz. Siz atrofni ko'rmaysiz. Lekin siz oldinga bitta qadam, orqaga, o'ngga va chapga bittadan juda kichkina qadam tashlab ko'radingiz (Xususiy hosilalarni hisobladingiz). Va barcha olgan bilimlaringizdan Gradient Vektorini yasadingiz.

Matematikaning eng ulug'vor qonunlaridan biri shuni tasdiqlaydiki:

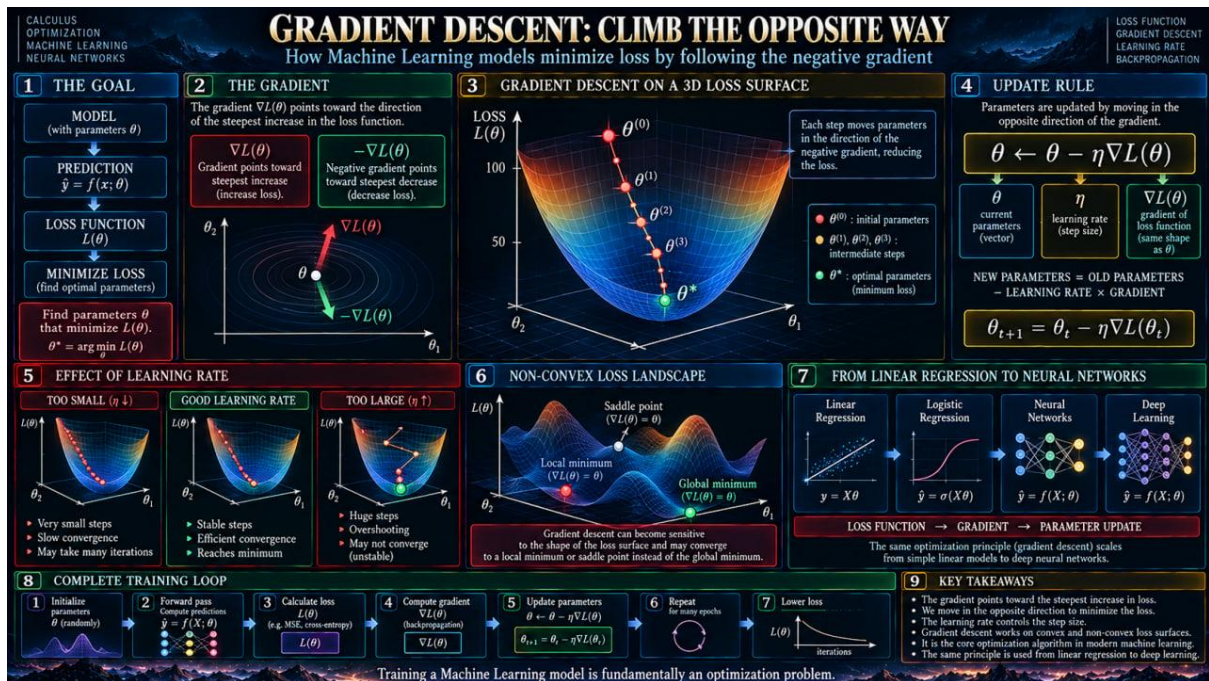
Agar siz ayni turgan nuqtangizda **Gradient Vektori** (∇f) **qaysi tomonni ko'rsatayotgan bo'lsa, roppa-rosa o'sha tomonga yursangiz, siz koinotdagi ENG TIKKA, ENG TEZ O'SISH (Steepest Ascent) yo'nalishiga qarab yurgan bo'lasiz!**

Ya'ni, Gradient vektori har doim tog'ning ENG CHO'QQISIGA, osmonga eng to'g'ri olib chiqadigan yo'lni ko'rsatuvchi magnitdir. Vektorning o'lchami (Normasi - uzunligi) esa u yerda nishablik qanchalik tik ekanligini anglatadi.

Lekin kuting... Machine Learningda bizning maqsadimiz o'zi nima edi? Biz bizdagi Loss Funktsiyani (Ya'ni bizning bashoratimiz haqiqatdan qanchalik noto'g'ri ekanligini, XATOLIKNI) **kattalashtirmoqchimidik yoki nolga qarab Kichraytirmoqchimidik?** Albatta KICHRAYTIRMOQCHIMIZ! Bizga tog'ning cho'qqisi kerak emas, biz o'z xatomizni nolga (Tog'ning eng pastki tubiga) olib tushishimiz kerak (Global Minimum).

Agar Gradient (∇f) har doim eng tez balandlikka olib chiqsa, u holda tog'ning eng pastki, xatolikning eng tub tubiga olib tushadigan **Eng tikka pasayish (Steepest Descent)** yo'nalishi qayerda joylashgan? Javob shu qadar go'zal va osonki: **Gradient Vektorining ROQQA-ROSA QARAMA-QARSHI TOMONIDA (Ya'ni, Minus Gradient tomoniga: $-\nabla f$)!!!**

Agar siz shunchaki Gradientni hisoblasangiz va uning oldiga bitta minus (-) ishorasini qo'yib, o'sha tomonga qadam tashlasangiz, siz qorong'u tog'ning eng chuqur vodiysiga qarab eng to'g'ri, eng qisqa va optimal marshrut bilan sho'ng'iyotgan bo'lasiz!



8-QISM: ML Intuition va Gradient Descent Algoritmi

Miyangizda paydo bo'lgan o'sha yorqin nur to'g'ri. Biz hozirgina sun'iy intellektning har qanday modellarini (Simple Linear Regression dan tortib, 100 milliard parametrli ChatGPT (Transformer) larga qadar) qanday qilib o'rganishini (Training) ta'minlovchi yurak motorini kashf qildik. Bu algoritmi fanda **Gradient Descent (Gradient Nishabligi / Gradient Usulida Pastlash)** deb ataladi.

Modelning o'rganish (o'zini moslashtirish) jarayoni 4 ta takrorlanuvchi qadamdan iborat. Bu yerda bizning Noma'lumlarimiz, O'zgaruvchilarimiz (Modelning og'irliklari, Vaznlari) - bular W deb belgilanadi (X o'rnida). Xatolik funksiyasi esa $J(W)$ (Tog'ning balandligi, Y o'rnida).

1. **Tasodifiy boshlash (Initialization):** Model barcha W (vazn) larga taxminiy (ko'r-ko'rona, tasodifiy) raqamlar beradi. U qorong'u tog'ning (Loss function) osmondagi qaysidir tasodifiy joyidan (Juda katta xatolik bilan) yerga tushadi.
2. **Gradientni hisoblash (Compute Gradient):** Kompyuter jami o'quv ma'lumotlarini qarab chiqib, o'sha turgan nuqtadagi Gradient Vektorini $\nabla J(W)$ ni (barcha xususiy hosilalarni) Limit qonunlaridan (yoki Backpropagationdan) foydalanib topadi. Bu vektor unga "Tog'ning uchi (xatoni o'sishi) anavi tomonda!" degan kompasni beradi.
3. **Minus Gradient Tomonga yurish (Update Step):** Endi model o'zidagi eski Vaznlarini (W_{eski}) yangisiga (W_{yangi}) almashtirishi kerak. U minus ishorasi orqali Orqaga, vodiya qarab qadam tashlaydi. Bu qadamning "qattiqligini" boshqaruvchi bitta o'ziga xos gaz pedali bor. Bu **Learning Rate (O'rganish tezligi - α , Alfa)** deb ataladi. Agar alfa juda katta bo'lsa, u ulkan qadam tashlab vodiya u tomonidan bu tomoniga o'tib, markazni o'tkazib yuboradi (Divergence). Agar alfa juda kichkina bo'lsa, u yillar davomida sudralib yuradi.
4. **Sehrlili (Oltin) Tenglama:** Mana shu jarayonning umumjahon formulasi quyidagicha yoziladi:

$$5. \quad W_{new} = W_{old} - \alpha \cdot \nabla J(W_{old})$$

6. (O'qilishi: Yangi vaznlar = Eski vaznlar Ayirilgan (O'rganish tezligi α ko'paytiruv Gradient kompasiga)). Biz bitta qadam tashladik! Xatolik pastladi. Endi shu to'rtta jarayon (Epoch / Iteration) yana boshidan aylanadi (for loop). Biz yangi joyda yana gradient olamiz, yana minus tomonga qadam tashlaymiz... Toki qayergacha? Toki bizning Gradientimiz $\nabla J(W)$ roppa-rosa Nol (0) ga teng bo'lib qolmaguncha! Nima uchun? Chunki darsning boshida aytgan edim: Hosila (qiyalik) nolga teng bo'lsa, demak biz Tog'ning (Kasaning) ENG TUBIGA kelib bo'ldik! U yerdan pastga boshqa tushishni iloji yo'q, xato nolga yaqinlashdi! Algoritm to'xtaydi (Convergence). Model Train bo'ldi va tayyor holatga keldi.

Xulosa

Aziz intiluvchan Data Scientist, siz bugun raqamli koinotdagi qotib qolgan nuqtalarni qanday qilib o'zgartirish tezligiga (Hosila - Derivative) va Geometrik qiyaliklarga (Slope) aylantirish sirlarini bilib oldingiz. Siz ko'p o'lchamli dunyoda qotirib qo'yish orqali Xususiy Hosilalar (Partial derivatives) olishni o'rgandingiz. Ushbu xususiy hosilalarni yig'ish orqali bitta Vektor – **Gradient** tushunchasi kelib chiqishini va u har doim osmonga qarab eng tik yo'lni ko'rsatishini anglab yetdingiz. Va eng hayajonlisi, "Minus Gradient" ga qarab qadam tashlash orqali butun boshli Sun'iy Intellect o'zining xatolaridan (Loss) qanday qutulib, qanday qilib optimal holatga (Minimumga) kelishini dildan his qildingiz.

Bu dars nafaqat matematika, balki u "Aqlning paydo bo'lish (O'rganish)" mexanizmi edi. Keyingi darsimizda (8-Dars: Applications of Derivatives) biz aynan mana shu Gradient Descent algoritmini yanada ko'proq chuqurlashtiramiz, Loss Function nima o'zi va modelimiz Local (yolg'onchi) va Global (Haqiqiy) chuqurliklarni qanday farqlashini o'rganamiz. Hosilaning qiyaliklarida charchamay yuring, g'alaba juda yaqin!

ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)

8-Dars: Hosilaning Amaliyoti - Xatolik Funktsiyasi va Optimizatsiya (Cost Function and Optimization)

Assalomu alaykum, aziz va qat'iyatli Data Scientist! O'tgan darsda biz barcha zamonlarning eng buyuk kashfiyoti – Hosila (Derivative) va uni ko'p o'lchamli olamga tarqatuvchi kuch – Gradient bilan tanishdik. Biz ular nima uchun tabiat qonunlarini va o'zgarish tezligini (Slope) o'lchashga qodir ekanligini ko'rdik.

Lekin faqat sof matematika bilan cheklanib qolish bizning maqsadimiz emas. Biz bu matematika orqali Kompyuterlarga qanday qilib Aql (Intelligence) o'rgatilishini tushunishimiz kerak. Biz o'tgan darsning oxirida qisqacha ko'rib o'tgan "Gradient Descent (Gradient yordamida pastlash)" algoritmi ustidan sirli pardani to'liq ko'tarish vaqti keldi.

Bugungi darsda biz tasavvurdagi tog'lar va qiyaliklardan amaliy Machine Learning masalasiga o'tamiz. **Xatolik (Cost/Loss) funksiyasi nima?** Nima uchun bizga aynan MSE (O'rtacha Kvadratik Xatolik) kerak? Va eng muhimi, qanday qilib ko'r-ko'rona boshlangan model hosila (Gradient) va qadamlar (Learning rate) yordamida haqiqiy va ideal yechimga

(Global Minimumga) qarab xatosiz sirpanib tushishini o'rganamiz. Miyangizni amaliy arxitekturaga tayyorlang, biz Optimizatsiya dunyosiga kirdik!

1-QISM: Biz Qanday Muammoni Yechyapmiz o'zi? (Optimizatsiya Asoslari)

Machine Learning – bu aslida bashorat qilish san'ati emas, bu **Optimizatsiya** (mavjud variantlar ichidan eng zo'rini, eng xatosizini topish) san'atidir. Tasavvur qiling, sizga juda oson bir vazifa berildi: "Uylarning maydoni (Kvadrat metr) berilgan. Ularning narxini bashorat qiluvchi chiziq (Linear Regression) chizib ber!"

Siz jadvalingizdagi nuqtalarni (masalan, 100 kv.m = 50k, 200 kv.m = 100k) grafikga Scatter plot qilib sohib chiqdingiz. Endi ularning o'rtasidan shunday bitta to'g'ri chiziq tortishingiz kerakki, bu chiziq imkon qadar barcha nuqtalarga eng yaqin, eng adolatli bo'lib o'tsin. Maktab algebrasidan ma'lumki, har qanday to'g'ri chiziqning tenglamasi quyidagicha:

$$\hat{y} = w \cdot x + b$$

Bu yerda:

- x — uyning maydoni (Sizga berilgan o'zgartirib bo'lmaydigan ma'lumot).
- $\hat{y}$ (y-hat) — sizning modelingiz aytayotgan (bashorat qilingan) narx.
- w (**Weight - Vazn**) — Chiziqning Qiyaligi (Slope). U x ning ta'sirini ko'paytiradi.
- b (**Bias - Og'ish**) — Chiziqning balandligi (Y o'qini qayerdan kesib o'tishi).

Siz (yoki sizning modelingiz) bilmaydigan va topishi kerak bo'lgan Noma'lum qahramonlar faqat va faqat shu w va b dir. Bu ikkala raqam ML modelning **Parametrlari (Model parameters)** deb ataladi. Agar siz bu ikkala raqamni aniq topsangiz, siz eng zo'r bashorat chizig'ini topgan bo'lasiz. Lekin ular millionlab xil (1.5, -2, 500, 0.001) bo'lishi mumkin. Qaysi biri eng zo'r ekanligini qayerdan bilamiz?

Buning uchun bizga ularning ishiga "Baho beruvchi", ya'ni ularning xatolarini o'lchab beruvchi tarozi kerak! Ana shu tarozi – **Cost Function (Xatolik Funksiyasi)** deyiladi.

2-QISM: Cost Function va MSE (O'rtacha Kvadratik Xatolik)

Tasavvur qiling, model tasodifan (randomly) $w = 2$ va $b = 10$ deb tanladi va bitta "axmoqona" chiziq tortdi. Endi biz bu chiziq qanchalik axmoqona (xato) ekanligini hisoblaymiz: Biz haqiqiy (real) jadvaldagi har bir uyning narxiga (y) qaraymiz va uni model aytgan "soxta" narx ($\hat{y}$) bilan solishtiramiz. Ikkalasi o'rtasidagi farq ($y - \hat{y}$) bu bizning Xatoyimiz (Residual Error).

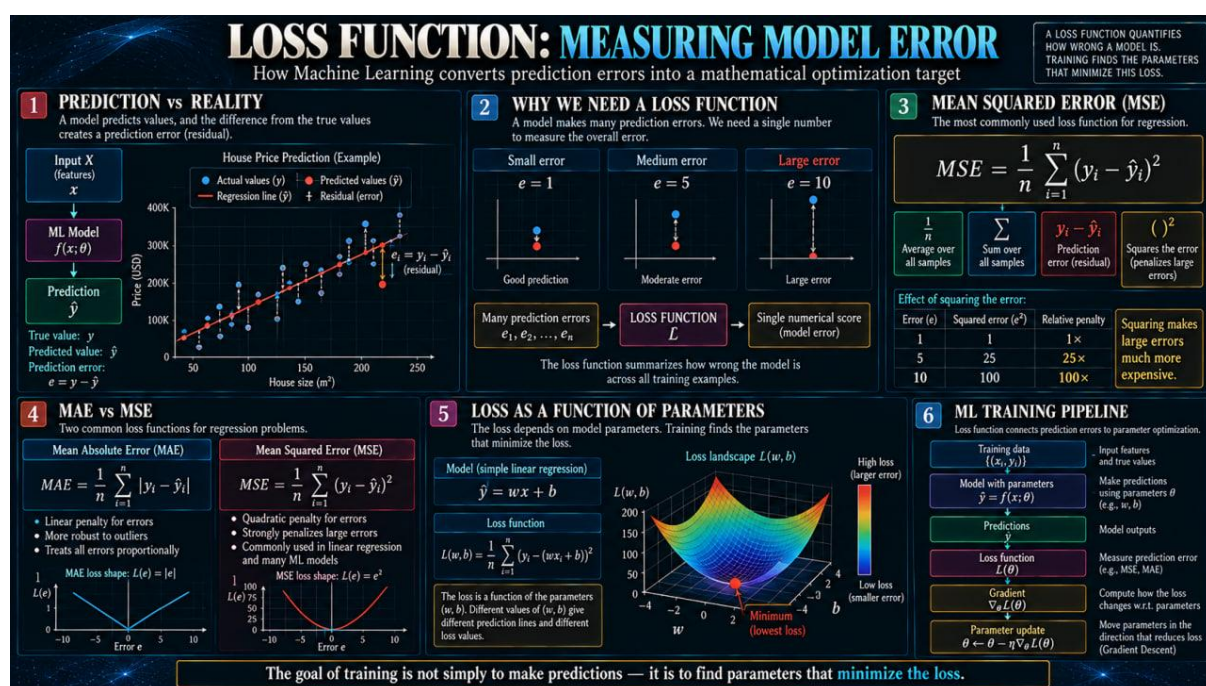
Agar biz hamma xatolarni oddiygina qo'shib chiqsak, musbat (tepadagi uylar) va manfiy (pastdagi uylar) xatolar bir-birini nolga aylantirib, model aslida xato qilayotgan bo'lsa ham "Mening xatom Nolga teng" deb bizni aldaydi. Shuning uchun biz xatolikni, asosan, **Kvadratga (²)** oshiramiz! Kvadrat qilinganda barcha minuslar yo'qoladi, va eng asosiysi — uzoqqa (juda xato) ketgan nuqtalar kvadrat ta'sirida daxshatli darajada kattalashib, modelni "qattiqroq jazolaydi".

Barcha ma'lumotlar uchun shu kvadrat xatoni qo'shib, keyin ularning soniga (N) bo'lib (O'rtachasini olsak), Mashinali O'rganishning eng mashhur va klassik funksiyasi **MSE (Mean Squared Error)** kelib chiqadi. Biz unga ko'pincha $J(w, b)$ (J funksiyasi) deb nom beramiz:

$$J(w, b) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \frac{1}{N} \sum_{i=1}^N (w \cdot x_i + b - y_i)^2$$

Ana endi diqqat qiling! Bu Cost Funksiyasi $J(w, b)$ geometrik qanday shaklda bo'ladi? Siz ko'p x va y larni uning ichiga tiqib qo'yganingiz va xatoni faqat kvadrat (²) ko'rinishida o'lchaganingiz uchun, uning grafigi xuddi kosaga, yoki parabola (chuqurlik) shakliga kelib qoladi! Tog' cho'qqisidan (juda yomon w va b lar, xatolik katta) boshlab, u asta-sekin pasayib keladi va eng o'rtada, eng tubida (Minimumida) o'zining oltin nuqtasiga yetadi.

Bizning yagona va oliy maqsadimiz (Optimization Objective): **Ushbu xatolik kosasining $J(w, b)$ eng pastki, Xatolik (Cost) Eng Kichik bo'lgan tub nuqtasini izlab topishdir!**



3-QISM: Zanjir Qoidasi Amalda (MSE ni Hosila bilan yechamiz)

"Tub nuqtani qidirish" — matematikada "Hosilani nolga tenglash yoki hosila bo'ylab yurish" degani ekanligini bilamiz. Xo'sh, o'tgan darslarda o'rgangan Kalkulyus qoidalarimiz (Zanjir qoidasi va Daraja qoidasi) bu ulkan MSE formulasiga qanday qilib tishini botira oladi?

Keling, jami xatoni faqat w ga nisbatan o'zgarishini topuvchi xususiylar hosila: $\frac{\partial J}{\partial w}$ ni qo'lda keltirib chiqaramiz. Bizda quyidagi ifoda bor (summani va qisqalikni inobatga olmaganda bitta qator uchun): $J = (w \cdot x + b - y)^2$

Qadam-ba-qadam Zanjir Qoidasi (Chain Rule):

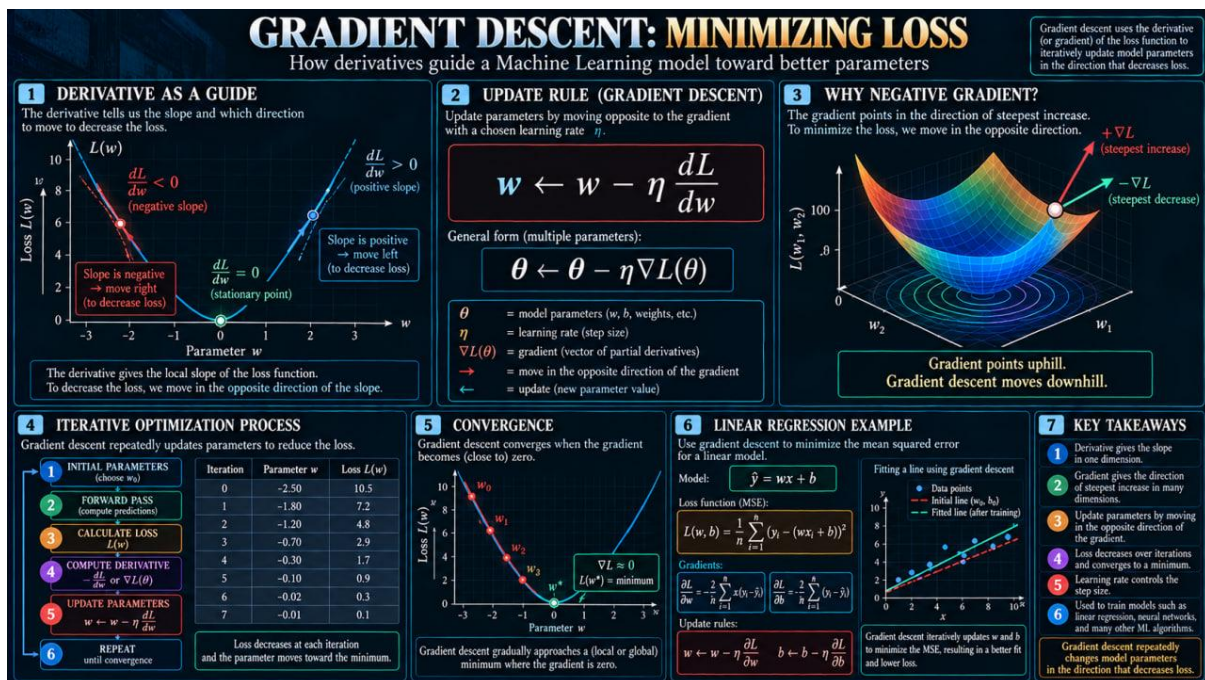
1. **Tashqi funksiya:** Bu yerdagi eng tashqi narsa $()^2$ (kvadrat). Daraja qoidasiga ko'ra, 2 oldinga ko'paytuvchi bo'lib tushadi: $2 \cdot (w \cdot x + b - y)^1$.
2. **Ichki funksiya:** Ichkaridagi ifoda $w \cdot x + b - y$. Biz bunga faqat w ga nisbatan hosila olyapmiz.
 - $w \cdot x$ dan w ga nisbatan hosila: faqat yonidagi x qoladi.
 - b va y lar w ga aloqasi bo'lmagan o'zgarmas (konstant) sonlardek ko'riladi, shuning uchun hosilalari 0 bo'lib ketadi.
3. **Zanjir bo'yicha ko'paytiramiz:** Tashqi natija $\times$ Ichki natija:

$$\frac{\partial J}{\partial w} = \frac{1}{N} \sum_{i=1}^N 2x_i(w \cdot x_i + b - y_i)$$

Huddi shu qoida bilan b uchun ham hosila olamiz (faqat b ning hosilasi 1 ga teng bo'lgani uchun x chiqmaydi):

$$\frac{\partial J}{\partial b} = \frac{1}{N} \sum_{i=1}^N 2(w \cdot x_i + b - y_i)$$

Tabriklayman! Mana shu ikkita yozgan formulamiz Deep Learning'dagi butun boshli Xatoliklarni Orqaga Tarqatish (Backpropagation) algoritmining ilk kurtagi hisoblanadi. Kompyuter har qadamda mana shu formulalarni hisoblab chiqadi.



4-QISM: Nega Oddiy Yo'l Bilan Yecha Olmaymiz?

Siz so'rashingiz mumkin: "Bizda hosilaning aniq formulasi borku! O'sha $\frac{\partial J}{\partial w} = 0$ qilib, tenglamani ishlab w ni bitta qadamda aniq topsak bo'lmaydimi?" (Bunga fanda *Normal Equation* yoki *Analytic Solution* deyiladi). Bu usul 2-3 ta o'zgaruvchi va kichik jadvallar uchun zo'r ishlaydi. U bitta zarba bilan aniq javobni topadi.

Lekin... Neural Network (Neyron Tarmoqlari) da w lar (vaznlar) soni ikkita emas. Ular 10 mingta, 1 millionta, GPT-4 kabi yirik modellarda esa **trillionlab (!)** shunday vaznlar (w) bor! Trillionlab noma'lumli tenglamalar sistemasini Nolga tenglashtirib teskari matritsa (Inverse matrix) usulida yechish uchun dunyodagi barcha kompyuterlarni birlashtirsangiz ham yuz yillar ketib qoladi. Matematik analitik yechim ulkan (Big Data) olamida yaroqsiz qotib qoladi.

Aynan shuning uchun bizga tenglamani teskari yechadigan emas, balki shunchaki har qadamda o'z xatosini bilib, *To'g'ri tomonga aql bilan qadam tashlab* pastga tushadigan Iterativ (qaytariladigan) mexanizm kerak edi. Bu mexanizm — **Gradient Descent (Gradient Nishabligi)** dir!

5-QISM: Gradient Descent Qanday Ishlaydi? (Step-by-Step)

Tasavvur qiling, sizga minglab W lar berilgan qorong'u vodiysiz. O'zgaruvchilar ko'p, funksiya daxshatli. Biz qanday qilib tubini (eng zo'r w larni) topamiz? Algoritm 4 ta mantiqiy qadamdan tashkil topgan:

QADAM 1: Random Initialization (Ko'r-ko'rona boshlash)

Biz hamma w va b largi mutlaqo tasodifiy raqamlar (masalan $w = 0, b = 0$) beramiz. Biz Xatolik tog'ining g'alati, baland bir nuqtasida paydo bo'ldik. O'sha joyda turib, jami ma'lumotlarimiz bo'yicha Xatoyimizni $J(w, b)$ hisoblaymiz (bu Loss yoki Cost deb ataladi). Xato tabiiyki juda baland chiqadi.

QADAM 2: Gradient Vektorini Hisoblash (Kompasni ochish)

Bizga pastga (tubga) olib boradigan yo'nalish kerak. O'tgan darsdan eslaymiz: Xususiylar hosilalardan yig'ilgan **Gradient Vektor** (∇J) har doim o'sish yo'nalishini (yuqorini) ko'rsatadi. Yuqorida biz ko'rgan formulalar bilan kompyuter jami qatorlar bo'yicha $\frac{\partial J}{\partial w}$ va $\frac{\partial J}{\partial b}$ qiyaliklarni hisoblab chiqadi. U Gradient Vektorini (Yo'nalish kompasini) qo'lliga oladi.

QADAM 3: Minus Tomonga Qadam Tashlash (Update Parameters)

Biz Xatoni (Cost) nolga tushirish uchun doim Gradient ko'rsatgan yo'nalishning **Qarama-qarshisiga (Minus tomoniga, $-\nabla J$)** yurishimiz shart! Yurish (Vaznlarni o'zgartirish) formulasi quyidagicha yoziladi:

$$w_{new} = w_{old} - \alpha \cdot \frac{\partial J}{\partial w}$$

$$b_{new} = b_{old} - \alpha \cdot \frac{\partial J}{\partial b}$$

Siz formulaning o'rtasidagi Minus ($-$) belgisini ko'ryapsizmi? Mana shu minus bizni to'ppa-to'g'ri vodiydagi Minimumga qarab sudrab tushayotgan mo'jizaviy mexanizmdir! Agar qiyalik musbat bo'lsa, formula w dan nimadir ayirib uni qisqartiradi. Agar qiyalik manfiy bo'lsa, minusga minus ko'payib, unga qandaydir son qo'shiladi va u o'ngga suriladi.

QADAM 4: Tsiklni Davom Ettirish (Iteration)

Yangi topilgan (ozgina yaxshilangan) w va b lar bilan biz Tog'dan bitta kichkina qadam pastga tushdik. Xatolik ozroq kamaydi. Endi nima qilamiz? Hamma narsa (2-qadam va 3-

qadam) boshqadan boshlanadi! Toki bizning Xatolik funksiyamiz o'zgarishdan to'xtab (Cost minimumga yetib), Gradient (Qiyalik) roppa-rosa 0 ga aylanib qolmagunicha ushbu sikl aylanib yuraveradi (Epoxalar). Tubga yetgach algoritmi qotadi — **Model O'qitildi (Trained)!**

6-QISM: Gradient Descentning Uch Xil Jangavor Turi (Batch, Stochastic, Mini-Batch)

Biz "Qadam 2 da kompyuter JAMI qatorlar bo'yicha xatolikni hisoblab chiqadi" dedik. Lekin jadvalda 10 millionta mijoz bo'lsa, har bitta millimetr qadam tashlashdan oldin butun boshli 10 millionlik jadvalni qo'shib chiqish qanchalik uzoq vaqt olishini tasavvur qilyapsizmi? Buni yengillashtirish uchun fanda Gradient Descent (GD) ning turli "xulq-atvori" ixtiro qilingan:

1. **Batch Gradient Descent (Standart GD):** Model butun boshli 10 millionta qatorni boshidan oxirigacha hisoblab chiqadi, o'rtacha gradientni oladi va shundan KEYINGINA bittagina aniq qadam tashlaydi. U juda silliq va ishonchli pastga tushadi, ammo bitta qadam uchun kompyuter operativ xotirasi (RAM) ga ortiqcha yuk va vaqt sarflaydi. Katta datalarda deyarli ishlatilmaydi.
2. **Stochastic Gradient Descent (SGD - Stoxastik GD):** Bu mutlaq teskarisi. Model 10 million qatorni kutmaydi! U jadvaldan *faqat bitta tasodifiy qatorni* (masalan 1-mijozni) oladi, faqat o'shanga qarab tezkor gradient hisoblaydi va bitta qadam tashlaydi! Keyin 2-mijozni oladi, va yana qadam tashlaydi. Buning yaxshi tomoni — o'rganish tezligi yashin tezligida bo'ladi (10 million qadam bitta iteratsiyaning o'zidayoq qilinish bo'linadi). Yomon tomoni — Uning qadamlari "Mast odamdek" titrab, u yonga bu yonga sakrab, betartib bo'ladi. U tubga yetib kelgach ham, tinch turmasdan markaz atrofida sakrab yuraveradi.
3. **Mini-Batch Gradient Descent (Oltin O'rtalik):** Zamonaviy Machine Learning va Deep Learningda *faqat shu usul ishlatiladi*. Biz datani bitta-bitta ham olmaymiz, butunlay 10 millionni birga ham olmaymiz. Biz ularni guruhlarga (Mini-batch) bo'lamiz! Masalan, 32 ta, 64 ta yoki 128 ta qatordan iborat paketlar qilib jo'natamiz. GPU videokartalari 32 ta ma'lumotni paralel ravishda 1 ta ma'lumot qadar tez hisoblay oladi. Bu ham silliq qadamni ta'minlaydi (shovqin kam), ham xotira tejaladi, ham tezlikni kosmik darajaga chiqaradi!

7-QISM: O'rganish Tezligi (Learning Rate - α) va Uning Sirlari

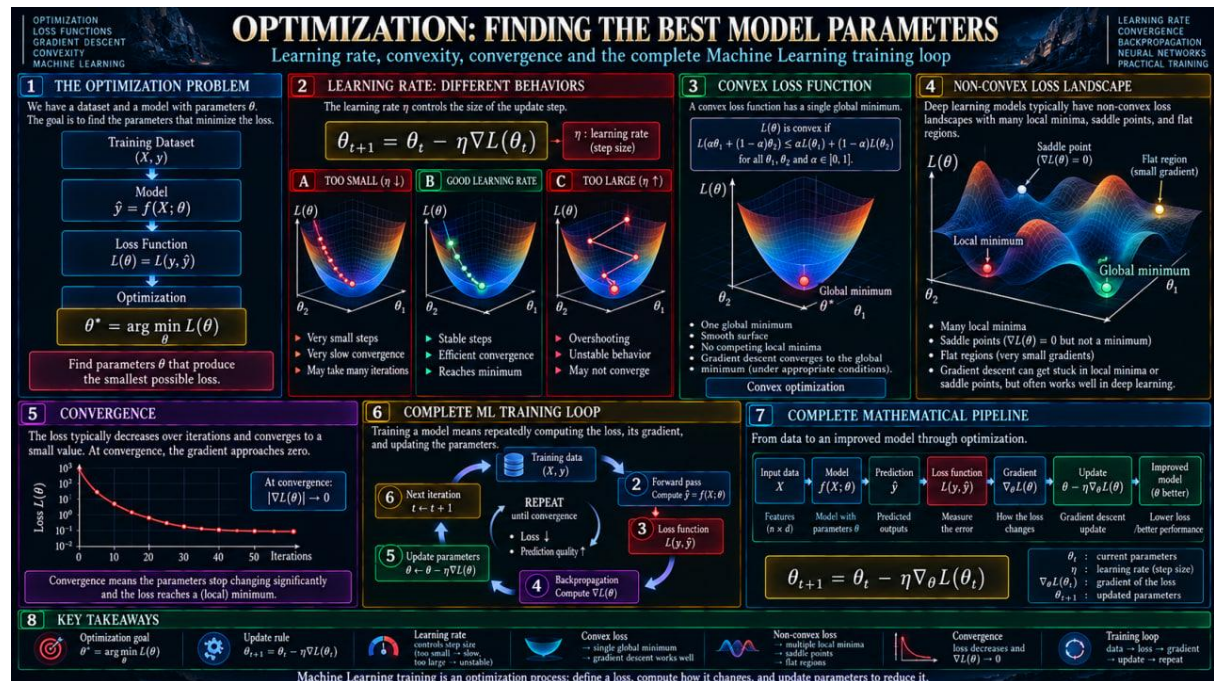
Tepadagi Update (Qadam tashlash) formulasida e'tibor qilsangiz bitta begona grek harfi turibdi: α (Alfa). Bu belgi Machine Learning olamida **Learning Rate (O'rganish tezligi)** deb ataladi va u eng muhim Hyperparameter (Biz o'z qo'limiz bilan sozlaydigan sozlama) hisoblanadi.

Gradient $\frac{\partial J}{\partial w}$ bizga "Qaysi yo'nalishga borish" ni aytadi. Ammo α bizga "Shu yo'nalish bo'ylab qanchalik **KATTA** yoki **KICHIK** qadam tashlashimiz" ni (gaz pedalini) belgilab beradi.

1. **Learning Rate JUDA KICHIK (masalan, $\alpha = 0.0000001$) bo'lsa:** Qadamlaringiz shunchalik mayda bo'ladiki, model to'g'ri ketsa ham tubiga yetib borish uchun oylar va yillar kerak bo'lib qoladi.
2. **Learning Rate JUDA KATTA (masalan, $\alpha = 10$) bo'lsa:** Siz vodiylar markazidan o'tib ketib, narigi devorga urilasiz! Keyingi qadamda yana xato qilib undan ham kattaroq qadam bilan ikkinchi devorga o'tasiz. Siz xatolik tubiga tushish o'rniga, zig-zag

shaklida osmonga (cheksizlikka) o'sib ketasiz. Bu muammo fanda **Divergence (Ochilib ketish / O'ldirish)** deb ataladi. Agar xatoligingiz *NaN* (Not a Number) ga aylanib qolsa, bilingki α ni judayam katta berib yuborgansiz.

3. **Learning Rate Decay (Moslashuvchan qadamlar):** Eng zo'r muhandislik yechimi — qadamlarni o'zgaruvchan qilish! Dastlab tog'ning tepasidan (uzoqdan) kelayotganda α ni kattaroq qilib qadamni yirik tashlaymiz. Lekin nishonga (vodiy markaziga) yaqinlashgan sari kompyuter α ni avtomatik ravishda maydalashtirib, nolga yaqinlashtirib boradi (Learning Rate Decay / Scheduling). Bu tormozni sekin bosib, ayni mo'ljalga taq etib to'xtash imkonini beradi.



8-QISM: Local Minimum va Global Minimum (Yashirin Qopqonlar)

Linear Regression dagi kabi biz chizgan Xatolik (Cost) funksiyasining shakli bitta silliq chuqur kosaga o'xshardi (buni **Convex Function** deymiz). Unda qayerdan boshlasangiz ham, pastga dumalab faqat bitta haqiqiy markazga (tubga) tushib borasiz.

Ammo Neural Networks (Deep Learning) da vaznlar (w) va non-linear aktivatsiya funksiyalari shunchalik ko'pki, bizning Xatolik tog'imiz bitta tekis kosa emas, balki Yuzlab chuqurliklar, past-balandliklar, adirlar va cho'qqilardan iborat "Kraterlar vodiysiga" (Non-Convex) aylanib ketadi!

Mana shu yerda Gradient Descent qopqonga tushishi mumkin:

1. **Local Minimum (Mahalliy chuqurlik / Yolg'onchi tub):** Tasavvur qiling, siz tasodifan bitta tog'ning yonida paydo bo'ldingiz va pastga qarab yura boshladingiz. Siz qandaydir kichikroq chuqurlikka tushib qoldingiz. Atrofingiz o'ralgan, gradient (qiyalik) = 0. Algoritm "Urta, men eng zo'r javobni topdim!" deb qotib qoladi. Lekin aslida haqiqiy eng chuqur, eng xatosiz vodiy (Global Minimum) undan ancha narida joylashgan edi!

2. **Momentum (Inertsia):** Bu xatodan qochish uchun biz Gradient Descent ga Momentum (Tezlanish yig'ish) xususiyatini qo'shamiz. Bu huddi og'ir toshni tepadan dumalatishdek gap. U shiddat bilan dumalab kelayotgani uchun, yo'lidagi kichkina chuqurchalarga (Local minima) tushib qolsa ham, inertsiasini (yig'gan kuchi) hisobiga chuqurchadan sakrab o'tib ketaveradi.

9-QISM: Asosiy Xulosa (Nega buni bilish muhim?)

Qadrli Data Scientist! Hozir biz siz bilan suhbatlashgan ushbu algoritmlar shunchaki kitoblardagi chiroyli teoriya emas. Bugun dunyoni titratayotgan ChatGPT kabi trillion parametrli modellar ham, shifoxonalardagi AI rentgen apparatlari ham, va yuzingizni taniydigan iPhone tizimlari ham — **BARCHASI 100% holatda AYNAN Gradient Descent algoritmi asosida ishlaydi!**

Barcha Machine Learning arxitekturalari mana shu oddiy, qat'iy uchburchak sikl ustiga qurilgan:

1. Ma'lumot kiritiladi va javob olinadi (Forward pass).
2. Xatolik (Cost/Loss - MSE yoki CrossEntropy) orqali biz ideal haqiqatdan qanchalik uzoq ekanimiz o'lchanadi.
3. **Hosila (Calculus)** yordamida Gradient (Qiyalik) topiladi va **Optimizatsiya** (Minus tomonga qadam tashlash) orqali vaznlar yangilanadi (Backward pass).

Siz bu mexanizm ichidagi α (Learning rate) nimaga javob berishini, SGD va Mini-batch orasidagi tejamkorlik farqini va Zanjir qoidasi aslida kompyuterni qanday yurgizishini matematik aniqlik bilan his qildingiz.

Linear Algebra (Matritsalar) yordamida koinot va ma'lumotlar yaratildi. Ehtimollik (Probability) unga mantiq berdi. Kalkulyus (Hosila va Gradient Descent) esa unga **Hayot (O'rganish va aqlli roq bo'lish)** berdi! Siz Machine Learning qanday yashashining fundamentini tamomladingiz. Keyingi darsda biz Hosilaning teskarisi (Integral) nima ekanini o'rganamiz. U bizning modellarimiz o'z imtihonidan (Metrikalardan) qanday o'tishini tasdiqlash uchun kerak bo'ladi. Oldinga olg'a!

ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)

9-Dars: Integral va Egri Chiziq Ostidagi Yuza (Integral and Area Under the Curve)

Assalomu alaykum, kelajakni o'zgartiruvchi Sun'iy Intellekt muhandisi! O'tgan ikkita dars davomida biz Hosila (Derivative) olamida harakat qildik. Biz uni kompyuterning o'rganish (o'z xatosini bilish) mexanizmi (Gradient Descent) uchun qanchalik hayot-mamot masalasi ekanligini ko'rdik.

Lekin Matematik Analiz (Calculus) ning faqat bitta yuzi (Hosila) yo'q. Uning tanga kabi ikkinchi, shuningdek shohona bir qanoti bor. Agar Hosila biron bir narsani (funksiyani)

bo'laklab, qismlarga (laxzali qadamlarga) ajratib tashlab o'rganish san'ati bo'lsa... Uning mutlaq teskarisi bo'lgan jarayon — parchalangan mayda qismlarni qaytadan, chiroyli qilib yig'ish va Yagona butun (Butunlik) ga aylantirish san'ati **Integral** deb ataladi.

Integral bizga nima beradi? ML Engineer sifatida biz Ehtimollar Nazariyasidagi Uzluksiz (Continuous) taqsimotlarni hisoblaganimizda, yoki Modelimiz qanchalik to'g'ri va noto'g'ri javoblarni bashorat qilganini (ROC-AUC ko'rsatkichini) o'lchaganimizda faqat va faqat Integral mantig'iga suyanamiz.

Miyangizni abstrakt chegaralardan "maydon" yoki "yuza" kabi real fizik o'lchovlarga ko'chirishga tayyorlang. Biz "Egri Chiziq ostidagi Yuza" degan sirli jumla 2500 dan ortiq so'z bilan, hayotiy va amaliy sirlarni ochib beruvchi ajoyib sayohatga kirdik! Darsimizni boshladik!

1-QISM: O'zi Nima U Integral? (Asosiy Mantiq)

Sizning oldingizda bitta oddiy to'g'ri to'rtburchak turibdi (eni 4, bo'yi 5). Uning yuzasini topish so'ralgan. Maktab matematikasi bu savolni bir soniyada hal qiladi: $S = eni \times bo'yi = 4 \times 5 = 20$. Buning uchun hatto kalkulyator ham kerak emas.

Lekin koinot, tabiat (va ma'lumotlar bazasi) har doim ham mukammal tekis (to'rtburchak) yoki uchburchak bo'lib kelmaydi. Tasavvur qiling, sizga **yuqori qismi qandaydir g'alati, to'liqinsimon, egri chiziq bilan chegaralangan** tepalikning (yoki yuzaning) aniq maydonini (yuzasini) hisoblash buyurildi. Bu egri chiziqni qandaydir $f(x) = -x^2 + 4x$ degan formula tasvirlaydi. Siz to'g'ri to'rtburchak formulasini ishlata olmaysiz, chunki uning tepasi tekis emas. Geometriyada bunday g'alati shakllarning yuzasini topadigan tayyor "formula" yo'q!

Ana shu cheksizlikka va tilsimotga aylanib qolgan muammoni miloddan avvalgi Yunonistonda Arximed hisoblab ko'rishga uringan, keyinroq esa Isak Nyuton o'zining "Calculus" fani bilan daxshatli va mo'jizaviy sodda usul bilan hal qilib bergan. Bu usul **Integral** deb ataladi.

Integral mantiqan nima degani? Bu shaklni shunchaki bitta katta qilib o'lchamaymiz. Biz uni minglab, millionlab **ipdek o'ta ingichka, tikka turgan to'g'ri to'rtburchaklarga bo'lib (tilkalab) tashlaymiz!** Har bir ingichka to'rtburchakning:

- Balandligi: o'sha nuqtadagi funksiya qiymatiga ($f(x)$ ga) teng.
- Eni (qalinligi): shu qadar ingichkaki, biz uni "juda mayda o'zgarish" deymiz va dx (delta x) bilan belgilaymiz.

Endi bitta ingichka to'rtburchakning (ipning) yuzasi: Balandlik $\times$ Eni $= f(x) \cdot dx$ bo'ladi. Integral amali — ana shu son-sanoqsiz, milyonlab ingichka, ipdek to'rtburchakchalarning barchasini **Bir-biriga qator qilib qo'shib chiqadi (Summation)!** Va yig'indi bizga butun boshli tepalikning mukammal aniq maydonini (Yuzasini) beradi!

Integral belgisi lotincha S ("Summa" - yig'indi) harfining cho'zilgan (kamon kabi) shakli bo'lib, quyidagicha yoziladi: $\int$.

2-QISM: Riman Yig'indisi (Riemann Sum) - Kompyuter qanday hisoblaydi?

Yuqorida biz "millionlab ingichka to'rtburchaklarga bo'lib qo'shib chiqamiz" dedik. Bu matematika falsafasida (Rigorously) nemis olimi Bernhard Riman nomi bilan **Riemann Sum (Riman Yig'indisi)** deb ataladi.

Tasavvur qiling, biz aniq matematikaga kirmasdan, Riman usuli bilan qo'lda yuza hisoblamochimiz. Shaklimiz $x = 0$ dan $x = 4$ gacha cho'zilgan.

1. Biz o'rtani bor-yo'g'i 4 ta qalin (eni $\Delta x = 1$ bo'lgan) to'rtburchakka bo'lamiz. Har birining balandligi $f(x)$ qanday bo'lsa o'shanga teng bo'ladi. Ushbu 4 ta qalin blokning yuzalarini qo'shib chiqamiz. Bu taxminiy, qo'pol, lekin qandaydir natija beradi (masalan, 14). Shaklimizning tepasi egri bo'lgani uchun to'rtburchak uchlari chiziqdan chiqib yoki yetmay qolib "xato (Error)" qiladi.
2. Endi biz to'rtburchaklarni nozikroq qilamiz, ularning sonini 4 ta emas, 10 ta qilamiz (eni $\Delta x = 0.4$). Xatolar sezilarli darajada kamayadi. Yig'indi 11.5 bo'ldi.
3. Keling, 10,000 ta ipdek ingichka (eni $\Delta x = 0.0004$) chiziq tortamiz. Tepadagi burchaklari shakl egrisiga shu qadar chiroyli, zich yopishib qoladiki, ko'zimiz xatoni umuman ilg'amay qoladi!
4. Va nihoyat, Limit (chegara) mantig'ini yoqamiz: To'rtburchaklar sonini $N \rightarrow \infty$ (Cheksizlikka) intiltiramiz, ularning eni esa $\Delta x \rightarrow 0$ (Nolga) intiladi. Mana shu jarayon uzluksiz, aniq (mo'jizaviy qat'iy) maydonni tug'diradi!

$$\text{Area} = \lim_{n \rightarrow \infty} \sum_{i=1}^n f(x_i) \cdot \Delta x = \int f(x) dx$$

$$\text{Area} = \lim_{n \rightarrow \infty} \sum_{i=1}^n f(x_i) \cdot \Delta x = \int f(x) dx$$

Bu formula nega muhim? Chunki Pythonda (**NumPy** va **SciPy**) kompyuterlar qandaydir tasavvur qilib bo'lmaydigan qonuniyatlar bilan emas, balki aynan shu (Riman) usulida mitti bo'laklarni (numerical integration) soniyaning milliardon bir bo'lagida qo'shib yig'ib beradi. O'rganilgan har bir narsa asosida shu qattiq arxitektura yotibdi.

3-QISM: Hosila va Integral (Antiderivative) Ning Yagona Olamligi

17-asrdagi Matematika olamidagi eng aql bovar qilmas kashfiyot shu bo'ldiki (Nyuton va Leybnits buni bir-biridan bexabar bitta vaqtda kashf etishdi), yuqorida biz ko'rgan mayda qismlarni yig'ish (Integral) amali — biz o'tgan darsda o'rgangan Hosila (Qiyalik topish) amalining **Mutlaqo va Aniq TESKARISIDIR (Antiderivative)!**

Bu Kalkulyusning Asosiy Teoremasi (Fundamental Theorem of Calculus) deb ataladi. U aytadiki: Agar siz biron bir $F(x)$ funksiyadan Hosila olsangiz, u $f(x)$ ga (tezlikka) aylanadi. Agar siz shu chiqqan $f(x)$ tezlikdan Integral olsangiz (yuzasini hisoblasangiz), u xuddi Sehgarning tayoqchasi kabi yana qaytib boshlang'ich $F(x)$ funksiyaning o'ziga aylanib qoladi!

Bu tushuncha huddi oldinga qadam bosish va uni orqaga qaytarish kabi mutanosibdir. Biz Hosila qoidalarini (Derivative Rules) o'rgangan edik. Integralni yechish o'sha qoidalarni "teskarisiga" (Orqaga qarab) o'qishdir!

Hosiladan Integralga aylanish (Daraja Qoidasi - Power Rule): Hosila olayotganda biz:

Darajani oldinga tushirib, tepadan bitta (1) ayirib qo'yar edik. $\frac{d}{dx} x^3 = 3x^2$. Integral

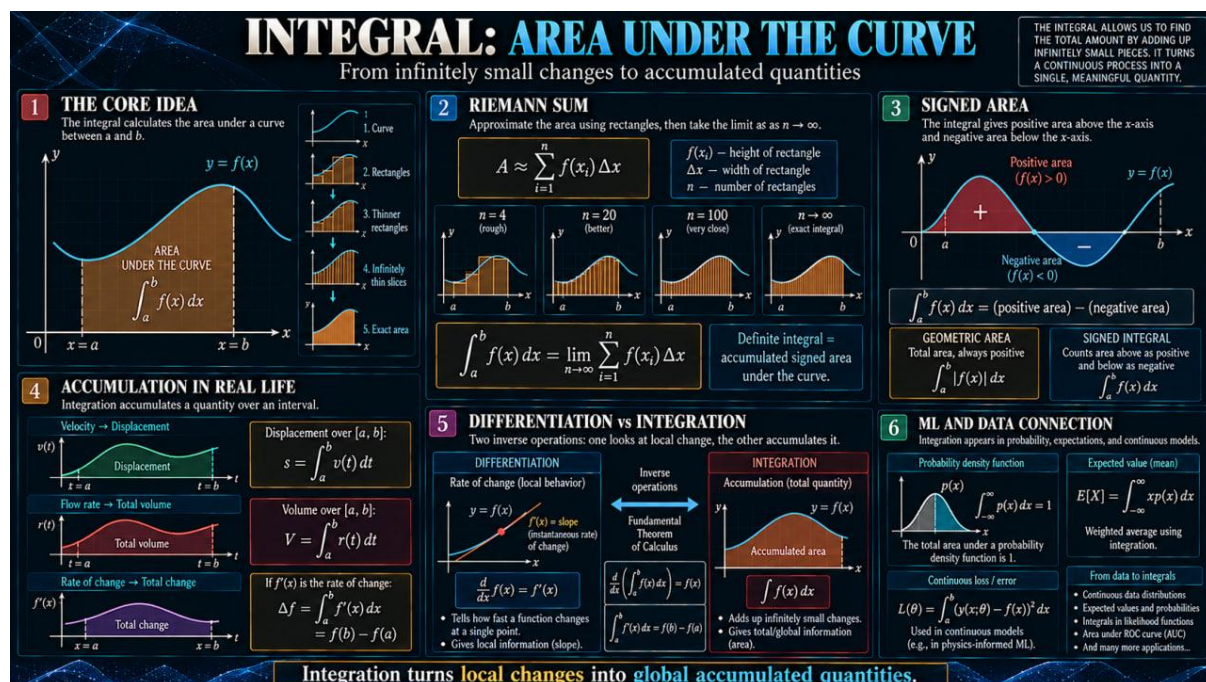
olayotganda biz bu ishlarni teskari tartibda qilamiz: **Avval x ning tepasidagi darajaga bitta (1) qo'shamiz, keyin chiqqan yangi darajani oldinga tushirmaymiz, balki o'sha raqamga bo'lib (/) yuboramiz!**

$$\int x^n dx = \frac{x^{n+1}}{n+1} + C$$

Misollar:

- $\int x^2 dx = \frac{x^{2+1}}{2+1} = \frac{x^3}{3} + C$
- $\int x^4 dx = \frac{x^5}{5} + C$
- $\int 1 dx$ (ya'ni oddiygina sonning yuzasi) $= 1x = x + C$ (Chunki x ning hosilasi 1 bo'lardi!).

Tepadagi C (Constant - O'zgarmas son) nimani anglatadi? Esingizdami, $f(x) = x^2 + 5$ va $f(x) = x^2 + 10$ ni hosilasi bir xil $2x$ chiqadi. Chunki yonidagi o'zgarmas sonlar (5 yoki 10) Nol bo'lib yo'qolib ketardi. Endi siz faqatgina tayyor $2x$ ga qarab turib, uni Integral qilib asliga (x^2 ga) qaytardingiz, lekin uni oldin qornida $+5$ bormidi, yoki $+10$ bormidi (Nol bo'lib ketsa izi qolmaydi) hech qachon aniq ayta olmaysiz! Ana shu "Mavhum va sirli, yo'qolib ketgan, endi aniqlab bo'lmaydigan raqamni" matematiklar ehtiyotkorlik yuzasidan bitta katta C (**Integration Constant**) harfi bilan ifodalab qo'yishadi. Agar qo'limizda yetarli "Boshlang'ich shartlar (Initial conditions)" bo'lsa, biz keyinchalik o'sha C aynan necha ekanligini topib olamiz. (Bunday pinhona Integral "Aniqmas Integral - Indefinite Integral" deyiladi).



4-QISM: Aniq Integral (Definite Integral) va Uni Hisoblash

Xo'p, formulalarni teskari aylantirish chiroyli. Lekin boshidagi muammo: "Men konkret egri chiziq va yer orasidagi Aniq Maydonni (Yuzani/Kvadrat metrini) raqamlarda topishim kerak" bo'lsa-chi? Unda men qanday qilib C lar aralashib yurgan mavhum holatdan aniq $S = 20.5$ degan javobga kelaman?

Buning uchun biz **Aniq Integral (Definite Integral)** ni ishlatamiz. Uning oddiy (aniqmas) integraldan farqi shundaki, Integrash $\int$ belgisining pasiga va tepasiga bittadan son yozilgan bo'ladi. Ular **Limitlar (Chegaralar)** deb ataladi. Pastdagi chegara a (qaerdan boshlaymiz?), tepadagi chegara b (qayerda tugatamiz?).

$$\int_a^b f(x)dx$$

Buni yechish insoniyat dahosining eng ajoyib formulalaridan biri (Nyuton-Leybnits formulasi) bilan qilinadi. Qoida oddiy:

1. Birinchi navbatda $f(x)$ ni integral (antiderivative) formulasini ishlatib "Boshlang'ich funksiyasi" $F(x)$ ni (C harfisiz) topamiz.
2. Endi topilgan yangi $F(x)$ qolipning ichiga, x ning o'rniga eng Tepadagi (ohirgi manzil) b raqamini qo'yib hisoblaymiz. Javobini eslab qolamiz.
3. Shundan so'ng x ning o'rniga Pastki a (boshlang'ich nuqta) raqamini qo'yib hisoblaymiz. Javobini eslab qolamiz.
4. Va ikkala chiqqan raqamni bir-biridan Olib tashlaymiz (Ayiramiz)! Eng tepadagisidan, Pastdagisini!

Nyuton-Leybnits formulasi:

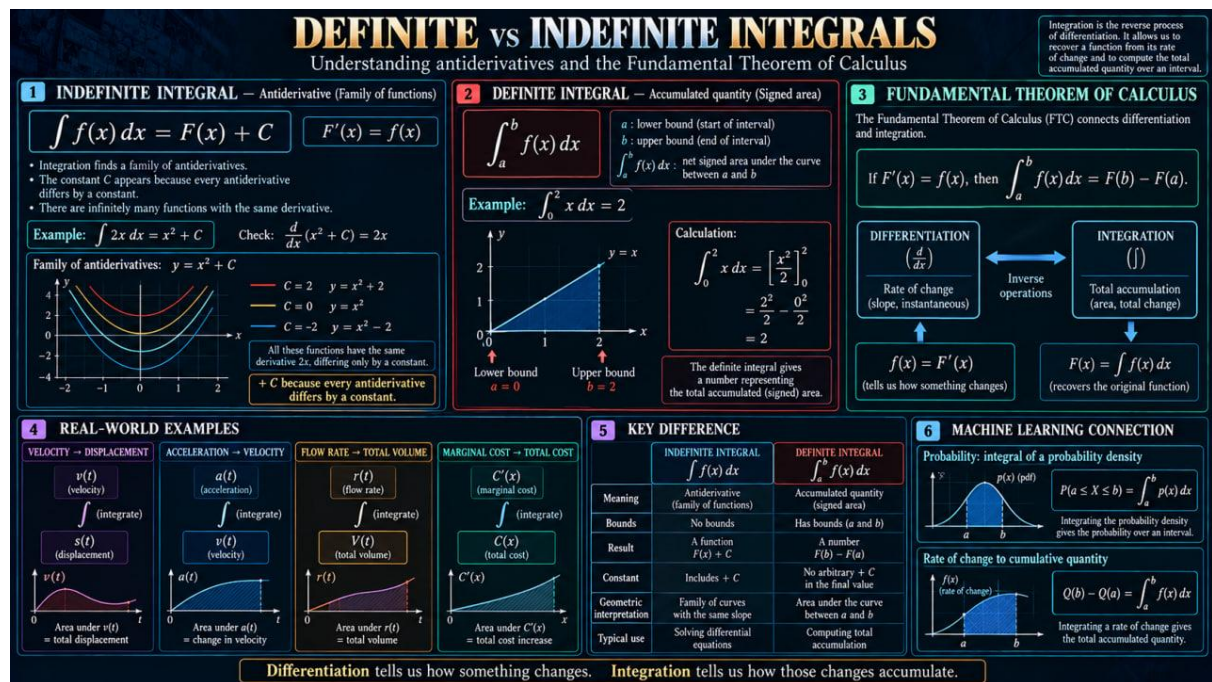
$$\int_a^b f(x)dx = F(b) - F(a)$$

Amaliy raqamli misol: Aytaylik bizga $f(x) = 2x$ funksiyasining $x = 1$ nuqtadan to $x = 3$ nuqtagacha bo'lgan qismining (yer bilan oradagi) yuzasi kerak! (Aslida bu trapetsiya shakli, geometriya orqali topsa ham bo'ladi, biz Integralni sinaymiz).

1. Boshlang'ich funktsiyani topamiz (Teskari hosila): $2x$ qachon kelib chiqadi? Albatta $\frac{2x^2}{2} = x^2$ dan. Demak $F(x) = x^2$.
2. Limitlarni qo'yamiz ($a = 1$, $b = 3$). Formulaga asoslanamiz: $F(3) - F(1)$.
3. Tepadagi raqamni qo'yamiz: $F(3) = 3^2 = 9$.
4. Pastdagi raqamni qo'yamiz: $F(1) = 1^2 = 1$.
5. Ayiramiz: $9 - 1 = 8$.

Bo'ldi! Ana shu $f(x) = 2x$ to'g'ri chizig'ining ostidagi,

oralig'idagi shaklning Haqiqiy aniq Yuzasi **8 kv.birlik** ekan! Va hech qanaqa qiyin limitlar yoki taxmini Riman to'rtburchakchalarini chizib chiqishga, qotishga xojat yo'q. Qoida ishlaydi!



5-QISM: Raqamli Integrallash (Numerical Integration) va Pythonda Yechish

Yuqoridagi Nyuton-Leybnits usuli (Analytic method) agar sizda funksiyaning tayyor chiroyli formulasi (masalan, $x^2, \sin(x)$) bo'lsagina ishlaydi. Lekin Data Scienceda bizda ko'pincha biron "qotib qolgan formula" bo'lmaydi. Bizda faqatgina minglab qatorli (Data pointlardan iborat) jadval (CSV) bo'ladi xolos! Ma'lumotlarning aniq uzluksiz funksiyasi (chizig'i) yo'q.

Bunday paytda biz yana tarixga — **Riman to'rtburchakchalarini qo'shish (Numerical Integration) usuliga** qaytishga majburmiz. Bu holda biz integrallarni nuqtama-nuqta jadvallash asosida qo'shib (Trapetsiya qoidalari yoki Simpson qoidalari orqali) hisoblaymiz. Buni hech kim qog'ozda qilmaydi, buni Python o'ta katta tezlikda yechib beradi.

Pythonda bunga ikkita ajoyib kutubxona xizmat qiladi:

1. SymPy orqali aniq formula bilan ishlash (Analitik usul): Aytaylik, biz $f(x) = x^2$ funksiyaning $x = 0$ dan $x = 5$ gacha integralini (yuzasini) olmoqchimiz.

```
import sympy as sp
```

```
x = sp.Symbol('x')
```

```
f = x**2
```

```
# Aniqmas integral (faqat formula, C siz)
```

```
indef_integral = sp.integrate(f, x)
```

```
print("Boshlang'ich funksiya: ", indef_integral) # x**3 / 3
```

```
# Aniq Integral (Maydon yuzasini hisoblash 0 dan 5 gacha)
def_integral = sp.integrate(f, (x, 0, 5))
print("Maydon yuzasi: ", def_integral) # 125/3 (ya'ni ~41.66)
```

2. SciPy orqali faqat Jadvaldagi (Data) nuqtalardan yuza topish (Raqamli usul): Agar bizda formula emas, tayyor x (masalan, vaqt) va y (masalan, tezlik) nuqtalari mavjud bo'lsa (jadval ko'rinishida), biz bu nuqtalar ustiga tortilgan ko'rinmas chiziq ostidagi yuzani `scipy.integrate.trapz` (Trapetsiya qoidasi) yordamida hisoblaymiz.

```
import numpy as np
from scipy.integrate import trapz
```

```
# Bizda formula emas, faqat Data points (Jadval) bor
x_data = np.array([0, 1, 2, 3, 4, 5])
y_data = np.array([0, 1, 4, 9, 16, 25]) # Bu aslida x^2 nuqtalari
```

```
# Egri chiziq ostidagi yuzani Trapetsiyalar orqali taqribiy hisoblash
area = trapz(y_data, x_data)
print("Data asosidagi Yuza (Area): ", area) # Javob 42.5 chiqadi.
# Taxminan 41.66 ga juda yaqin! Qatorlar qancha ko'p bo'lsa, xato shuncha kamayadi.
```

Klassik Machine Learning va Data Analitikada (Ayniqsa AUC-ROC kabi metrikani hisoblashda) kompyuter faqat va faqat ikkinchi usuldan (Raqamli integrallashdan) foydalanib yuzalarni qo'shib chiqadi.

6-QISM: ML va Ehtimollik Nazariyasidagi Integrallarning Ahamiyati

Siz buni o'qirkansiz "Xo'p, matematik qiziqarli ekan, lekin Neyron tarmoqlarda bizga Hosila (Gradient) ni topib, vaznlarni o'zgartirsa (o'rgasak) yetardi-ku? Integrallarni nega aralashtiryapmiz?" deb o'ylashingiz tabiiy. Integrallar to'g'ridan-to'g'ri o'qitishda (Training) ishlatilmasada, ular atrof-muhit va Metrikalar qurilishida hal qiluvchi (podshoh) qoidalarga ega. Ularning Data Sciencedagi 2 ta yirik va klassik aloqadorligini yoritamiz:

A. Probability Density Function (Uzluksiz Ehtimollik Yuzasi)

Ehtimollar nazariyasi (Probability) kursimizning 3-darsini eslang. Biz X tasodifiy miqdor **Uzluksiz (Continuous)** bo'lgan (masalan, odamlarning bo'yi, mashinalar narxi, vaqt kabi to'xtovsiz raqamlar oraliqlarida), aniq bir nuqtaning (masalan, bo'yi aynan 175.000000 sm bo'lishi) ehtimolligi har doim nol (0) ga teng deb yozgan edik! Nima uchun?

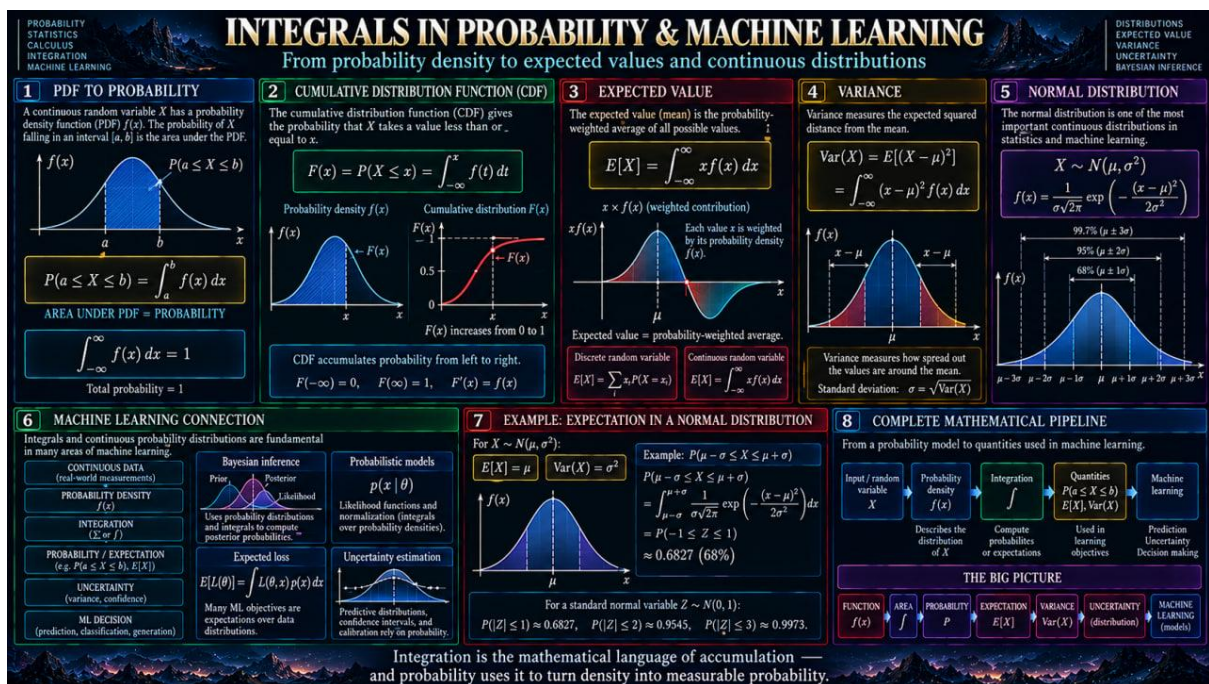
Chunki chiziqdagi bitta nuqtaning eni yo'q, va Integral ostidagi bitta nuqtaning **Yuzasi** (Maydoni) bo'lmaydi. Nol yuza — nol ehtimol degani. Shuning uchun uzluksiz dunyoda har doim "Ushbu qiymatning A (170) va B (180) oralig'ida bo'lish (tushish) ehtimolligi qancha?" deb so'raladi va u ehtimollik zichligi funksiyasi — **PDF ($f(x)$)** orqali hisoblanadi. Va o'sha A dan B gacha tushish Ehtimolligini to'g'ri topish — ularning PDF egri chizig'i ostidagi **Yuzasini hisoblash, ya'ni INTEGRAL qilishdan** boshqa narsa emas!

$$P(a \leq X \leq b) = \int_a^b f(x)dx$$

Bundan tashqari, Ehtimollar dunyosining eng muqaddas, buzilmas oltin qoidasi bor: O'sha uzluksiz PDF egri chizig'ining Minus Cheksizlikdan ($-\infty$) to Plus Cheksizlikkacha ($+\infty$) cho'zilgan hududining jami, hamma (yagona, butun) Yuzasi (Integrali) majburiy ravishda roppa-rosa **1 (100% Ehtimollik)** ga teng bo'lishi ShART! Agar u 1 ga teng bo'lmasa, u umuman ehtimollik funksiyasi bo'la olmaydi.

$$\int_{-\infty}^{+\infty} f(x)dx = 1$$

ML modellar natija (Probability prediction) chiqarganda, ular o'z bilimi ostidagi jami variantlarni Integral (Area=1) deb biladi. Shu kabi shartli va majburiy normallashtirishlarning yuragi yana Integral bo'lib qoladi.



B. Classification Metrikasi: ROC-AUC (Area Under the Curve)

Machine Learningning Supervised Classification (Klassifikatsiya) modellarida, masalan, Logistic Regression, model biron bir emailni "Spam" yoki "Spam Emas" deb topishi kerak. Va biz modelni qanchalik zo'r ishlaganini baholaymiz (Evaluate). Bu erda ko'pchilikni chalkashtirib qo'yuvchi mashhur grafika — **ROC Curve (Receiver Operating Characteristic)** chiziladi.

ROC grafigida modelning Xatoligi (False Positive Rate) X o'qida, Haqiqiyliigi (True Positive Rate) Y o'qida yotadi. Bu chiziq model turli xil chegaralar (thresholds) ni o'zgartirganda qanday moslashishini ifodalaydi. Modelning qanchalik mukammal (Yoki ahmoqona) ekanligini anglatuvchi bitta donagina yakuniy Rakam (Score) bor va u **AUC (Area Under the ROC Curve)** deb ataladi. Tarjimasi: "Egri chiziq ostidagi Yuza"! Siz anglab yetgan bo'lsangiz kerak, Scikit-Learn kutubxonasidagi `roc_auc_score(y_true, y_pred)` funksiyasi model ortiga

qaytib ROC egrisini yasaydi va o'z qornining ichida aynan biz hozir o'rgangan Trapetsiya yoki Integral (Summation) formulasini ishlatib, uni 0 va 1 (quti yuzasi $1 \times 1 = 1$) orasidagi bitta Rakamga (Area Score ga) aylantiradi. Agar AUC Integrali 0.5 (yuzaning yarmi) chiqsa — model ko'zi o'jizdek tasodifiy (50/50, tanga kabi) bashorat qilyapti degani, uni tashlab yuboring. Agar AUC Integrali 1.0 (Maksimal hamma maydonni o'zlashtirgan yuza) bo'lsa — modelingiz "Oliygo'h daho talabasi" dek **100%** mutlaq xatosiz bashorat qilmoqda degani! Integral kompyuterning imtihon bahosiga aylandi.

Xulosa (Conclusion)

Qadrli Data Scientist va Intellekt muhandisi, tabriklayman! Bugun biz Kalkulyus koinotining eng abadiy masalalaridan biri — uzuq yulib, maydalanib ketgan haqiqatlarni bitta Butun kunga qanday yig'ib chiqish (Integration) san'atini kashf etdik.

Siz nafaqat Arximedning qattiq g'oyasi, Riman yig'indisining "chiziqlarga tilish" logikasini his qildingiz, balki eng asosiy Fundamental Qoida — Integral amali bu shunchaki **Teskari Hosila** (Antiderivative) ekanligini, orqaga vaqt mashinasida qaytish ekanligini tushunib oldingiz. Siz Aniq (chegarali) Integrallar orqali qanday qilib egri va mavhum yuzalarni aniq bir soniyali (matematik va raqamli Code) javoblarga aylantirish mumkinligini kashf qildingiz. Biz bu Yuzaning Ehtimollik Nazariyasi o'zgaruvchilaridagi va AI Klasifikatsiya Modelining bashorat metrikalaridagi (ROC-AUC) hal qiluvchi (jazo yoki baho) roliga dildan ishonch xosil qildik.

Biz faqat bitta o'zgaruvchili (bitta x li) 2D grafiklar bilan cheklanib o'tira olmaymiz. Real ML modellari o'n minglab va yuz minglab x (Data features) o'zgaruvchilar, parametrlar bilan ishlaydi. Keyingi darsda (10-Dars: Ko'p o'zgaruvchili funksiyalar) biz to'g'ridan-to'g'ri ko'p o'lchamli qabariqlar va 3D tog'larga sakraymiz, va gradientlarni bu yangi va daxshatli olamga qanday ko'chirishni o'rganamiz. Yuzalarni hisoblashdagi zo'r kashfiyotlaringiz siz bilan qolsin, endi koinotning ko'p o'lchamli kengliklariga olg'a!

ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)

10-Dars: Ko'p O'zgaruvchili Funksiyalar (Multivariable Functions)

Assalomu alaykum, bilimdon va to'xtamas Machine Learning muhandisi! O'tgan darsimizda (Integrallar bilan) biz Kalkulyus olamidagi so'nggi eng asosiy mantiqiy fundamentni yopib oldik. Shu kungacha biz Kalkulyus qonunlarini juda sodda, ikki o'lchamli dunyoda — $f(x)$ ya'ni Y faqat va faqat bitta X ga bog'liq bo'lgan (qog'ozdagi yassi chiziqlar) dunyosida kashf etib keldik.

Ammo Real Hayot ham, Data Science ham qog'ozdek yassi emas! Bir o'ylab ko'ring: Uylarning narxi (Y) faqatgina uning xonalar soni (X_1) ga qarab o'zgaradimi? Yo'q. Uning joylashgan manzili (X_2), maydoni (X_3), qavat (X_4) kabi o'nlab, yuzlab parametrlar Y ga ta'sir qiladi! Demak, model bitta xato chizig'ini emas, balki bir paytning o'zida yuzlab o'qlar orasida

chalkashib ketgan g'alati Tog'larni, vodiylarini va ko'p o'lchamli koinotlarni kechishiga to'g'ri keladi.

Kalkulyus qanday qilib o'sha bitta X uchun topilgan qoidalarini, milliardlab parametrli yirik modellar olamiga ko'chirib o'tkazadi? Bugun biz to'liq 3D (va undan yuqori) o'lchamli dunyoga kirib boramiz: **Ko'p o'zgaruvchili funksiyalar, Xususiy Hosilalar (chuqurlashtirilgan) va barchasini birlashtiruvchi Gradientlar**. Miyangizni yassi chiziqlardan hajmli vizualizatsiyaga sozlang, biz kompyuterlar tafakkur qiladigan olamning to'liq manzarasi tamon sakraymiz!

1-QISM: Ko'p O'zgaruvchili Funksiya o'zi nima? (Multivariable Function)

Matematikada **Multivariable Function** — bu bir nechta turli xil Kirish (Input) o'zgaruvchilarini qabul qilib, ularni qandaydir qayta ishlab, Chiqish (Output) da faqat va faqat bitta yagona javob (skalyar son) chiqarib beradigan funktsiyadir.

- 1D Funksiya (Qog'ozdagi grafik): $y = f(x)$. Faqat bitta x bor.
- 2D Funksiya (Fazodagi Sirt / Surface): $z = f(x, y)$. Ikkita mustaqil o'zgaruvchi (Input) bor: x va y . Javob esa balandlik bo'lib z o'qiga tushadi. Geometrik tarzda bu huddi yassilangan pol (x va y o'qlari tekisligi) tepasida qotib qolgan Tog'lar va Adirlarga o'xshash uch o'lchamli ko'rpa, sirt (Surface) hosil qiladi.
- n -D Funksiya (Hypersurface): $L = f(x_1, x_2, x_3, \dots, x_n)$. Agar uchta va undan ko'p kiruvchi ma'lumot bo'lsa, u to'rt, besh, million o'lchamli giperfazo (Hyperspace) hosil qiladi. Inson ko'zi uni ko'ra olmaydi (biz 3D maxluqlarmiz), lekin Matematika va Matritsalar uchun ularni osongina formula bilan hisob-kitob qilishning umuman farqi (qiyinchiligi) yo'q.

ML Intuitsiyasi (Cost Function / Loss Function): Siz Chiziqli regressiyani quryapsiz.

Sizda yuzlab o'zgaruvchilar (Vaznlar - Weights: $w_1, w_2, \dots$ va Bias: b) bor. Ularning ma'lum bir miqdorini tanladingiz. Bu tanlovlarning jami natijasi "Modelning Xatosi - Loss" bo'lib (faqat bitta yagona Raqam) chiqadi. Ana shu Loss function $J(w_1, w_2, \dots, w_n, b)$ aslida tipik, yirik **Ko'p o'zgaruvchili Funksiyaning o'zginasi ekan!** Barcha optimizatsiya algoritmlarimiz (Gradient Descent) ana shu ulkan ko'p o'lchamli Sirt (Surface) ustida sayr qiladi.

2-QISM: Vizualizatsiya - Biz Ularni Qanday Ko'ramiz? (Contour and Surface Plots)

Ko'z oldimizda bitta shunday funktsiya berildi: $f(x, y) = x^2 + y^2$. Ushbu tog' manzarasi qanday shaklda? Buni ko'rishning Python'da (va matematik tasavvurda) ikkita yirik asosiy usuli bor:

A. Sirt Chizmasi (Surface Plot - 3D yondashuv)

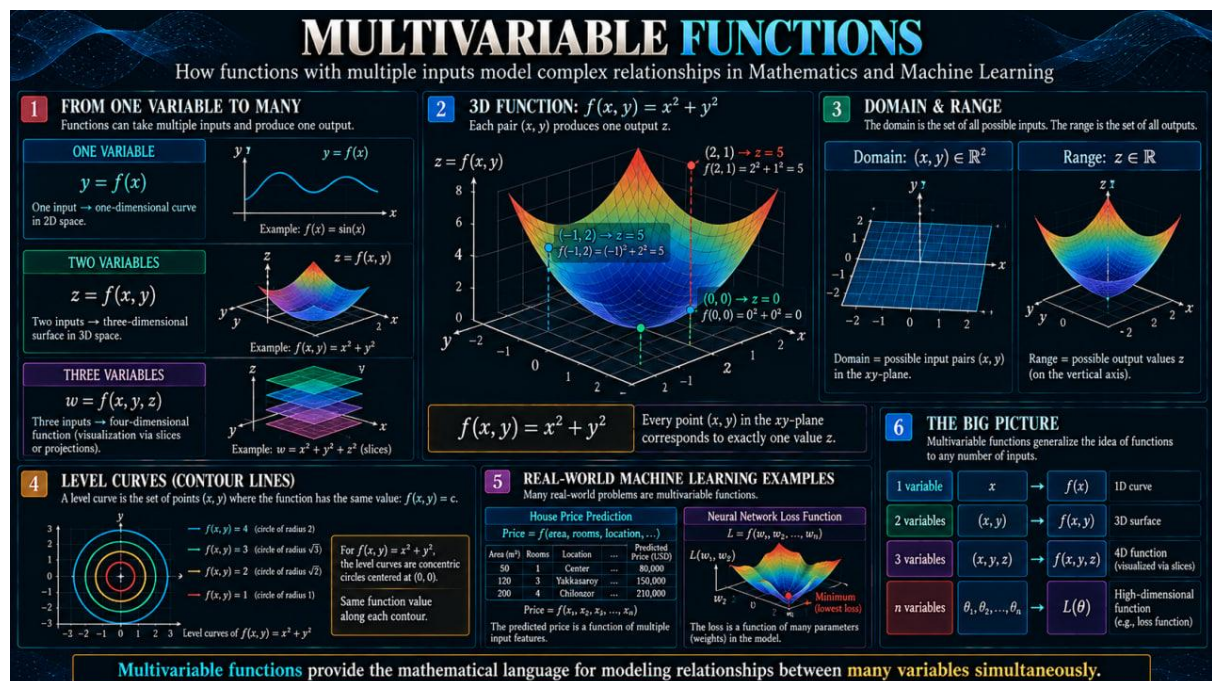
Biz pastga tekis pol shaklida X va Y o'qlarini chizamiz. Ularning ustidan vertikal balandlik tarzida uchinchi — Z o'qini tiklaymiz. Agar biz x va y ga turli xil $(0,0)$, $(1,2)$, $(-3,4)$ nuqtalarni (koordinatalarini) qo'yib, javobi qancha balandlikda chiqishini topib (masalan $f(-3,4) = (-3)^2 + 4^2 = 9 + 16 = 25$), 25 qavat balandlikka bitta tosh qoyib kelaversak... va minglab toshlarni qator terib chiqsak — bizda dum-dumaloq, kosasimon Kosa / Chuqurlik (Paraboloid) shakli hosil bo'ladi. U haqiqiy 3D grafik (Surface Plot) deb ataladi. U joylashuvni eng aniq vizual ko'rsatib beruvchi ajoyib model.

B. Kontur Chizmasi (Contour Plot - 2D Yassi yondashuv / Xarita)

Ammo kompyuter yoki kitob sahifalari 2D (yassi) bo'lgani uchun, ba'zan 3D rasmni atrofini to'liq ko'rish qiyin bo'ladi. Bunday paytda Geologlar va ML Muhandislari (Optimizationni ko'rish uchun) **Kontur xaritalardan** foydalanadilar! Kontur grafiki nima o'zi? Tasavvur qiling, biz boyagi $f(x, y) = x^2 + y^2$ (Kosa) ning ichiga pichoq tiqib, uni roppa-rosa poldan (Z balandligidan) 10 m balandlikdan, 20 m balandlikdan parallel qilib **Yotiq tilkalarga (tilik-tilik qilib)** kesib tashladik.

- $Z = 10$ balandlikdan kesilgan hamma joyi dumaloq (aylana) bitta uzluksiz chiziq bo'ladi.
- $Z = 20$ balandlikdan kesilgan yana kattaroq (boshqa) aylana bo'ladi.

Va biz bu kesib, ranglab olingan qavatlarini shunchaki YUQORIDAN (Osmondan pastga qarab, qush uchishi balandligidan) qog'ozga yassilab ezib yopishtiramiz! Natijada biz turli o'lchamdagi aylanalar bir-birining ichiga kirib ketgan niqon (xarita) grafikasiga (Contour Plot) ega bo'lamiz. Aylanalar ustiga o'sha 10,20,50 kabi balandliklari (Loss darajasi) yozib qoyiladi. Agar aylanalar (chiziqlar) bir-biriga juda zich, tiqilinch joylashgan bo'lsa — u joy o'ta qiya (tikka jarlik) ekanligini, chiziqlar keng-keng ochiq-yoyiq bo'lsa — joy yassiroq (tekisroq qiyalikda) ekanligini anglatadi. ML da eng chuqur, eng xatosiz tub nuqtani topish (Optimizatsiya markazini topish) shu kabi halqalar ichma-ich kirib ketgan eng kichkina nuqta bo'lib ko'rinadi (Masalan ko'zlangan Nishon!).



3-QISM: Xususiyl Hosilalar (Partial Derivatives) Mantiqiy Qiyalikni Topish

Tasavvur qiling siz Loss $J(w, b)$ tog'ining yonbag'ridagi qandaydir nuqtada adashib qoldingiz (ko'zingiz bog'langan). Pastda vodiy (minimum tubi) yashiringan, lekin u qaerdaligini ko'rmaysiz. O'tgan darsdagi oddiy 2D (yassi chiziq) dagi $f'(x)$ hosilasi sizga yordam bera olmaydi, chunki o'ngga-chapga emas, oldinga-orqaga, diagonaliga va hokazo cheksiz ko'p yo'nalishlar mavjud.

Qiyalik (Tezlik, Slope) ni qanday o'lchaymiz? Biz uni faqat bitta ma'lum bir o'qqa (yo'nalishga) bog'lab o'lchashga majburmiz! Fanda buni **Xususiy Hosila (Partial Derivative)** deb ataladi. Va u egilgan "d" (del) (∂) harfi bilan yoziladi. Mantiq sodda:

1. $\frac{\partial f}{\partial x}$ (**x ga nisbatan hosila**): Bu sizning Shimol va Janub (faqat X o'qi bo'ylab) oldinga yoki orqaga 1 qadam yurganingizda tog'ning tepaga yoki pastga tushish qiyaligi tezligini o'lchaydi. Uni hisoblash siri nimada edi? Xuddi o'sha vaqtda Sharq-G'arbga yurmaysiz (Y o'qi y doim doimiy, Constant bo'lib muzlab turadi)!
2. $\frac{\partial f}{\partial y}$ (**y ga nisbatan hosila**): Bu sizning Sharq va G'arb (faqat Y o'qi) bo'ylab yonboshga tashlagan kichkina qadamingiz (qiyalik) qandayligini o'lchaydi. U paytda o'z-o'zidan X o'qi (x) Constant, ya'ni muzlagan o'zgarmas son bo'lib qoladi.

Algebraik amaliyot (Murakkabroq formula bilan ko'ramiz): Funksiya berilgan: $f(x, y) = 2x^3y^2 - \cos(y) + 5x$

- $\frac{\partial f}{\partial x}$ ni qidiramiz (y larni raqam deb faraz qilamiz!): Birinchi qism ($2x^3y^2$) da y^2 qoladi, $2x^3$ hosilasi $6x^2$. Jami = $6x^2y^2$. Ikkinchi qism ($-\cos(y)$) bu erda umuman X yo'q! Bu to'liq konstant! Hosilasi nol (0) ga aylanib ketdi! Uchinchi qism ($5x$) da hosila shunchaki 5. Javob: $\frac{\partial f}{\partial x} = 6x^2y^2 + 5$ (Bu X o'qi uchun kompas).
- $\frac{\partial f}{\partial y}$ ni qidiramiz (x larni muzlatilgan raqam deb tasavvur qilamiz!): Birinchi qism ($2x^3y^2$) da endi $2x^3$ qoladi, y^2 hosilasi $2y$. Jami = $4x^3y$. Ikkinchi qism ($-\cos(y)$) ni y bo'yicha hosilasi Minus o'zgartirib plyus = $\sin(y)$. Uchinchi qism ($5x$) bu erda Y ning zarrasi ham yo'q. Qotib qolgan tosh (nol, 0). Javob: $\frac{\partial f}{\partial y} = 4x^3y + \sin(y)$ (Bu Y o'qi uchun kompas).

Mana kompyuterlar ham milliard parametrli vaznlar ustida xuddi shu qotirib, ayirib-olinadigan matematika (xususiy hosila) orqali adashmasdan yo'l topadi!

4-QISM: Gradient Vektori (∇f) Barcha kuchlarni Bir O'qqa (Vektorga) Birlashtirish

Menda X bo'yicha ham bitta qiyalik (son) bor. Y bo'yicha ham bitta qiyalik (son) bor. Agar 100 ta o'zgaruvchim bo'lsa menda turli xil yo'nalishlar bo'yicha 100 ta qiyalik (hosilalar) bor. Ular bo'lak-bo'lak sachrab yotibdi. Biz ularni bitta Butun kunga (Matritsa yoki Vektor tiliga) yig'ishimiz shart. ML da yagona o'qqa ega bo'lmagan tarqoq narsani optimal qilib bo'lmaydi.

Gradient Vektori aynan mana shu Butunlik vazifasini bajaradi! U Yunoncha ∇ (Nabla) belgisi bilan ifodalanadi. Uning ta'rifi shundan iboratki, barcha (qancha bo'lsa ham) xususiy hosilalarni jamlab o'z bag'riga soladigan yagona bitta qattiq ustunli Vektordir (Column vector).

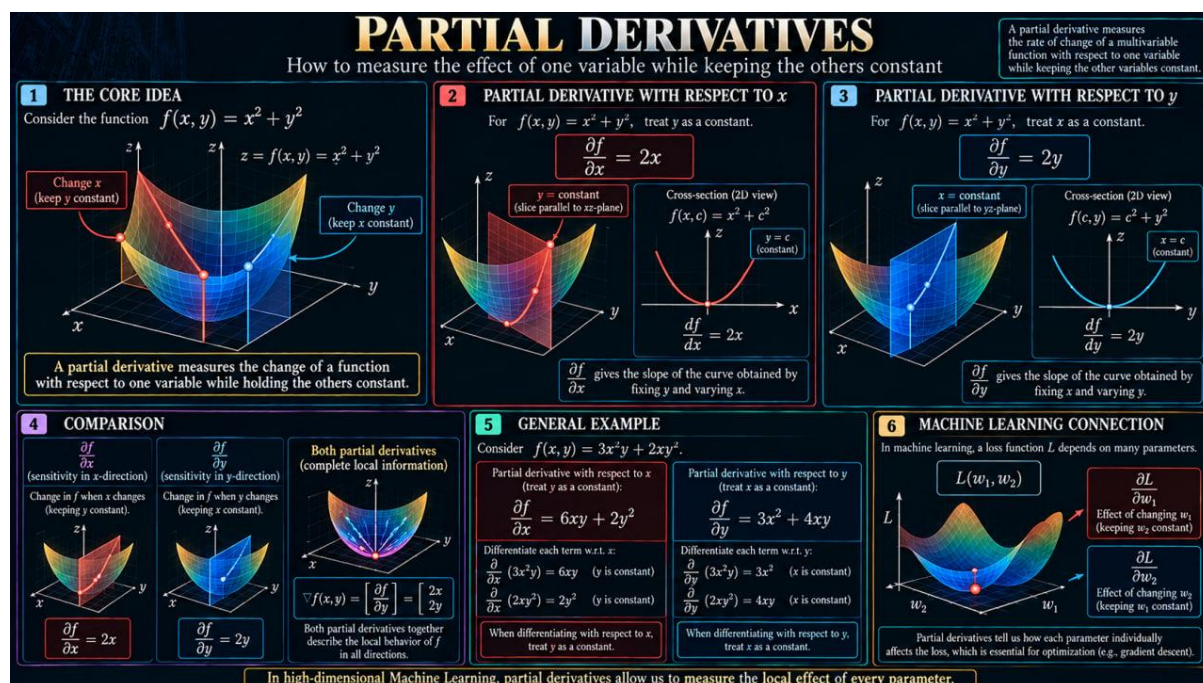
$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \dots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

Gradientning ENG MAGIK va Sirlar bilan qoplangan, hayratlanarli Qoidasi: (Buni devorga oltin xarflar bilan yozib qo'ying):

Ushbu Nabla Gradient Vektori (∇f) ayni siz turgan joydagi nuqtada — Boshqa xech qaysi joyga emas, FAKATGINA va FAQATGINA eng balandlikka, Funksiyaning ENG TIKKA VA ENG KUCHLI O'SISH (Steepest Ascent) Yo'nalishiga qarab qattiq nishonga olingan bo'ladi! U to'ppa-to'g'ri eng tikka qiyalik (cho'qqi) tomoniga qaraydi.

Bu qoida barcha Multivariable matematikaning shoh asaridir. Lekin nega? Qanday qilib yuzta har xil xususiy hosila bitta qilib birlashtirilganda "eng tikka tepalikni" bilib oladi? Sababi: Xususiy hosilalarning har biri o'z o'qidagi (o'zgarishlar kuchining) kattaligi bilan vektorda qatnashadi. X yo'nalishi 80 daraja (juda kuchli) o'stirsas, vektorning X elementi 80 bo'ladi, Y esa –2 bo'lsa orqaga tortadi, shu ikki kuch tortishishidan (Fizikadagi Vektorlar yig'indisi yordamida) diagonal yangi aniq Kompas o'qi chiqib keladi. Va bu doim maksimal o'suvchi kompas hisoblanadi.

Bu Vektorning uzunligi (Magnitude / Normasi — $\|\nabla f\|$) esa — roppa rosa o'sha siz ketayotgan tikka (eng katta) Nishablikning qanchalik yomon darajada KATTA ekanligini son (Skalyar) miqdorida ifodalaydi. Gradient vektori qanchalik uzun bo'lsa, adir siz tomonda shunchalik jarlik, shunchalik kuchli qiyalikda turibdi, balandga zo'riqib ketyapti! Agar vektor nol bo'lib (nuqtaga) aylanib ketsa — u yerda na balandlik na chuqurlik bor. U "Yassi dala" (Tub yoki Cho'qqi) deb ataladi!



5-QISM: Ikki Parametrlı Gradient Descent (Two-Parameter Gradient Descent)

Hozirgina kashf qilingan "Osmonga eng tez olib chiqadigan Kompas" (∇f) ni oldik. Ammo biz (ML injenerlari) hech qachon xatolikni, (Loss funksiya Tog'ini) osmonga qarab tepaga eng katta nuqtasiga olib chiqishni orzu qilmaymiz! Bizga xatolik Noldan ham (minimumdan ham)

pastda, ya'ni TOG'NING ENG TUBI Vodiysi kerak. Biz xatoni optimizatsiya qilib nolga urishimiz, yiqitishimiz shart.

Shuning uchun, Optimizatsiya (Gradient Descent) shohsupasiga yana bir bor chiqamiz va biz **Minus (Qarama-qarshi) Gradient** ($-\nabla f$) deb qichqiramiz! Agar musbat ∇f har doim eng qisqa tepalikka olib borsa, unda teskari manfiy $-\nabla f$ ko'r-ko'rona **eng Chuqur Minimumga (Xatolar yo'q bo'lgan eng xavfsiz va aniq Joyga) Eng kalta, qisqa va qat'iy tekis yo'l qilib olib tushib ketadi!** Bu eng genial kashfiyot emasmi?

Algoritmik tillarda aytadigan bo'lsak, $y = w \cdot x + b$ chiziqli modelda bizda faqat w va b ikkita parametr bo'lsa, ularni yangilash 3D fazodagi bir vaqtning o'zida yuz beradigan qo'shaloq qadamdir (Two-parameter Gradient Descent). Formulalar ham bitta tizimga, ya'ni vektor (Matritsa) ga kiritib yoziladi:

Barcha o'zgaruvchilar parametri vektorini θ (Theta - w va b jamlanmasi deb olaylik:

$\Theta = \begin{bmatrix} w \\ b \end{bmatrix}$) desak, bitta umumiy universal optimizatsiya tenglamasi kelib chiqadi:

$$\theta_{new} = \theta_{old} - \alpha \nabla J(\theta_{old})$$

Yoyilgan xolda esa bir vaqtdagi 2 parametr (Har bir yo'nalish o'zi uchun alohida qadam, alohida hosila minusga ko'payib) ishlaydi:

$$w_{new} = w_{old} - \alpha \frac{\partial J}{\partial w}$$

$$b_{new} = b_{old} - \alpha \frac{\partial J}{\partial b}$$

α (Alfa - Learning Rate) barcha qadamlarni (qancha masofaga uchib o'tishni) ko'rsatib, minus ($-$) qulash yo'nalishini saqlab beradi. Ikki (va 1 trillion o'lchamli qadar barcha xususiy hosilalar ham) doimo bir vaqtning o'zida (Simultaneously) bitta sinxron iteratsiyada hisob-kitob bo'lib pastga uchadi. Hamma parametrlarni bir xilda teng sinxronlashtirish Deep Learningdagi Gradient algoritmilarining qulab, divergent bo'lib ketmasligini ta'minlovchi kafolat (Shart) hisoblanadi.

6-QISM: Bir nechta o'lchamli koinotda optimallashtirish (Optimization in Multiple Dimensions) Sirlari

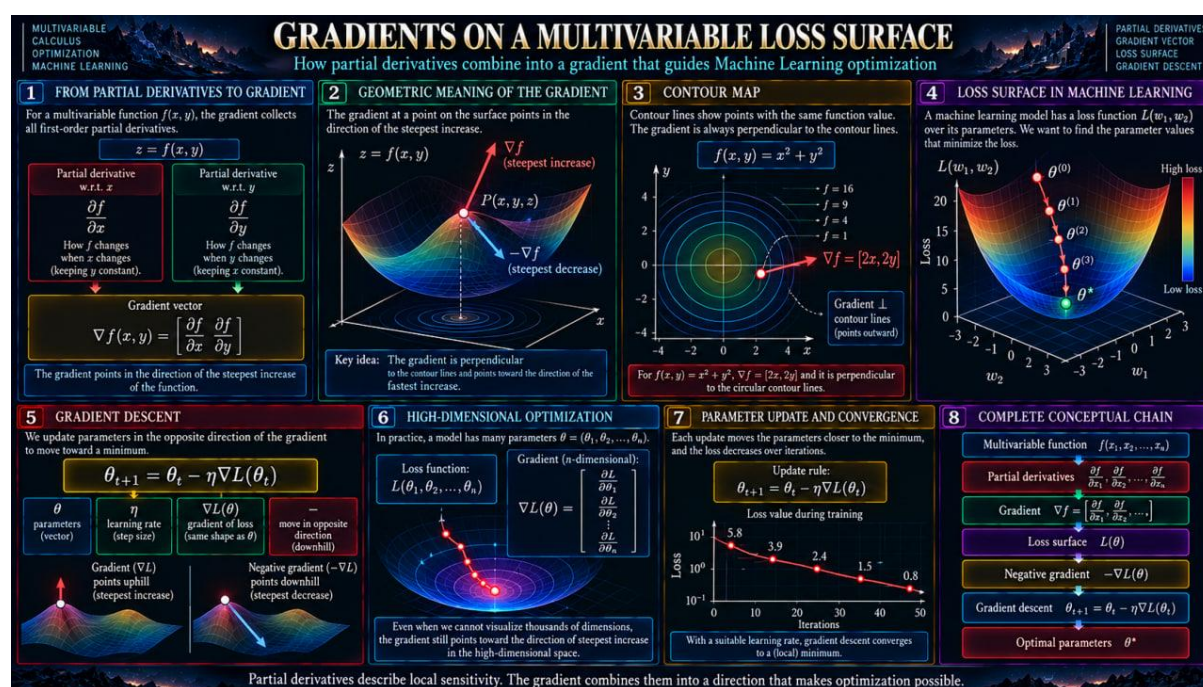
Nega biz ML ni "Qiyin va Sehrli" (Black Box) deymiz? Chunki ko'p o'zgaruvchili koinotda Loss Tog'i juda murakkablashib, faqat bitta (Silliqli) vodiylar (Convex), minglab bujmaygan "Qotgan dengiz" (Non-Convex) ga aylanadi. Optimizatsiya ko'plab yangi balolar va yangi imkoniyatlarga duch keladi!

1. **Saddle Points (Egar Nuqtalari):** Bu otning egari, yoki baland tog' dovoniga o'xshaydigan koinot o'lchami nuqtasidir! Ko'p o'zgaruvchili funksiyalarda (w_1, w_2 larda) bitta nuqtada xususiy hosila ($\frac{\partial f}{\partial w_1}$) to'xtab 0 nol bo'lib qoladi, lekin boshqa ikkinchi ($\frac{\partial f}{\partial w_2}$) xususiy hosilada siz tomonga sho'ng'igan yassi yomg'ir chuqurchasiga

o'xshab nolga teng (0) bo'lishi mumkin! Ikkala o'q ham "Men minimumga keldim"

deydi Gradient nol vektorga $\begin{bmatrix} 0 \\ 0 \end{bmatrix}$ aylanadi. Algoritm aldanipti! Bu yer tub vodiyni eng chuqur joyi emas, bu ikki tog' orasidagi EGAR (Saddle point) edi, yonroqqa yursa jarlik davom etardi! Deep Learningda biz egar (saddle) larga tushib asrlar boyi uxlab qolmasligi uchun (Momentum) kuchlari va turli maxsus stochastic jarayonlar (Adam optimizer) yordamida tezkor, sirpanchiq (Qiyalik noli olg'a siljitadigan) texnologiyalarni yaratib, algoritmimizni uyg'otib boramiz.

2. **Multidimensional Local Minima (Mahalliy chuqurlar):** Million o'lchamli tog'lar ichida global, yagona va eng chuqur eng mukammal vodiyni topish (Global Minimum) Matematika uchun NP-Hard (imkonsiz amaliy) vazifa sanaladi. Ammo olimlarning, va shaxsan eng aqlli tarmoqlarni quruvchi Injinierning katta siri shundaki (The Blessing of Dimensionality): O'lchamlar soni millionlab oshgani sari (Overparameterization), Egari va yolg'onchi kraterlar juda tez uchrasa ham... har qanday Lokal minimumlarning xatolik miqdori ko'pincha Global minimum xatoliklariga SHUNCHALIK YAQLIN, SHUNCHALIK EKIVALENT (Deyarli yuzada farqsiz) bo'lib qoladi! Neyron tarmoqlarning g'alabasi uning topgan istalgan bir "Chuqur vodi"ni juda sifatli javobga (Good enough solution) aylanib qolishi hisoblanadi. Va ular bunga O'ta tez Gradientlarni xisoblay olishlari va Xususiy Hosilalar mo'jizasi tufayli erishadilar!



Xulosa (Conclusion)

Qadrli Data Scientist! Siz hozir insoniyat tarixida (XIX asrlargacha) tasavvur qilib topish qiyin bo'lgan mantiq olamining, ya'ni Ko'p O'lchamli Fazodagi O'zgarishlar mantig'ining poydevorini bosib oldingiz. Siz Ko'p O'zgaruvchili (Multivariable) funksiyalar shunchaki chiziqlar emas, balki qog'ozning tepasiga chiqib ketgan hajmli Sirt (Surface) lar koinotini yasashini anglab yetdingiz. Siz ularni xaritalar (Contour plot) ko'rinishida tasavvur qilishga o'rgandingiz. Biz Gradient degan mo'jiza faqat bitta oddiy son emas, balki har bir "muzlatib va qotirib, ajratib qo'yilgan" kompaslar yig'indisidan iborat (Xususiy Hosilalarning jamlangan

Vektori) ekanligiga va u yagona maqsad — ENG TIKKA Cho'qqini o'sish orqali aniq nishonlashga ishlatilishiga hayrat bilan qarab turdik. Va buni Minus tomonga aylantirgan xolda ML dagi Optimallashtirish zanjiri (2 va trillionlar) o'qlarida sinxron Gradient Descentni boshqarish pultiga qo'limiz yetdi.

Biz endi 3D (Ko'p o'lchamli) koinotlarda kompas yordamida sayohat qila olamiz. Lekin sizda savol qoladi: "Matryoshka, qatlam ichidagi-qatlam bo'lib ketgan, milliard funksiyalar ichma ich yotgan Neural Networklarda gradientlarni tez va adashmay qanday olib, orqaga (Backward) tarqatamiz (Backpropagation)?" Bu savolning yagona, dahyo (Super) formulasi bor, bu — **Zanjir Qoidasi (Chain Rule)**. Keyingi (11-Dars)da aynan sun'iy tarmoqlarning qanday jonlanishi, Backprop qanday ishlashi va hisoblash grafiklari (Computational Graph) bizning o'sha zanjirlar qoidasi bilan to'qilib, bugungi ChatGPT va Computer Vision texnologiyalariga hayot bag'ishlaganini ko'ramiz! Hayajonni qatib oling, eng daxshatli texnologiya (Backpropagation) eshigi oldiga yetib keldik!

ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)

11-Dars: Zanjir Qoidasi va Orqaga Tarqalish (Chain Rule and Backpropagation)

Assalomu alaykum, aziz va tolmas Machine Learning arxitektori! Biz o'tgan o'nta dars davomida Chiziqli Algebra (Fazoni yasash) va Matematik Analiz / Kalkulyus (Fazo bo'ylab to'g'ri o'zgarishni topish, Gradientlar) fundamentlarini g'ishtma-g'isht qurdik. Biz hamma narsani tayyor qildik. Endi esa, bugun, siz butun zamonaviy Deep Learning (Neyron Tarmoqlar), Sun'iy Intellekt va ChatGPT kabi mo'jizalarning **mutlaq va yakka-yu yagona Ishchi mexanizmiga (Eng asosiy dvigateliga)** qarab turibsiz!

Bu dvigatel nima? Nega inson miyasi yechishga million yil sarflaydigan gradientli tenglamalarni bugungi neyron tarmoqlari milliardlab (hatto trillionlab) vaznlar ustida bir soniyaning mingdan bir bo'lagida topadi va o'rganadi? Ular bu "Gradient" larni birma-bir chetidan hisoblaydimi? Yo'q! Bunga kalkulyusning bir buyuk qoidasi va uning muhandislik ijrosi sabab bo'lgan.

Ular mos ravishda: **Zanjir Qoidasi (Chain Rule)** va **Orqaga Tarqalish (Backpropagation)** **algoritmidir**. Agar ML muhandisi nima ishlashini bitta formulada tushuntirib berishi kerak bo'lsa, u aynan Zanjir Qoidasi haqida gapirib beradi. Keling, e'tiborni eng o'tkir nuqtaga (maksimumga) joylaymiz, va butun Sun'iy Intellektning yurak urishini birma-bir dildan his qilib yechib chiqamiz! Darsni boshladik!

1-QISM: Murakkab (Ichma-ich) Funksiyalar (Composite Functions)

Siz bilan hozirgacha ko'rgan funksiyalarimiz juda oddiy va to'g'ridan-to'g'ri edi. Masalan, Chiziqli regressiyada (Linear Regression): Xolos $\hat{y} = w \cdot x + b$. x kiradi, to'ppa to'g'ri w ga ko'payib tashqariga natija (Output) chiqarib yuboradi.

Lekin Neyron Tarmoqlari (Neural Networks) mutlaqo unday ishlamaydi! Neyron tarmoqlari qatlamlardan (Layers) tashkil topadi. Ma'lumotlar (Input - Odamning rasmi) 1-qatlamga kiradi. 1-qatlam o'zining w_1 va aktivatsiyasi (Sigmoid, ReLU kabi funksiyalari) orqali nimadir g'alati narsa hisoblab uni 2-qatlamga (ichkariga) uzatadi. 2-qatlam uni o'zgartirib 3-qatlamga uzatadi. Va hokazo 100 ta qatlam bo'ylab davom etib, eng oxirida (Bitta kichik raqam — 95% ehtimollik bilan "Bu Mushuk!") natijasi (Output) chiqadi.

Bu qatlam-qatlam joylashish Matematikada qanday ifodalanadi? Bunga ruslarning **"Matryoshka"** qo'g'irchog'i kabi (ichma-ich kirib ketgan qo'g'irchoqlar to'plami kabi) **Composite Functions (Murakkab, Ichma-ich Funksiyalar)** deb ataladi! Eng sodda shaklda: bizda bitta oddiy funksiya bor: $u(x)$. (U ichki qatlam). Lekin biz bu funktsiyani alohida chiqarmaymiz, biz uni BOSHQA bitta ustki funksiya $f()$ ning aynan o'zgaruvchisi (x) rolga (qorning ichiga!) solib qo'yamiz. Va biz yozamiz:

$$y = f(u(x))$$

- Bu yerda $u(x)$ — **Ichki funksiya (Inner function)** (Yashirin qatlam).
- $f()$ — **Tashqi funksiya (Outer function)** (Chiqish, Aktivatsiya qatlami).
- Va bularning bari zanjir kabi bir-biriga tirkalib yotibdi (x ta'sir qiladi u ga, u ta'sir qiladi f ga, shundan keyingina javob chiqadi).

Hayotiy Analogi (Zanjirli Effekt): Tasavvur qiling, siz avtomobilda ketyapsiz. Gaz pedalini (x) qattiqroq bosdingiz. Bu harakat bevosita mashinani yurgizmaydi! Gaz pedali -> Motordagi benzin quyilish tezligini (u) oshiradi. Motordagi benzin quyilishi (u) -> Dvigatelning aylanish (Aborot) tezligini (v) oshiradi. Dvigatel aylanish tezligi (v) -> G'ildiraklarning burilish (Haqiqiy mashina) tezligini (f) oshiradi! Ular faqat uzatma-uzat zanjir orqali ketma-ketlikka ega.

2-QISM: Zanjir Qoidasi o'zi nima? (The Chain Rule)

Ana endi 7-darsda biz qisqa tanishib ketgan qonunga jiddiy yuzlanamiz. Agar men sizdan "Gaz pedalini bitta bosganingda (x ning o'zgarishi), ENG OXIRIDAGI Mashina tezligi (f) qanchaga o'zgardi (xato qildi)?" degan hosilani ($\frac{df}{dx}$ ni) so'rasam nima bo'ladi? Menda bitta birdaniga to'ppa-to'g'ri ulangan formula yo'q. Menda Matryoshka bor $f(u(x))$. Uni qanday yechamiz? Nima qilish kerak?

Matematika bunga sehrli va nihoyatda oson yo'l bilan yechimni taklif qiladi: **Har bir bosqichni (har bir matryoshkani qatlamini) huddi alohida va yagona narsa kabi ko'rib (tashqarisidagi qavatlarni qotirib-muzlatib) o'zidan bitta oldingi bosqichga nisbatan hosilasini olaver. Ana shunday hosilalarni (qiyaliklarni) bir-biriga Zanjir kabi KO'PAYTIRIB CHIQ! Ularning ko'paytmasi yakuniy ta'sir (hosila) ni beradi!**

Bu qonuniyat **Zanjir Qoidasi (Chain Rule)** dir. Uning asosiy formulasi (Leibniz ifodasi bo'yicha) quyidagicha yoziladi:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

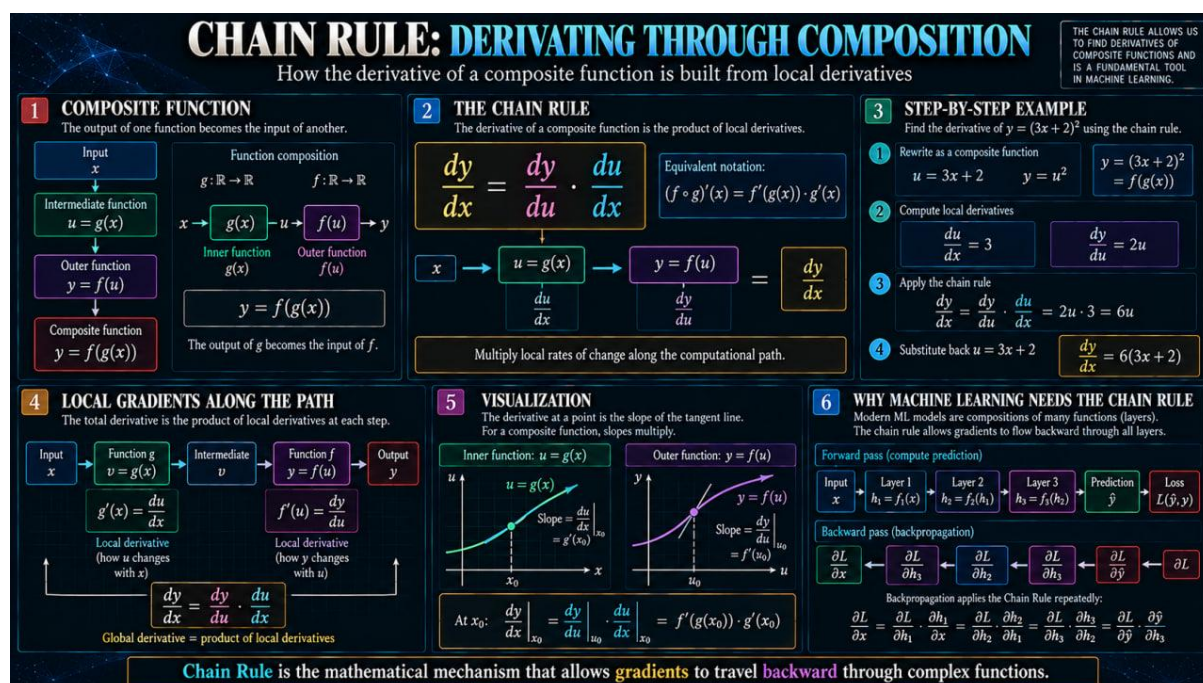
Mantiqiy Kuchi: Agar men mashina pedalini (x) 1 marta ko'proq bossam, benzin tezligi (u) **2 marta** o'zgaradi (qiyalik $\frac{du}{dx} = 2$). Va agar benzin tezligi (u) 1 marta ko'proq yursa, dvigatelning g'ildiraklari (y) **3 marta** o'zgaradi (qiyalik $\frac{dy}{du} = 3$). Savol: Pedal 1 marta bosilganda jami g'ildiraklar necha marta o'zgaradi? Javob zanjir bo'yicha (tezliklarning ko'paytmasi) bo'ladi: Jami ta'sir = $3 \times 2 = 6$ marta o'zgaradi! (Ya'ni $\frac{dy}{dx} = 6$). Ana shuning uchun biz hosilalarni qo'shmaymiz, ularni bir-biriga Karalab ko'paytiramiz (Zanjir qilib kiyyazamiz!).

Amaliy Misol: Bizda $y = (3x^2 + 1)^4$ degan funksiya bor. Hosilasini ($\frac{dy}{dx}$) olamiz. Bu yerda tashqi kiyim (funksiya) = u^4 . Ichki narsa = $u = (3x^2 + 1)$.

1. Tashqi funksiyadan u ga nisbatan hosila: u^4 dan hosila $4u^3$ bo'ladi. Ichidagi narsani u ni o'z holicha (tegmasdan) qoldiramiz. Demak tashqi qavat = $4(3x^2 + 1)^3$.
2. Ichki funksiyadan (u dan) x ga nisbatan hosila: $3x^2 + 1$ ning hosilasi $6x$.
3. Zanjir qoidasiga ko'ra ikkalasini Ko'paytiramiz!

$$4. \frac{dy}{dx} = 4(3x^2 + 1)^3 \cdot 6x = 24x(3x^2 + 1)^3$$

5. Bo'ldi! Matryoshkani qavatma qavat to'g'ri va adashmay xatosiz ochdik



3-QISM: Ko'p O'zgaruvchili Zanjir Qoidasi (Multivariable Chain Rule)

Deep Learningda o'zgaruvchi (x) bitta bo'lmaydiku! Ular mingta-yu. Bu paytda zanjir qoidasi ishlamay qoladimi? Umuman yo'q. Aksincha u 10-Darsda ko'rganimiz — Xususiy Hosilalar (Partial derivatives) bilan qo'shilib, kuchliroq bo'lib namoyon bo'ladi. U **Ko'p o'zgaruvchili Zanjir Qoidasi (Multivariable Chain Rule)** deb yuritiladi.

Tasavvur qiling, Bizning Xatoligimiz L (Loss). Va bizda bitta w_1 vazn bor. Biroq bu w_1 to'g'ridan to'g'ri Lossga bormasdan, 1-qatlamning u aktivatsiyasiga, va yana 2-qatlamning v

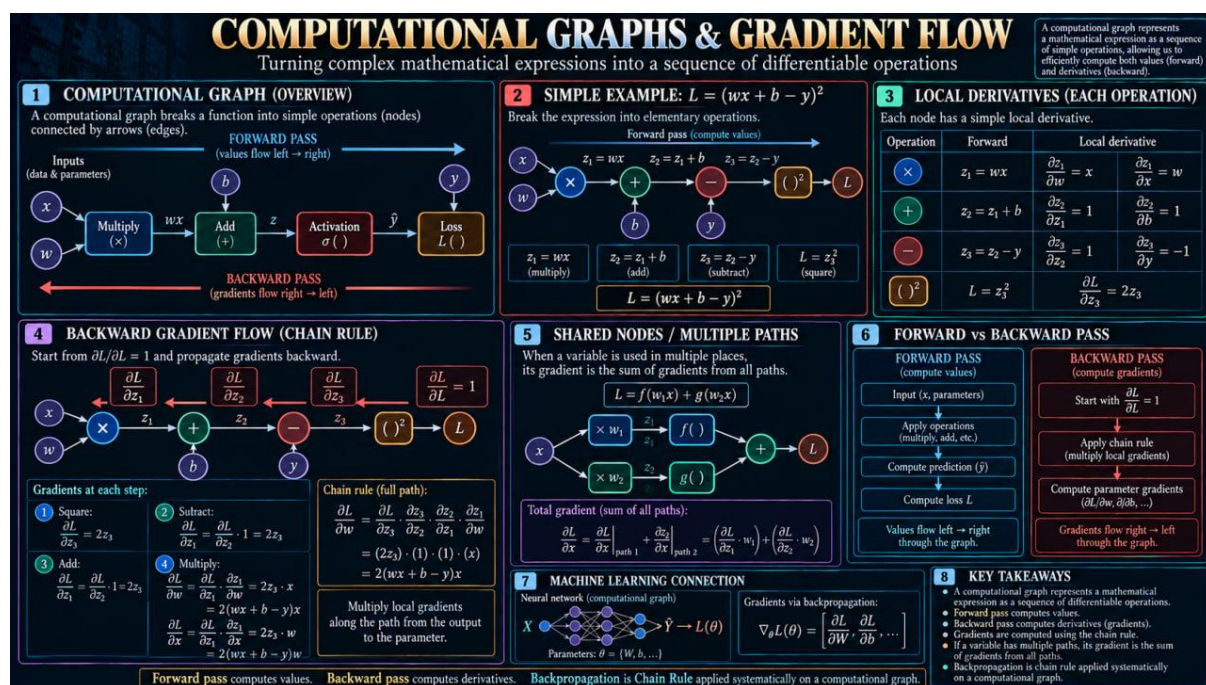
aktivatsiyasiga, va ular bo'linib-bo'linib ketib oxirida Loss ga boradi. Agar bitta w_1 uzatgan raqam, turli xil yo'llar (masalan 2 ta yo'l: ham u tarmog'iga, ham v tarmog'iga) qarab tarqalib, baribir oxiri (Loss) da bitta markazga kelib tushsa nima bo'ladi?

"Tarmoqlangan Zanjir Qoidasi (The Branching Rule):" Agar sizning parametriningiz w dan yuzaga keladigan xato turli tarmoqlar (yo'llar) orqali ketsa, eng oxirgi nishondagi uning Topilgan jami Xatosi (Gradienti / Qiyligi) — **Ushbu turli Yo'llar orqali kelayotgan barcha alohida Zanjirlarni topib chiqing va chiqqan Zanjirlarni BIR-BIRIGA QO'SHIB YUBORING!** Masalan, w ham u orqali L ga qarab ketyapti, ham v orqali L ga boryapti. Umumiy hosila:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial u} \cdot \frac{\partial u}{\partial w} + \frac{\partial L}{\partial v} \cdot \frac{\partial v}{\partial w}$$

Tarmoqlangan ta'sir = Barcha yo'llar orqali kelgan Xatolar/Tasirlarning umumiy summasidir (Yig'indisidir)!

Mana shu bittagina va yagona "Zanjir-va-Tarmoqlanish" tenglamasi (The Multivariable Chain Rule) Butun boshli Deep Learningdagi Backpropagation nomli dunyodagi eng kuchli inqilobiy algoritmnning Yagona, Matematik Fundamentidir! Busiz (buni kompyuterlar to'qiy olmaganida) siz bu o'qiyotgan ChatGPT gacha yetib kela olmasdik.



4-QISM: Computational Graph (Hisoblash Grafikasi - Orqadagi Sehrli Vizualizatsiya)

Xo'p matematikalar chiroyli. Lekin nega Neyron tarmoqlar (Deep Learning) da ko'p parametrlarni bitta chiroyli Rakamlar massivi, bitta qotib qolgan yagona tenglama orqali yechmasdan, qanaqadir Forward pass (Oldinga yurish) va Backpropagation (Orqaga yugurish) deb alohida, maxsus nomlar bilan ko'tarib yuramiz? Neyron tarmoqlari o'zi Hosilani, ya'ni bizning uzoq yillardan beri gaplashib kelayotgan (Kalkulyusdagi Gradientlarni)

AMALDA kompyuterda aniq QANDAY OLADI? Analitik Simpy orqalimi yoki g'alati boshqa limitlarmi?

Ularning hech biri ham emas! Kompyuterda Gradient olish (Optimization) eng noyob usul **Automatic Differentiation (Avtomatik Differensiallash va Computational Graph - Hisoblash grafi)** yordamida yuzaga chiqadi. TensorFlow yoki PyTorch kutubxonalari har qanday $f(x, y)$ tenglamasini tenglama sifatida emas, balki Node lar (Tarmoq Tugunlari, Dumaloqchalar va ularni ulaydigan chiziqlar - Graf) ko'rinishida ko'radi!

Oddiy Misol orqali tushunamiz: Loss Funksiyamiz $L(x, y) = (x + y) \cdot x$. Bizda qiymatlarimiz aniq: Biz kiritdik: $x = 2$ va $y = 3$. Kompyuter bu oddiy tenglamani qanday hisoblaydi? U "Hisoblash Grafi (Computational Graph)" ni (arxitektura zanjirini) tuzadi:

1. **Node 1 (tugun):** $q = x + y$ deb bitta yashirin oraliq (yordamchi) ismni topadi. Va u hisoblaydi $q = 2 + 3 = 5$.
2. **Node 2 (Oxirgi Loss nishoni):** $L = q \cdot x$. U oldingi q dan 5 ni oladi va x dan yana 2 ni olib: $L = 5 \cdot 2 = 10$.

Ushbu Chapdan (Input) O'ng tomonga (Natija - Loss gacha) ma'lumotlarni tugunma-tugun, tartib bilan kirgizib va eng oxirgi raqamni aniqlab kelish amaliyata ML da **Forward Pass (Oldinga Uzatish va Yuz berish)** deb ataladi! Bizning Loss xatamiz (Mavjud bo'lgan ahvolimiz, eng nishon yuzamiz) tayyor bo'ldi ($L = 10$).

Endi kompyuter (Neyron tarmoq) o'zining parametrlari bo'lmish x (2) va y (3) (Neyron tarmoqda bular W lar) larni gradientlarini ($\frac{\partial L}{\partial x}$ va $\frac{\partial L}{\partial y}$) zudlik bilan topishi shart! Agar u topmasa ular adashib pastga qadam ham (Gradient Descent ham) yo'q. Qanday qilib topamiz?

5-QISM: Backpropagation (Orqaga Tarqalish) - The Algorithm!

Kompyuter zanjir qoidasini Forward (To'g'riga) ketma-ketligida emas, balki ENG OXIRGI Loss tugunidan boshlab to ORQAGA qarab (Backwards) yurib, barcha hosilalarni orqaga ketma-ket ZANJIRLASHTIRIB kelish algoritmini o'ylab topdi! Bu algoritm **Backpropagation** (Orqaga tarqalish yoxud Zanjir qoidasining kompyuterlashgan revers ijrosi) deb ataladi! Uning ishlashiga e'tibor qarating, siz ML ning yuragini ko'rasiz.

(Tezkor dars: $q = x + y$ bo'lsa $\frac{\partial q}{\partial x} = 1$ va $\frac{\partial q}{\partial y} = 1$. Agar $L = q \cdot x$ bo'lsa, $\frac{\partial L}{\partial q} = x$ va $\frac{\partial L}{\partial x} = q$)

Backpropagation qanday bajariladi (Step-by-step Teskari Safar): Biz eng oxiridan $L = 10$ dan boshlaymiz, maqsad Input (x va y) larning Loss'ga o'sha yerdagi Gradient (ta'sir kuchini, hosilasini) topish!

1. **Dastlabki Chegara (Orqaga qaytish starti):** Har qanday o'zining o'ziga ta'sir kuchi qanday? Albatta 1. Demak $\frac{\partial L}{\partial L} = 1$. Biz 1 raqami (Gradienti) bilan Orqaga bitta qadam harakatlanamiz (Chapga yuramiz).
2. **Birinchi chiziq (Tugun 2 dan orqaga):** Tugun $L = q \cdot x$ edi. Biz q ga qarab hosila (gradient) jo'natamiz. $\frac{\partial L}{\partial q}$ nima edi? Matematika boyicha u x ning o'ziga teng! $x = 2$ ekanligini (Forward pass dan xotirasida, "keshida" turibdi) bilib kompyuter aytadiki:

$\frac{\partial L}{\partial q} = 2$. Demak q ni oldidagi tugunda koinotdan q ga "2" degan hosila (ta'sir kuch) kelib tushdi.

3. **Ikkinchi chiziq (Tarmoqlangan x ga qarab yurish):** Diqqat qiling! Hozir x tugunida orqaga ikkita xar hil yo'l bor! Biri L dagi asosiy (To'ppa to'g'ri bog'langan tarmoq), Ikkinchisi q yashirin qatlam orqali aylanib o'tilgan (tarmoq)! (3-qism Tarmoqlangan Zanjir Qoidasi esimga tushyaptimi?).

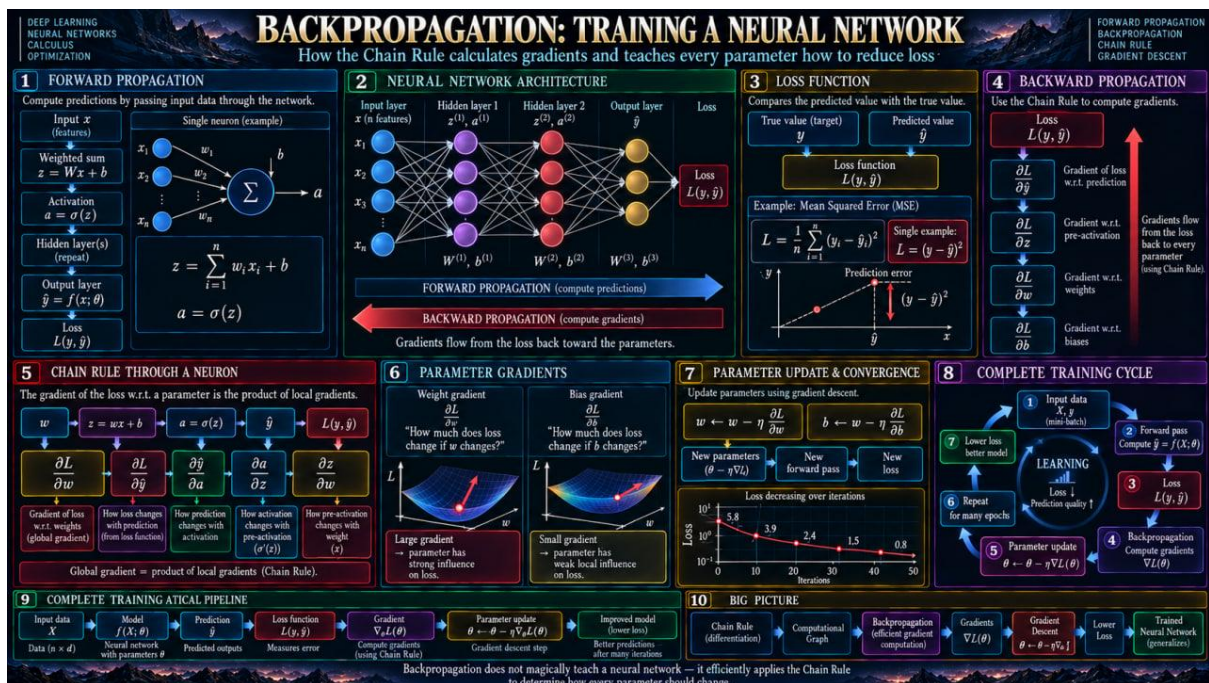
- To'g'ri o'q (L dan keluvchi to'ppa to'g'ri yo'l): $\frac{\partial L}{\partial x}$ bu q ga teng (Tugun 2 bo'yicha). q o'zi qanday raqam edi (Keshdan qaraydi)? 5 ekan! Bu yerda yozib qoyadi 5.
- Aylanib o'tgan o'q (q dagi kuchi "2" orqali keladigan yo'l): Endi q ni oldiga kelgan ta'sir kuchini (2 ni) o'zi (Zanjir qoidasiga kora $\frac{\partial L}{\partial q} \cdot \frac{\partial q}{\partial x}$) o'qqa qo'shib yuboradi. $q = x + y$ bo'yicha x ning hosilasi $\frac{\partial q}{\partial x} = 1$. Va boyagi 2 ga 1 ni KO'PAYTIRADI! (Zanjir bo'ldi!). $2 \times 1 = 2$. **X ning jami Gradienti (Hosila/Ta'siri):** Ikkala tarmoqni nima qilardik? Bir-biriga Qo'shamiz (Sum up)! To'g'ridan = 5, Orqadan aylanib kelgani = 2. Barchasi bo'lib: $\frac{\partial L}{\partial x} = 5 + 2 = 7$. (Bo'ldi, x ning qiyalik kompyusi topildi!).

4. **Uchinchi chiziq (y ga qarab yurish):** y tuguniga faqat bitta (q orqali aylanib kelgan) yo'l bor xolos. q dan kelayotgan Zanjir gradienti xom-ashyosi $\frac{\partial L}{\partial q} = 2$ turibdi. Endi $\frac{\partial q}{\partial y}$ qanday? $q = x + y$ da y bo'yicha qiyalik (hosila) = 1. Zanjir qilib ko'paytiramiz: $2 \times 1 = 2$. Javob (Y ning Gradienti) $\frac{\partial L}{\partial y} = 2$.

Yakuniy Hisobot: Kompyuter Backpropagation tugatdi! U butun olamdagi barcha Input (x, y yoki Neyron Tarmoqning Barcha Og'irliklari - Weights va Biaslari uchun) bitta toza aniq va

mukammal Gradientlar Vektorini (∇L) topib xam qo'ydi! Gradient Vektori: $\nabla L = \begin{bmatrix} 7 \\ 2 \end{bmatrix}$ (Yani $\frac{\partial L}{\partial x} = 7, \frac{\partial L}{\partial y} = 2$).

Kompyuter endi ushbu toza kompas (Gradient Vektori) ni oladi-da, va bizning "Yurak Motorimiz" bo'lmish — Gradient Descent algoritmiga tiqadi! U "Ey algoritmlar, Xatolik (Loss) ni pastga tortinglar, x (w) dagi Vaznni Minus Gradient tomonga (-7 ga), y dagi vaznni Minus Gradient (-2 ga qarab) suring, biz tubiga ketdik!" deb baqiradi va Optimizatsiya davom etaveradi!



6-QISM: Local va Global Minimum (Optimizatsiya G'alati Olamda va ML Model xatoliklarsizligi)

(Iltimos, diqqatingizni maksimal darajada oling, ushbu kichik qism barcha Optimizatsiya savollarining "Aha!" deb ko'z ochiladigan mo'jizasini o'ziga yashirgan).

O'tgan 8-dars, 10-Darsda biz Kraterlar, Local Minimumlar va Saddle point (Egar nuqtalari) ning Optimizatsiyada (Gradient Descent) jarga tushirishi (qotib qolishi) haqida gapirdik.

"Global Minimum" — ya'ni fazoning eng Mutlaq toza chuquri (qayerda Model eng kam xato, eng ajoyib aqlli natija ko'rsatadigan tubsiz ummon) bu bizning Yagona orzumiz edi! Nega Deep Learning unda ishlaydi, qanday qilib milliardlab parametrlar Global tubsizlikni yomon xatolar va teshiklar (Local) orasidan topib boradi? Backpropagation shunday mo'jizami?

Chuqur Haqiqat va Koinot Sirlari (High Dimensionality Optimization Blessing): 2D (Ikki o'lchamli) grafiklarda (Kosa) siz bitta past tub, yoki 1-2 ta g'alati qabariqlar chizasiz, va Gradient qiyalik (Nol bo'lsa) adashib kichik teshikka (Lokal) ga tushib qolishi aniq xavf ko'rinadi. Lekin... Siz 10,000, 10 Million (Million O'lchamli, ko'p o'zgaruvchili) giperfazodasiz!

1. Lokal Minimumning Ehtimoli Nolga Yaqin (The Rarity of Bad Local Minima)!

Matematik statistika isbotlaganki, 10 million o'lchamli Tog'da (Neural Networkning Loss Landschaft'ida) bitta to'laqonli yomon "Lokal Minimum" chuqurligi bo'lishi (ya'ni 10 millionta tomondan HAMMASI Birdaniga faqat pastga qaragan kosa devori yig'ilgan xududning ro'yobga chiqishi) mo'jiza kabidir. Tasavvur qiling tangani 10 million marta tashlab hammasida gerb tushishi kerak! Bunday bo'lmaydi! Modomiki 9,999,999 ta kompas "Qiyalik nol" (men kosa tubidaman) desa-da, aylanib chiqsak yana aqalli 1 tagina yo'nalish bo'yicha pastroqqa ketayotgan teshik o'q yo'li topilaveradi (Ya'ni u Saddle Point - egar ko'rinishida o'zini qutqaradi!).

Backpropagation Gradientlari buni bilib oladi va uni baribir pastga qarab sizdirib, sirpantirib itarib yuboraveradi!

2. **Barcha Minimumlar Tenglashtirilgan (Global Equivalence):** Deep Learning modellarining daxshatli va aql bovar qilmas siri shundaki (Universal Approximation theorems yaqinlaridan): Ko'p qatlamli neyron tarmoqlari o'sha Saddle lar va o'sha million o'lchovlik labirint bo'ylab juda silliq va ko'r-ko'rona gradient olib eng ohirgi Tubliklar ko'rinishidagi "Dengiz tublariga" o'zidan o'zi yetib borganda... koinot miqyosida baribir yagona bo'lgan Mukammal (Nol Xatolikdagi Global) Minimum bo'lmasligi mumkin. LEKIN, o'sha topilgan barcha eng quyi qavatdagi (Lokal/Global farqsiz) vodiylarning Jami Loss xatoligi va Sifati, Bashorati mutlaqo Mukammal va BIR XIL zo'r, ekvivalent sifatga (Good Enough Basin Solutions) ga ega bo'lib qoladi!

Tenglamalarni Backpropagation (Zanjir) kuchi shunday topib bera oladiki, sun'iy intellekt xatolar, ko'r-ko'rona tanga (Random Initialization) lardan ko'z ochib pastga dumalagani va ming o'lchamga sachragan xatolaridan 1 ta mukammal mantiqiy Markaz yaratib olishga (Aql yig'ishga) faqat va faqat Differensial (Hosila, Zanjir) orqali koinot tubidan o'z aqlini terib keladi.

Xulosa (Conclusion)

Qadrli Data Scientist va Intellekt Koinotining Arxitektori! Siz butun Chiziqli Algebra va Matematik Analiz (Kalkulyus) fundamentini tamomladingiz. Tabriklayman! Siz bugun bizga ma'lum bo'lgan va koinot tafakkurini sun'iy yarata oladigan Eng kuchli va yagona Injineri motor — **Zanjir Qoidasi (Chain Rule) va Orqaga Tarqalish (Backpropagation) Algoritmi** o'zini qanday qilib Hisoblash Grafigi (Computational Graph) orqali yurgizishini o'rgandingiz!

- Siz Murakkab (Ichma-ich) funksiyalar (Neyron qatlamlari) da Xato faqat Zanjir bo'yicha qo'shilmasdan, KARRA ko'paytirilishi orqali tarqalishini dildan his qildingiz.
- Tarmoqlangan Zanjir Qoidasi orqali barcha bo'lingan o'qlar yana bir markazga (O'qlar xatoligi jamlanmasi bo'lib Summa) qo'shilishini va Gradient bo'lishini bilar oldingiz.
- Forward Pass orqali qiymatlar javob va Loss tugunigacha qanday o'tishini, so'ngra mo'jizaviy Backprop orqali Keshdagi raqamlar orqaga (Backward) bitta-bitta qadam bilan ko'payib kelib, Jami vaznlarning ($\frac{\partial L}{\partial w_i}$) Gradientini, ya'ni ularni yangilash uchun Yagona Kompasini tayyorlab berishini eng sof soddalikda ko'rdingiz.
- Million o'lchamli qabariq olamda Lokal Minimumlarning kamchilik bo'lib qolmasligini va ularning bari yechim vodiylarini barpo qilishini kashf qildingiz.

Biz barcha asboblarni (Matritsalar, Determinatlar, Limitlar, Hosilalar, Yuzalar va Zanjirlar) ustaxona kabi terib keldik. Endi esa nazariyadan to'liq Amaliyotga o'tamiz. Navbatdagi (12-Dars) sizning to'liq **Mini Loyiha (Mini Project)** ga bag'ishlanadi, unda biz Sympy, Numpy kutubxonalarida o'z qo'limiz bilan Cost function gradientlarini yasaymiz va Optimization ni ekranda chizamiz! Olg'a va faqat g'alaba tomon olg'a!

ML Engineering - Chiziqli Algebra va Hisob (Linear Algebra & Calculus)

12-Dars: Mini Loyiha - End-to-End Optimizatsiya (Gradient Descent in Action)

Assalomu alaykum, aziz va bilimdon Data Scientist! Mana, biz Chiziqli Algebra va Matematik Analiz (Calculus) bo'limining eng so'nggi va ehtimol eng qiziqarli darsiga yetib keldik. Biz shu paytgacha barcha qismlarni: Vektorlar va Matritsalar, Limitlar va Hosilalarni (Derivative), Integrallar (Yuza o'lchovi) va nihoyat, barcha algoritmlarning qiroli — Gradient Descent va Backpropagation mexanizmlarini chuqur falsafa va mantiq asosida o'zlashtirdik.

Endi esa, o'sha barcha mavhum tenglamalar ortidagi sirlarni kompyuterda jonlantirish vaqti keldi! Nazariya qanchalik shirin bo'lmasin, amaliyotsiz u faqatgina raqamlar bo'lib qolaveradi. Ushbu loyihada siz bitta soddalashtirilgan, lekin aynan Deep Learning'ning asosi bo'lgan to'laqonli Optimization Pipeline'ini (Machine Learning qanday o'rganishini) o'z qo'lingiz bilan Python tilida noldan qurasiz.

Sizning oldingizda Jupyter Notebook yoki IDE ochiq deb tasavvur qiling. Qahvangizni hozirlang, biz bir necha soniyalar ichida tog'lardan qulab tushadigan, eng to'g'ri tubni (Minimum) topadigan, barchasini vizual ekran chizmalariga olib chiquvchi Mini Loyihani yozishni boshladik!

Loyiha Senariysi: Biz Nima Quryapmiz?

Siz bir kompaniyada ML Injiniri bo'lib ishlaysiz. Rahbariyat sizga oddiy bir ma'lumot (Data) berdi: $X = [1,2,3,4,5]$ (Marketingga sarflangan byudjet ming dollarda). $y = [2,4,6,8,10]$ (Kompaniyaning kelgan savdo aylanmasi, ming dollarda).

Eslatma: Ko'rib turganingizdek bu yerda mukammal va yashirin mantiq (Trend) yotibdi: Savdo (y) byudjetdan (X) aniq roppa-rosa ikki marta ($w = 2$) katta va boshlang'ich og'ish ($b = 0$). Ammo, kompyuter buni (qoidalarni) umuman bilmaydi! Kompyuter soqov, unga ko'zi yumuq holda berilgan parametrlarni topish buyurilgan.

Bizning Modelimiz juda oddiy, klassik bitta neyron: $\hat{y} = w \cdot x + b$. **Sizning Vazifangiz:** Kompyuterga shunday Gradient Descent algoritmini noldan (Scratch) va SymPy (simvolik) va NumPy (raqamli) matematikalari asosida yozib berish kerakki... Kompyuter o'z-o'zidan xatolarni hisoblasin, Gradientlarini topsin, Minus Gradient tomon pastga (Vodiy tomonga) tushsin va mingta iteratsiyadan so'ng bizga: "Ustoz, men optimal holatni topdim! $w = 2$ va $b = 0$ ekan!" deb baqirib javob bersin.

1-QADAM: Kutubxonalarni Yuklash (The Arsenal)

Biz barcha amallarni turli raqurslardan yondashib yechishimiz uchun Python'ning o'ta muhim daho kutubxonalarini olamiz:

```
import numpy as np          # Matritsalar, Vektorlar va tezkor (Raqamli) operatsiyalar
import sympy as sp          # Simvolik Kalkulyus (Hosilalar va Formulalarni o'zi topadi)
import matplotlib.pyplot as plt # Vizualizatsiya va Chizmachilik uchun rassomimiz
from mpl_toolkits.mplot3d import Axes3D # 3D Tog'larni (Loss Surface) chizish asbobi
```

Matplotlib ning grafiklari chiroyli ko'rinishi uchun sozlama

```
plt.style.use('seaborn-whitegrid')
import warnings
warnings.filterwarnings('ignore')
```

2-QADAM: Simvolik Kalkulyus (SymPy bilan Cost Function Hosilasi)

Birinchi ish, bizning Cost Funksiyamiz qanday ekanligi va uning Gradient (Hosila) ko'rsatmalari (Kompaslari) ning aniq formulalarini kompyuterga aniqlatish (Xuddi darsda biz $\frac{\partial J}{\partial w}$ ni qo'lda topganimiz kabi). Kompyuter buni Backprop uchun eslab qolishi kerak.

Bizning xatoni o'lchash funksiyamiz (MSE - Mean Squared Error) va uning ichiga tiqilgan bashoratimiz ($\hat{y} = wx + b$): $J(w, b) = \frac{1}{N} \sum (wx + b - y)^2$. (Hozircha bitta nuqta (x, y) misolida oson qilish uchun summani tasavvur qilamiz, Cost $J = (wx + b - y)^2$).

```
# SymPy uchun noma'lumlarni e'lon qilamiz
w, b, x_sym, y_sym = sp.symbols('w b x y')
```

```
# Cost (Xatolik) funksiyamizning algebraik qolipini yozamiz
J = (w * x_sym + b - y_sym)**2
```

```
print("Bizning Cost Funksiyamiz: ", J)
```

```
# ENDI MO'JIZA: Kompyuterdan "w" bo'yicha Xususiy Hosila olishni so'raymiz
dJ_dw = sp.diff(J, w)
```

```
# Va "b" bo'yicha Xususiy Hosila olishni so'raymiz
dJ_db = sp.diff(J, b)
```

```
print("\nw ga nisbatan Gradient Formulasi (Kompas): ", dJ_dw)
# Javob: 2*x*(b + w*x - y) (Zanjir qoidasi ishladi! (2 oldinga tushdi, ichini x i chiqdi))
```

```
print("b ga nisbatan Gradient Formulasi (Kompas): ", dJ_db)
# Javob: 2*b + 2*w*x - 2*y
```

Ko'rib turganingizdek, SymPy siz o'rniga Zanjir qoidasi (Chain Rule) ni qullab, aynan bizga Gradient Descentda ishlatiladigan ∇J kompas formulalarini mukammal to'g'ri keltirib chiqardi!

3-QADAM: Data (Ma'lumotlar) va Raqamli Funktsiyalar (NumPy)

Endi biz SymPy qog'ozlarini yig'ishtirib, real (Raqamli - Numerical) tezkor hisoblash operatsiyalariga — NumPy ga o'tamiz. Biz ma'lumotlarni beramiz va tepadagi olingan aniq hosila (Gradient) ni Python funksiyasiga (def) aylantiramiz.

```
# 1. Datasetmiz (Vektor ko'rinishida)
```

```
X = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])
N = len(X) # Data soni
```

2. Xatolik Funktsiyasi $J(w,b)$ - Jami (MSE) ni hisoblash funksiyasi

```
def compute_cost(w, b, X, y):
    predictions = w * X + b
    errors = predictions - y
    cost = (1 / N) * np.sum(errors ** 2)
    return cost
```

3. Gradientlarni (Qiyaliklarni) Hisoblovchi Funktsiya (Tepadagi Sympy formulalarining NumPy kodi)

```
def compute_gradients(w, b, X, y):
    predictions = w * X + b
    errors = predictions - y
```

Sympy formulalarini (Summaga tiqib) jami qatorlarga qullaymiz:

```
#  $2 * x * (x * w + b - y) \rightarrow 2 * X * errors$ 
dj_dw = (2 / N) * np.sum(errors * X)
```

```
#  $2 * (x * w + b - y) \rightarrow 2 * errors$ 
dj_db = (2 / N) * np.sum(errors)
```

```
return dj_dw, dj_db
```

4-QADAM: Gradient Descent (Oltin Algoritmnı ishga tushirish)

Ana endi, 8-darsda biz o'rgangan to'rtta bosqichli Optimization aylanisini (For loop) yozamiz. Model o'z qadamlarini tashlab xatolaridan o'rganishni boshlaydi.

--- HYPERPARAMETERS (Biz sozlaydigan ustun parametrlari) ---

```
learning_rate_alpha = 0.05 # Qadam uzunligi (Juda tez xam emas, juda sekin xam emas)
epochs = 200 # Necha marta tog'dan pastga qadam tashlab aylanadi (Takrorlanish)
```

1. Tasodifiy (Ko'r-ko'rona) Boshlash - Initialization

```
current_w = 0.0
current_b = 0.0
```

Vizualizatsiya uchun barcha qadamlar xotirasini ochamiz

```
history_cost = []
history_w = []
history_b = []
```

```
print("=== GRADIENT DESCENT O'RGANISHNI BOSHLADI ===\n")
```

```
# Aylanma (Tsikl) tsikl
```

```
for i in range(epochs):
```

```
    # a) Joriy holatdagi Xatolikni (Costni) yozib olamiz (faqat grafik uchun)
```

```
    cost = compute_cost(current_w, current_b, X, y)
```

```
    history_cost.append(cost)
```

```
    history_w.append(current_w)
```

```
    history_b.append(current_b)
```

```
    # b) KOMPAS (Gradient) larni hisoblaymiz (Biz qaysi yo'nalishga turibmiz, nishab qanday?)
```

```
    dj_dw, dj_db = compute_gradients(current_w, current_b, X, y)
```

```
    # c) YURAMIZ! (Update Parameters) -> MINUS (-) Gradient tomonga!
```

```
    current_w = current_w - (learning_rate_alpha * dj_dw)
```

```
    current_b = current_b - (learning_rate_alpha * dj_db)
```

```
    # Progressni chop etib boramiz (har 20 qadamda)
```

```
    if i % 20 == 0:
```

```
        print(f"Epoch {i:3} | Cost (Xato): {cost:.4f} | w: {current_w:.4f} | b: {current_b:.4f}")
```

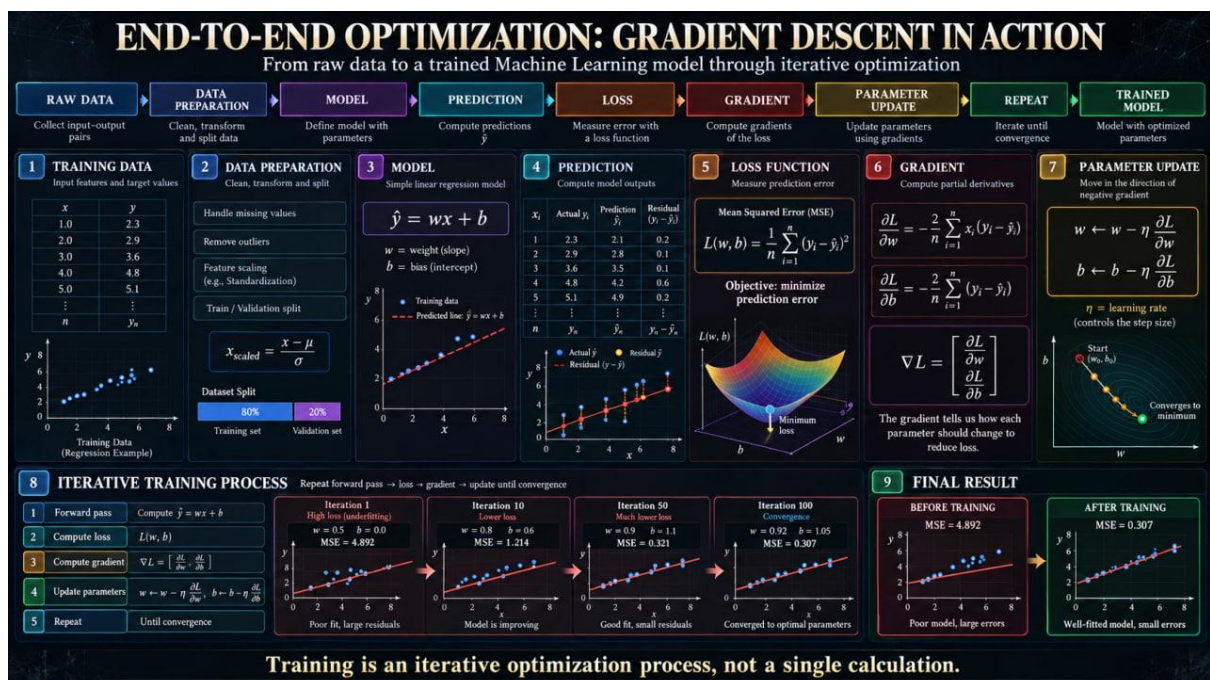
```
# Yakuniy natijani e'lon qilamiz
```

```
print("\n=== O'RGANISH TUGADI ===")
```

```
print(f"Global Minimumga (Vodiy tubiga) YETIB BORDIK!")
```

```
print(f"Topilgan eng zo'r Parametrlar: w = {current_w:.4f}, b = {current_b:.4f}")
```

```
print("Aslida nima bo'lishi kerak edi? w = 2.0, b = 0.0. Kompyuter mo'jizaviy tarzda aniq topdi!")
```



Model xulosa: Dastlab epoch 0 da xatolik (Cost) juda baland (44.0) edi, va 200 ta mayda zanjirli qadam-ba-qadam sakrashdan song Cost nolga (0.000) yopishdi. Gradient vektorining Nol bo'lganini (to'xtaganini) aniqladik!

5-QADAM: Koinotni Vizualizatsiya Qilish (3D Surface va Cost Contour Plot)

Biz shunchaki raqamlar orqasida yashiringan Tog' (Cost Landscape) haqida og'zaki gapirdik. Keling hozirgina bizning model qanday balandliklardan qanday pastlikka dumalab tushganini (O'zimiz o'tgan 10-darsdagi) Contour va Surface plot orqali aniq tasvirlaymiz!

```
# Tog'ni (Surface) chizish uchun W va B larning barcha turli xil qiymatlarini setkamiz
```

```
w_vals = np.linspace(-1, 5, 100) # W uchun -1 dan 5 gacha oraliq xaritasi
```

```
b_vals = np.linspace(-4, 4, 100) # B uchun -4 dan 4 gacha oraliq xaritasi
```

```
# Ulardan to'quv (Meshgrid - Koinot zamini) yasaymiz
```

```
W_grid, B_grid = np.meshgrid(w_vals, b_vals)
```

```
J_grid = np.zeros_like(W_grid) # Z o'qi, Balandlik (Loss) xaritasi bo'm bo'sh
```

```
# Har bir w va b kombinatsiyasi uchun balandlikni hisoblaymiz (Kosaning shakli)
```

```
for i in range(len(w_vals)):
```

```
    for j in range(len(b_vals)):
```

```
        J_grid[i, j] = compute_cost(W_grid[i, j], B_grid[i, j], X, y)
```

```
# -----
```

```
# Chizma 1: COST (Xatolikning qulashi) chizig'i
```

```
# -----
```

```
plt.figure(figsize=(15, 5))
```

```
plt.subplot(1, 2, 1)
```

```
plt.plot(history_cost, color='red', linewidth=2)
```

```
plt.title("Xatolik (Loss) ning Epoxalar bo'yicha tushishi", fontsize=14)
```

```
plt.xlabel("Iteratsiyalar (Qadamlar)")
```

```
plt.ylabel("Cost (J)")
```

```
# -----
```

```
# Chizma 2: 2D Contour Plot (Osmon tepasidan qarab Model qanday yurgani)
```

```
# -----
```

```
plt.subplot(1, 2, 2)
```

```
# Kontur (Xarita halqalarini) chizamiz
```

```
cp = plt.contour(W_grid, B_grid, J_grid, levels=np.logspace(-1, 3, 20), cmap='viridis')
```

```
plt.colorbar(cp)
```

```
# Algoritmimiz bosib o'tgan izlarni qizil yulduzchalar qilib ustiga yopishtiramiz
```

```
plt.plot(history_w, history_b, marker='o', color='red', markersize=4, linestyle='dashed')
```

```
plt.plot(2, 0, marker='X', color='black', markersize=12) # Eng tubidagi Haqiqiy javob belgisi!
```

```
plt.title("Contour Xaritasi: Qorong'u vodiy bo'ylab sho'ng'ish", fontsize=14)
```

```

plt.xlabel("Weight (w)")
plt.ylabel("Bias (b)")
plt.show()

# -----
# Chizma 3: Haqiqiy 3D Sirtni Ko'ramiz (O'ta darajadagi Go'zallik)
# -----

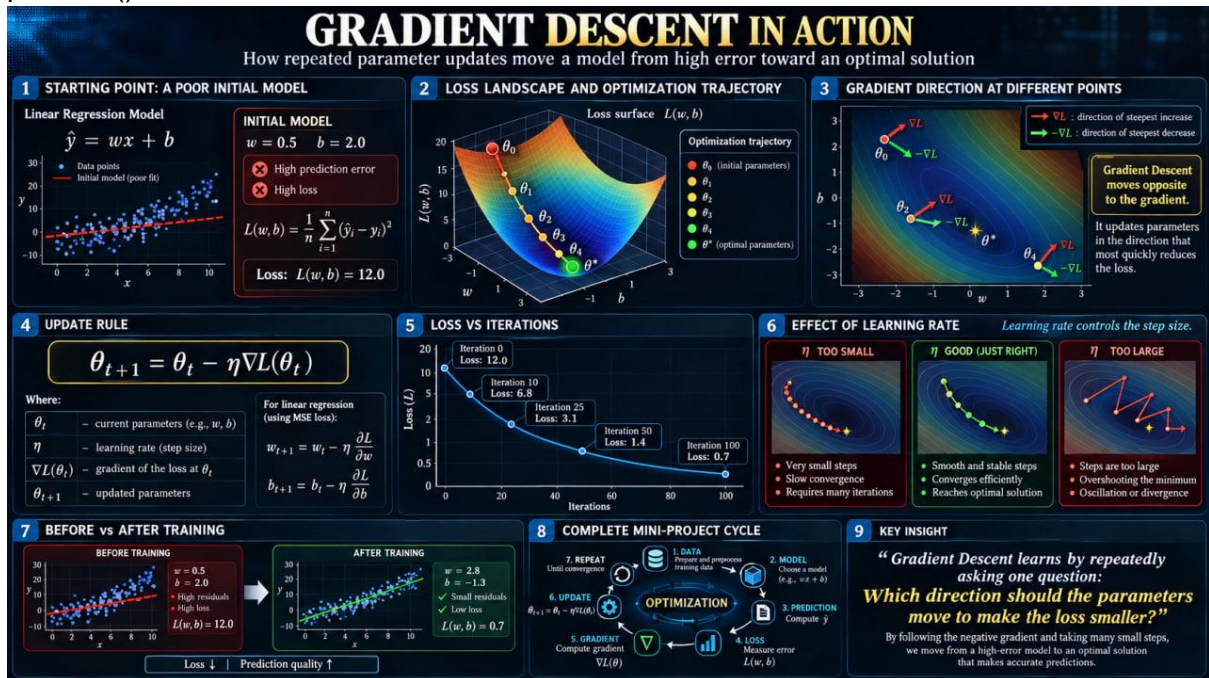
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')

# Kosani chizamiz
surf = ax.plot_surface(W_grid, B_grid, J_grid, cmap='viridis', alpha=0.6, edgecolor='none')

# Model qanday qulab (sirpanib) tushganini qizil o'q kabi sirtga tikamiz!
history_j = [compute_cost(w, b, X, y) for w, b in zip(history_w, history_b)]
ax.plot(history_w, history_b, history_j, color='red', marker='o', markersize=3, linewidth=2)

ax.set_title("3D Loss Sirti va Bizning Qulash Trayektoriyamiz", fontsize=16)
ax.set_xlabel("Weight (w)")
ax.set_ylabel("Bias (b)")
ax.set_zlabel("Cost (J)")
plt.show()

```



(Siz kompyuteringizda kodni ishga tushirganingizda sizni hayratga soluvchi 3ta grafik hosil bo'ladi. Birinchisi qanday qilib Xatolik J ning nolgacha tez tushganini ko'rsatsa, ikkinchi 2D va uchinchi 3D kosa grafiklar — xuddi koinotning egilgan matosida "qizil" sharchaning (modelning tasodifiy boshlang'ich $w=0, b=0$ nuqtasidan) qanday qilib Zanjir hosila va Minus Gradient yo'nalishi orqali shiddat bilan aylanib, markazga — Qop-qora Olmos Nishonga

($w=2$, $b=0$) qulab tushganini jonli (Izlari bilan) ko'rsatib beradi! Siz Optimallashtirishning fizikasini va Geometriyasini o'z ko'zingiz bilan kuzatasiz!)

Mantiqiy Yakun va Buyuk Xulosa (The Ultimate Conclusion)

Qadrli Data Scientist! Tabriklayman! Bugun siz faqat kod yozmadingiz. Bugun siz Koinot mexanikasi kabi ishlovchi Sun'iy Intellektning noldan-to-oxirigacha bo'lgan poydevorini (Engine) ni to'laqonli qurib bitkazdingiz.

Siz ko'rdingizki:

- Vektorlar va Matritsalar (X, y) bizning koinotdagi chiziqlarimiz va ma'lumotlar omborimiz ekan.
- Cost Function (MSE, Kosasimon paraboloid) — bu bizning xatolarimizni va ular baland-pastligini o'lchovchi adolat tarozisi ekan.
- **SymPy va Hosilalar (Calculus)** — Zanjir qoidasi orqali biz qanday ko'r joyda bo'lmaylik, osmonga (tikka o'sishga) qarab kompas chizib beradigan matematik tilsimot ekan.
- Va bularning hammasining bitta tugunga, bitta hal qiluvchi tsiklga, **Minus Gradient Descent (Backpropagation)** algoritmi orqali kompyuterning cheksiz yugurishi va koinotni tushuna boshlashi (O'rganish) jarayonini kashf qildik!

Bizning **Chiziqli Algebra va Matematik Analiz (Linear Algebra & Calculus)** ga bag'ishlangan yirik epik sayohatimiz o'zining yorqin marrasiga yetdi. Endi siz shunchaki Pythonda kutubxonalarni chaqiradigan dasturchi emassiz. Siz ularning "Qornida" millionlab hosilalar va vektorlar oqib o'tayotganini tasavvur qila oladigan Matematik-Arxitektorsiz.

Bu darslarni saqlab qo'ying, ulardagi tamoyillarni unutmang. Endi esa, yo'riqnomamizdagi keyingi Buyuk Bo'lim: Barcha sirlarni tayyor modellar qatoriga olib chiquvchi, Real Dataga tatbiq etiladigan **Classical Machine Learning (Klassik Mashinali O'rganish — Chiziqli regressiya, Logistik regressiya, SVM, Daraxtlar)** olamiga yorug' yuz bilan qadam tashlashimiz mumkin. Siz bu ilmni egallashga to'liq tayyorsiz!